

Chapter 3. Processes

The concept of a process is fundamental to any multiprogramming operating system. A process is usually defined as an instance of a program in execution; thus, if 16 users are running *vi* at once, there are 16 separate processes (although they can share the same executable code). Processes are often called *tasks* or *threads* in the Linux source code.

In this chapter, we discuss static properties of processes and then describe how process switching is performed by the kernel. The last two sections describe how processes can be created and destroyed. We also describe how Linux supports multithreaded applications — as mentioned in [Chapter 1](#), it relies on so-called lightweight processes (LWP).

Processes, Lightweight Processes, and Threads

The term "process" is often used with several different meanings. In this book, we stick to the usual OS textbook definition: a *process* is an instance of a program in execution. You might think of it as the collection of data structures that fully describes how far the execution of the program has progressed.

Processes are like human beings: they are generated, they have a more or less significant life, they optionally generate one or more child processes, and eventually they die. A small difference is that sex is not really common among processes — each process has just one parent.

From the kernel's point of view, the purpose of a process is to act as an entity to which system resources (CPU time, memory, etc.) are allocated.

When a process is created, it is almost identical to its parent. It receives a (logical) copy of the parent's address space and executes the same code as the parent, beginning at the next instruction following the process creation system call. Although the parent and child may share the pages containing the program code (text), they have separate copies of the data (stack and heap), so that changes by the child to a memory location are invisible to the parent (and vice versa).

While earlier Unix kernels employed this simple model, modern Unix systems do not. They support *multithreaded applications* — user programs having many relatively independent execution flows sharing a large portion of the application data structures. In such systems, a process is composed of several *user threads* (or simply *threads*), each of which represents an execution flow of the process. Nowadays, most multithreaded applications are written using standard sets of library functions called *pthread (POSIX thread) libraries*.

Older versions of the Linux kernel offered no support for multithreaded applications. From the kernel point of view, a multithreaded application was just a normal process. The multiple execution flows of a multithreaded application were created, handled, and scheduled entirely in User Mode, usually by means of a POSIX-compliant *pthread* library.

However, such an implementation of multithreaded applications is not very satisfactory. For instance, suppose a chess program uses two threads: one of them controls the graphical chessboard, waiting for the moves of the human player and showing the moves of the computer, while the other thread ponders the next move of

the game. While the first thread waits for the human move, the second thread should run continuously, thus exploiting the thinking time of the human player. However, if the chess program is just a single process, the first thread cannot simply issue a blocking system call waiting for a user action; otherwise, the second thread is blocked as well. Instead, the first thread must employ sophisticated nonblocking techniques to ensure that the process remains runnable.

Linux uses *lightweight processes* to offer better support for multithreaded applications. Basically, two lightweight processes may share some resources, like the address space, the open files, and so on. Whenever one of them modifies a shared resource, the other immediately sees the change. Of course, the two processes must synchronize themselves when accessing the shared resource.

A straightforward way to implement multithreaded applications is to associate a lightweight process with each thread. In this way, the threads can access the same set of application data structures by simply sharing the same memory address space, the same set of open files, and so on; at the same time, each thread can be scheduled independently by the kernel so that one may sleep while another remains runnable. Examples of POSIX-compliant *pthread* libraries that use Linux's lightweight processes are *LinuxThreads*, *Native POSIX Thread Library (NPTL)*, and IBM's *Next Generation Posix Threading Package (NGPT)*.

POSIX-compliant multithreaded applications are best handled by kernels that support "thread groups ." In Linux a *thread group* is basically a set of lightweight processes that implement a multithreaded application and act as a whole with regards to some system calls such as `getpid( )`, `kill( )`, and `_exit( )`. We are going to describe them at length later in this chapter.

Process Descriptor

To manage processes, the kernel must have a clear picture of what each process is doing. It must know, for instance, the process's priority, whether it is running on a CPU or blocked on an event, what address space has been assigned to it, which files it is allowed to address, and so on. This is the role of the *process descriptor* — a `task_struct` type structure whose fields contain all the information related to a single process.^[*] As the repository of so much information, the process descriptor is rather complex. In addition to a large number of fields containing process attributes, the process descriptor contains several pointers to other data structures that, in turn, contain pointers to other structures. [Figure 3-1](#) describes the Linux process descriptor schematically.

The six data structures on the right side of the figure refer to specific resources owned by the process. Most of these resources will be covered in future chapters. This chapter focuses on two types of fields that refer to the process state and to process parent/child relationships.

Process State

As its name implies, the `state` field of the process descriptor describes what is currently happening to the process. It consists of an array of flags, each of which describes a possible process state. In the current Linux version, these states are mutually exclusive, and hence exactly one flag of `state` always is set; the remaining flags are cleared. The following are the possible process states:

TASK_RUNNING

The process is either executing on a CPU or waiting to be executed.

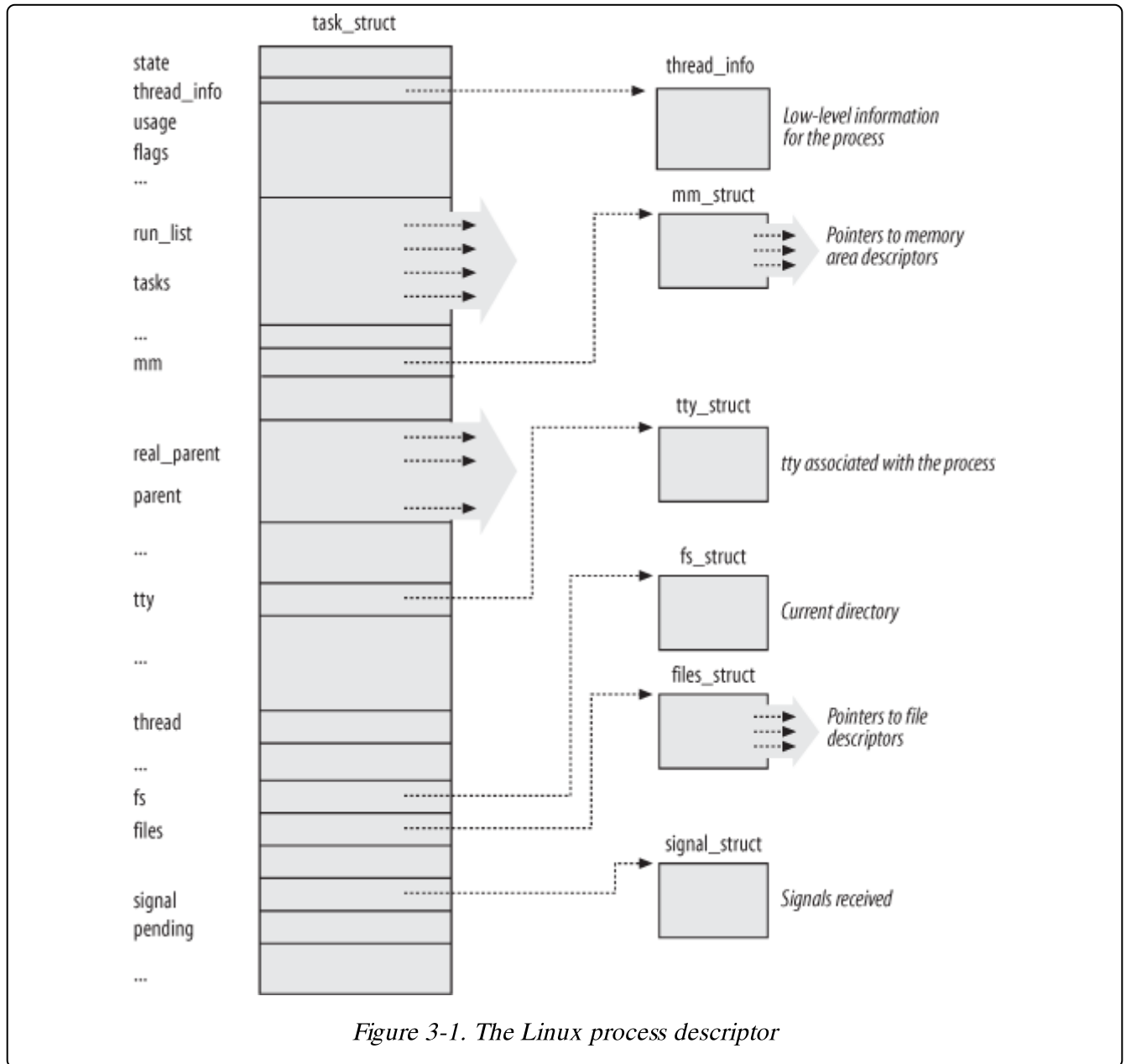
TASK_INTERRUPTIBLE

The process is suspended (sleeping) until some condition becomes true. Raising a hardware interrupt, releasing a system resource the process is waiting for, or delivering a signal are examples of conditions that might wake up the process (put its state back to `TASK_RUNNING`).

TASK_UNINTERRUPTIBLE

Like `TASK_INTERRUPTIBLE`, except that delivering a signal to the sleeping process leaves its state unchanged. This process state is seldom used. It is valuable, however, under certain specific conditions in which a process must wait until a

given event occurs without being interrupted. For instance, this state may be used when a process opens a device file and the corresponding device driver starts probing for a corresponding hardware device. The device driver must not be interrupted until the probing is complete, or the hardware device could be left in an unpredictable state.



TASK_STOPPED

Process execution has been stopped; the process enters this state after receiving a SIGSTOP, SIGTSTP, SIGTTIN, or SIGTTOU signal.

TASK_TRACED

Process execution has been stopped by a debugger. When a process is being monitored by another (such as when a debugger executes a `ptrace( )` system call to monitor a test program), each signal may put the process in the `TASK_TRACED` state.

Two additional states of the process can be stored both in the `state` field and in the `exit_state` field of the process descriptor; as the field name suggests, a process reaches one of these two states only when its execution is terminated:

EXIT_ZOMBIE

Process execution is terminated, but the parent process has not yet issued a `wait4( )` or `waitpid( )` system call to return information about the dead process.^[*]

Before the `wait( )`-like call is issued, the kernel cannot discard the data contained in the dead process descriptor because the parent might need it. (See the section "[Process Removal](#)" near the end of this chapter.)

EXIT_DEAD

The final state: the process is being removed by the system because the parent process has just issued a `wait4( )` or `waitpid( )` system call for it. Changing its state from `EXIT_ZOMBIE` to `EXIT_DEAD` avoids race conditions due to other threads of execution that execute `wait( )`-like calls on the same process (see [Chapter 5](#)).

The value of the `state` field is usually set with a simple assignment. For instance:

```
p->state = TASK_RUNNING;
```

The kernel also uses the `set_task_state` and `set_current_state` macros: they set the state of a specified process and of the process currently executed, respectively. Moreover, these macros ensure that the assignment operation is not mixed with other instructions by the compiler or the CPU control unit. Mixing the instruction order may sometimes lead to catastrophic results (see [Chapter 5](#)).

Identifying a Process

As a general rule, each execution context that can be independently scheduled must have its own process descriptor; therefore, even lightweight processes, which share a large portion of their kernel data structures, have their own `task_struct` structures.

The strict one-to-one correspondence between the process and process descriptor makes the 32-bit address^[†] of the `task_struct` structure a useful means for the kernel to identify processes. These addresses are referred to as *process descriptor pointers*.

Most of the references to processes that the kernel makes are through process descriptor pointers.

On the other hand, Unix-like operating systems allow users to identify processes by means of a number called the *Process ID* (or *PID*), which is stored in the `pid` field of the process descriptor. PIDs are numbered sequentially: the PID of a newly created process is normally the PID of the previously created process increased by one. Of course, there is an upper limit on the PID values; when the kernel reaches such limit, it must start recycling the lower, unused PIDs. By default, the maximum PID number is 32,767 (`PID_MAX_DEFAULT - 1`); the system administrator may reduce this limit by writing a smaller value into the `/proc /sys/kernel/pid_max` file (`/proc` is the mount point of a special filesystem, see the section "[Special Filesystems](#)" in [Chapter 12](#)). In 64-bit architectures, the system administrator can enlarge the maximum PID number up to 4,194,303.

When recycling PID numbers, the kernel must manage a `pidmap_array` bitmap that denotes which are the PIDs currently assigned and which are the free ones. Because a page frame contains 32,768 bits, in 32-bit architectures the `pidmap_array` bitmap is stored in a single page. In 64-bit architectures, however, additional pages can be added to the bitmap when the kernel assigns a PID number too large for the current bitmap size. These pages are never released.

Linux associates a different PID with each process or lightweight process in the system. (As we shall see later in this chapter, there is a tiny exception on multiprocessor systems.) This approach allows the maximum flexibility, because every execution context in the system can be uniquely identified.

On the other hand, Unix programmers expect threads in the same group to have a common PID. For instance, it should be possible to send a signal specifying a PID that affects all threads in the group. In fact, the POSIX 1003.1c standard states that all threads of a multithreaded application must have the same PID.

To comply with this standard, Linux makes use of thread groups. The identifier shared by the threads is the PID of the thread group leader, that is, the PID of the first lightweight process in the group; it is stored in the `tgid` field of the process descriptors. The `getpid( )` system call returns the value of `tgid` relative to the current process instead of the value of `pid`, so all the threads of a multithreaded application share the same identifier. Most processes belong to a thread group consisting of a single member; as thread group leaders, they have the `tgid` field equal

to the `pid` field, thus the `getpid( )` system call works as usual for this kind of process.

Later, we'll show you how it is possible to derive a true process descriptor pointer efficiently from its respective PID. Efficiency is important because many system calls such as `kill( )` use the PID to denote the affected process.

Process descriptors handling

Processes are dynamic entities whose lifetimes range from a few milliseconds to months. Thus, the kernel must be able to handle many processes at the same time, and process descriptors are stored in dynamic memory rather than in the memory area permanently assigned to the kernel. For each process, Linux packs two different data structures in a single per-process memory area: a small data structure linked to the process descriptor, namely the `thread_info` structure, and the Kernel Mode process stack. The length of this memory area is usually 8,192 bytes (two page frames). For reasons of efficiency the kernel stores the 8-KB memory area in two consecutive page frames with the first page frame aligned to a multiple of 2^{13} ; this may turn out to be a problem when little dynamic memory is available, because the free memory may become highly fragmented (see the section "[The Buddy System Algorithm](#)" in [Chapter 8](#)). Therefore, in the 80×86 architecture the kernel can be configured at compilation time so that the memory area including stack and `thread_info` structure spans a single page frame (4,096 bytes).

In the section "[Segmentation in Linux](#)" in [Chapter 2](#), we learned that a process in Kernel Mode accesses a stack contained in the kernel data segment, which is different from the stack used by the process in User Mode. Because kernel control paths make little use of the stack, only a few thousand bytes of kernel stack are required. Therefore, 8 KB is ample space for the stack and the `thread_info` structure. However, when stack and `thread_info` structure are contained in a single page frame, the kernel uses a few additional stacks to avoid the overflows caused by deeply nested interrupts and exceptions (see [Chapter 4](#)).

[Figure 3-2](#) shows how the two data structures are stored in the 2-page (8 KB) memory area. The `thread_info` structure resides at the beginning of the memory area, and the stack grows downward from the end. The figure also shows that the `thread_info` structure and the `task_struct` structure are mutually linked by means of the fields `task` and `thread_info`, respectively.

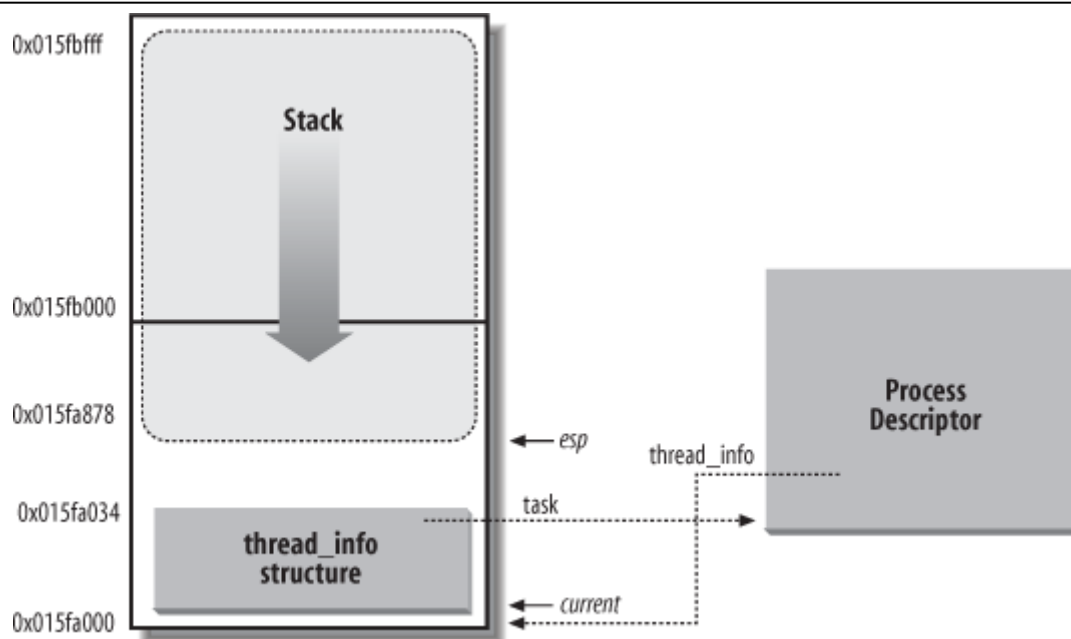


Figure 3-2. Storing the `thread_info` structure and the process kernel stack in two page frames

The `esp` register is the CPU stack pointer, which is used to address the stack's top location. On 80x86 systems, the stack starts at the end and grows toward the beginning of the memory area. Right after switching from User Mode to Kernel Mode, the kernel stack of a process is always empty, and therefore the `esp` register points to the byte immediately following the stack.

The value of the `esp` is decreased as soon as data is written into the stack. Because the `thread_info` structure is 52 bytes long, the kernel stack can expand up to 8,140 bytes.

The C language allows the `thread_info` structure and the kernel stack of a process to be conveniently represented by means of the following union construct:

```
union thread_union {
    struct thread_info thread_info;
    unsigned long stack[2048]; /* 1024 for 4KB stacks */
};
```

The `thread_info` structure shown in [Figure 3-2](#) is stored starting at address 0x015fa000, and the stack is stored starting at address 0x015fc000. The value of the `esp` register points to the current top of the stack at 0x015fa878.

The kernel uses the `alloc_thread_info` and `free_thread_info` macros to allocate and release the memory area storing a `thread_info` structure and a kernel stack.

Identifying the current process

The close association between the `thread_info` structure and the Kernel Mode stack just described offers a key benefit in terms of efficiency: the kernel can easily obtain the address of the `thread_info` structure of the process currently running on a CPU from the value of the `esp` register. In fact, if the `thread_union` structure is 8 KB (2^{13} bytes) long, the kernel masks out the 13 least significant bits of `esp` to obtain the base address of the `thread_info` structure; on the other hand, if the `thread_union` structure is 4 KB long, the kernel masks out the 12 least significant bits of `esp`. This is done by the `current_thread_info( )` function, which produces assembly language instructions like the following:

```
movl $0xfffffe000,%ecx /* or 0xfffff000 for 4KB stacks */
andl %esp,%ecx
movl %ecx,p
```

After executing these three instructions, `p` contains the `thread_info` structure pointer of the process running on the CPU that executes the instruction.

Most often the kernel needs the address of the process descriptor rather than the address of the `thread_info` structure. To get the process descriptor pointer of the process currently running on a CPU, the kernel makes use of the `current` macro, which is essentially equivalent to `current_thread_info( )->task` and produces assembly language instructions like the following:

```
movl $0xfffffe000,%ecx /* or 0xfffff000 for 4KB stacks */
andl %esp,%ecx
movl (%ecx),p
```

Because the `task` field is at offset 0 in the `thread_info` structure, after executing these three instructions `p` contains the process descriptor pointer of the process running on the CPU.

The `current` macro often appears in kernel code as a prefix to fields of the process descriptor. For example, `current->pid` returns the process ID of the process currently running on the CPU.

Another advantage of storing the process descriptor with the stack emerges on multiprocessor systems: the correct current process for each hardware processor can be derived just by checking the stack, as shown previously. Earlier versions of Linux did not store the kernel stack and the process descriptor together. Instead, they were forced to introduce a global static variable called `current` to identify the process descriptor of the running process. On multiprocessor systems, it was necessary to define `current` as an array—one element for each available CPU.

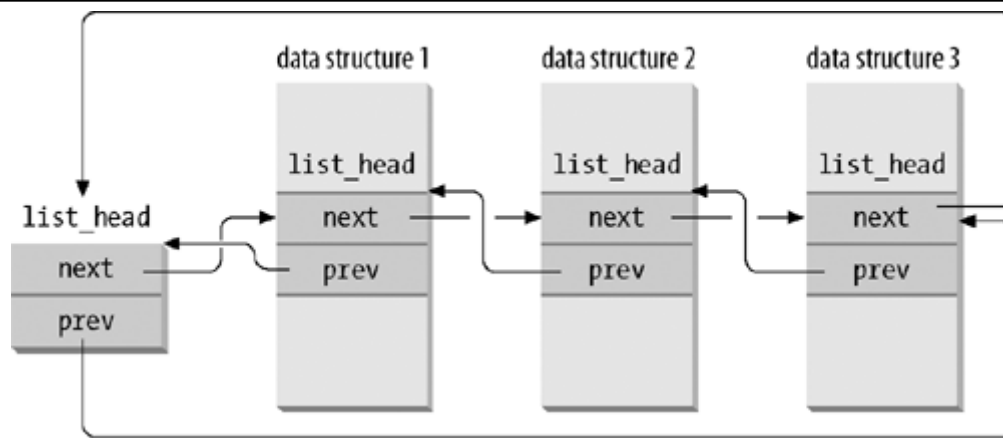
Doubly linked lists

Before moving on and describing how the kernel keeps track of the various processes in the system, we would like to emphasize the role of special data structures that implement doubly linked lists.

For each list, a set of primitive operations must be implemented: initializing the list, inserting and deleting an element, scanning the list, and so on. It would be both a waste of programmers' efforts and a waste of memory to replicate the primitive operations for each different list.

Therefore, the Linux kernel defines the `list_head` data structure, whose only fields `next` and `prev` represent the forward and back pointers of a generic doubly linked list element, respectively. It is important to note, however, that the pointers in a `list_head` field store the addresses of other `list_head` fields rather than the addresses of the whole data structures in which the `list_head` structure is included; see [Figure 3-3](#) (a).

A new list is created by using the `LIST_HEAD(list_name)` macro. It declares a new variable named `list_name` of type `list_head`, which is a dummy first element that acts as a placeholder for the head of the new list, and initializes the `prev` and `next` fields of the `list_head` data structure so as to point to the `list_name` variable itself; see [Figure 3-3](#) (b).



(a) a doubly linked list with three elements



(b) an empty doubly linked list

Figure 3-3. Doubly linked lists built with *list_head* data structures

Several functions and macros implement the primitives, including those shown in Table [Table 3-1](#).

Table 3-1. List handling functions and macros

Name	Description
<code>list_add(n,p)</code>	Inserts an element pointed to by <code>n</code> right after the specified element pointed to by <code>p</code> . (To insert <code>n</code> at the beginning of the list, set <code>p</code> to the address of the list head.)
<code>list_add_tail(n,p)</code>	Inserts an element pointed to by <code>n</code> right before the specified element pointed to by <code>p</code> . (To insert <code>n</code> at the end of the list, set <code>p</code> to the address of the list head.)
<code>list_del(p)</code>	Deletes an element pointed to by <code>p</code> . (There is no need to specify the head of the list.)
<code>list_empty(p)</code>	Checks if the list specified by the address <code>p</code> of its head is empty.
<code>list_entry(p,t,m)</code>	Returns the address of the data structure of type <code>t</code> in which the <code>list_head</code> field that has the name <code>m</code> and the address <code>p</code> is included.
<code>list_for_each(p,h)</code>	Scans the elements of the list specified by the address <code>h</code> of the head; in each iteration, a pointer to the <code>list_head</code> structure of the list element is returned in <code>p</code> .
<code>list_for_each_entry(p,h,m)</code>	Similar to <code>list_for_each</code> , but returns the address of the data structure embedding the <code>list_head</code> structure rather than the address of the <code>list_head</code> structure itself.

The Linux kernel 2.6 sports another kind of doubly linked list, which mainly differs from a `list_head` list because it is not circular; it is mainly used for hash tables, where space is important, and finding the the last element in constant time is not. The list head is stored in an `hlist_head` data structure, which is simply a pointer to the first element in the list (NULL if the list is empty). Each element is represented by an `hlist_node` data structure, which includes a pointer `next` to the next element, and a pointer `pprev` to the `next` field of the previous element. Because the list is not circular, the `pprev` field of the first element and the `next` field of the last element are set to NULL. The list can be handled by means of several helper functions and macros similar to those listed in [Table 3-1](#): `hlist_add_head( )`, `hlist_del( )`, `hlist_empty( )`, `hlist_entry`, `hlist_for_each_entry`, and so on.

The process list

The first example of a doubly linked list we will examine is the *process list*, a list that links together all existing process descriptors. Each `task_struct` structure includes a

tasks field of type `list_head` whose `prev` and `next` fields point, respectively, to the previous and to the next `task_struct` element.

The head of the process list is the `init_task` `task_struct` descriptor; it is the process descriptor of the so-called *process 0* or *swapper* (see the section "[Kernel Threads](#)" later in this chapter). The `tasks->prev` field of `init_task` points to the `tasks` field of the process descriptor inserted last in the list.

The `SET_LINKS` and `REMOVE_LINKS` macros are used to insert and to remove a process descriptor in the process list, respectively. These macros also take care of the parenthood relationship of the process (see the section "[How Processes Are Organized](#)" later in this chapter).

Another useful macro, called `for_each_process`, scans the whole process list. It is defined as:

```
#define for_each_process(p) \
    for (p=&init_task; (p=list_entry((p)->tasks.next, \
                                     struct task_struct, tasks) \
                                     ) != &init_task; )
```

The macro is the loop control statement after which the kernel programmer supplies the loop. Notice how the `init_task` process descriptor just plays the role of list header. The macro starts by moving past `init_task` to the next task and continues until it reaches `init_task` again (thanks to the circularity of the list). At each iteration, the variable passed as the argument of the macro contains the address of the currently scanned process descriptor, as returned by the `list_entry` macro.

The lists of **TASK_RUNNING** processes

When looking for a new process to run on a CPU, the kernel has to consider only the runnable processes (that is, the processes in the `TASK_RUNNING` state).

Earlier Linux versions put all runnable processes in the same list called *runqueue*. Because it would be too costly to maintain the list ordered according to process priorities, the earlier schedulers were compelled to scan the whole list in order to select the "best" runnable process.

Linux 2.6 implements the *runqueue* differently. The aim is to allow the scheduler to select the best runnable process in constant time, independently of the number of runnable processes. We'll defer to [Chapter 7](#) a detailed description of this new kind of *runqueue*, and we'll provide here only some basic information.

The trick used to achieve the scheduler speedup consists of splitting the runqueue in many lists of runnable processes, one list per process priority. Each `task_struct` descriptor includes a `run_list` field of type `list_head`. If the process priority is equal to `k` (a value ranging between 0 and 139), the `run_list` field links the process descriptor into the list of runnable processes having priority `k`. Furthermore, on a multiprocessor system, each CPU has its own runqueue, that is, its own set of lists of processes. This is a classic example of making a data structures more complex to improve performance: to make scheduler operations more efficient, the runqueue list has been split into 140 different lists!

As we'll see, the kernel must preserve a lot of data for every runqueue in the system; however, the main data structures of a runqueue are the lists of process descriptors belonging to the runqueue; all these lists are implemented by a single `prio_array_t` data structure, whose fields are shown in [Table 3-2](#).

Table 3-2. The fields of the `prio_array_t` data structure

Type	Field	Description
int	nr_active	The number of process descriptors linked into the lists
unsigned long [5]	bitmap	A priority bitmap: each flag is set if and only if the corresponding priority list is not empty
struct list_head [140]	queue	The 140 heads of the priority lists

The `enqueue_task(p,array)` function inserts a process descriptor into a runqueue list; its code is essentially equivalent to:

```
list_add_tail(&p->run_list, &array->queue[p->prio]);
__set_bit(p->prio, array->bitmap);
array->nr_active++;
p->array = array;
```

The `prio` field of the process descriptor stores the dynamic priority of the process, while the `array` field is a pointer to the `prio_array_t` data structure of its current runqueue. Similarly, the `dequeue_task(p,array)` function removes a process descriptor from a runqueue list.

Relationships Among Processes

Processes created by a program have a parent/child relationship. When a process creates multiple children , these children have sibling relationships. Several fields must

be introduced in a process descriptor to represent these relationships; they are listed in [Table 3-3](#) with respect to a given process P. Processes 0 and 1 are created by the kernel; as we'll see later in the chapter, process 1 (*init*) is the ancestor of all other processes.

Table 3-3. Fields of a process descriptor used to express parenthood relationships

Field name	Description
<code>real_parent</code>	Points to the process descriptor of the process that created P or to the descriptor of process 1 (<i>init</i>) if the parent process no longer exists. (Therefore, when a user starts a background process and exits the shell, the background process becomes the child of <i>init</i> .)
<code>parent</code>	Points to the current parent of P (this is the process that must be signaled when the child process terminates); its value usually coincides with that of <code>real_parent</code> . It may occasionally differ, such as when another process issues a <code>ptrace()</code> system call requesting that it be allowed to monitor P (see the section " Execution Tracing " in Chapter 20).
<code>children</code>	The head of the list containing all children created by P.
<code>sibling</code>	The pointers to the next and previous elements in the list of the sibling processes, those that have the same parent as P.

[Figure 3-4](#) illustrates the parent and sibling relationships of a group of processes. Process P0 successively created P1, P2, and P3. Process P3, in turn, created process P4.

Furthermore, there exist other relationships among processes: a process can be a leader of a process group or of a login session (see "[Process Management](#)" in [Chapter 1](#)), it can be a leader of a thread group (see "[Identifying a Process](#)" earlier in this chapter), and it can also trace the execution of other processes (see the section "[Execution Tracing](#)" in [Chapter 20](#)). [Table 3-4](#) lists the fields of the process descriptor that establish these relationships between a process P and the other processes.

Table 3-4. The fields of the process descriptor that establish non-parenthood relationships

Field name	Description
group_leader	Process descriptor pointer of the group leader of P
signal->pgrp	PID of the group leader of P
tgid	PID of the thread group leader of P
signal->session	PID of the login session leader of P
ptrace_children	The head of a list containing all children of P being traced by a debugger
ptrace_list	The pointers to the next and previous elements in the real parent's list of traced processes (used when P is being traced)

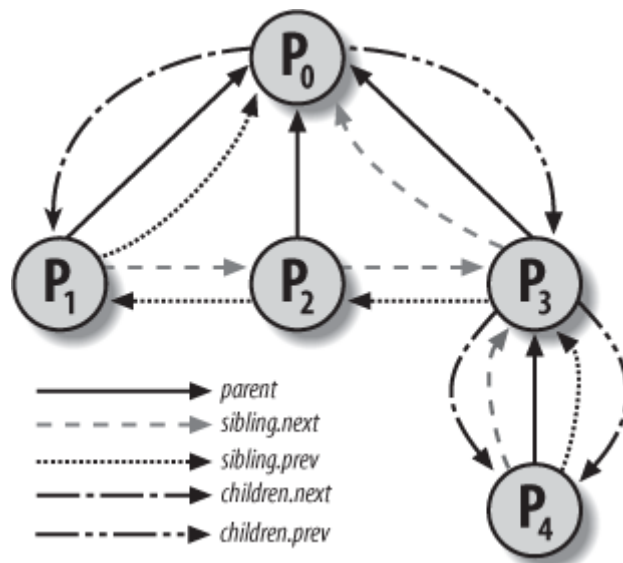


Figure 3-4. Parenthood relationships among five processes

The pidhash table and chained lists

In several circumstances, the kernel must be able to derive the process descriptor pointer corresponding to a PID. This occurs, for instance, in servicing the `kill( )` system call. When process P₁ wishes to send a signal to another process, P₂, it invokes the `kill( )` system call specifying the PID of P₂ as the parameter. The kernel derives the process descriptor pointer from the PID and then extracts the pointer to the data structure that records the pending signals from P₂'s process descriptor.

Scanning the process list sequentially and checking the `pid` fields of the process descriptors is feasible but rather inefficient. To speed up the search, four hash tables

have been introduced. Why multiple hash tables? Simply because the process descriptor includes fields that represent different types of PID (see [Table 3-5](#)), and each type of PID requires its own hash table.

Table 3-5. The four hash tables and corresponding fields in the process descriptor

Hash table type	Field name	Description
PIDTYPE_PID	pid	PID of the process
PIDTYPE_TGID	tgid	PID of thread group leader process
PIDTYPE_PGID	pgrp	PID of the group leader process
PIDTYPE_SID	session	PID of the session leader process

The four hash tables are dynamically allocated during the kernel initialization phase, and their addresses are stored in the `pid_hash` array. The size of a single hash table depends on the amount of available RAM; for example, for systems having 512 MB of RAM, each hash table is stored in four page frames and includes 2,048 entries.

The PID is transformed into a table index using the `pid_hashfn` macro, which expands to:

```
#define pid_hashfn(x) hash_long((unsigned long) x, pidhash_shift)
```

The `pidhash_shift` variable stores the length in bits of a table index (11, in our example). The `hash_long( )` function is used by many hash functions; on a 32-bit architecture it is essentially equivalent to:

```
unsigned long hash_long(unsigned long val, unsigned int bits)
{
    unsigned long hash = val * 0x9e370001UL;
    return hash >> (32 - bits);
}
```

Because in our example `pidhash_shift` is equal to 11, `pid_hashfn` yields values ranging between 0 and $2^{11} - 1 = 2047$.

THE MAGIC CONSTANT

You might wonder where the 0x9e370001 constant (= 2,654,404,609) comes from. This hash function is based on a multiplication of the index by a suitable large number, so that the result overflows and the value remaining in the 32-bit variable can be considered as the result of a modulus operation. Knuth suggested that good results are obtained when the large multiplier is a prime approximately in golden ratio to 2^{32} (32 bit being the size of the 80x86's registers). Now, 2,654,404,609 is a prime near to $2^{32} \times (\sqrt{5}-1)/2$ that can also be easily multiplied by additions and bit shifts, because it is equal to $2^{31} + 2^{29} - 2^{25} + 2^{22} - 2^{19} - 2^{16} + 1$.

As every basic computer science course explains, a hash function does not always ensure a one-to-one correspondence between PIDs and table indexes. Two different PIDs that hash into the same table index are said to be *colliding*.

Linux uses *chaining* to handle colliding PIDs; each table entry is the head of a doubly linked list of colliding process descriptors. [Figure 3-5](#) illustrates a PID hash table with two lists. The processes having PIDs 2,890 and 29,384 hash into the 200th element of the table, while the process having PID 29,385 hashes into the 1,466th element of the table.

Hashing with chaining is preferable to a linear transformation from PIDs to table indexes because at any given instance, the number of processes in the system is usually far below 32,768 (the maximum number of allowed PIDs). It would be a waste of storage to define a table consisting of 32,768 entries, if, at any given instance, most such entries are unused.

The data structures used in the PID hash tables are quite sophisticated, because they must keep track of the relationships between the processes. As an example, suppose that the kernel must retrieve all processes belonging to a given thread group, that is, all processes whose `tgid` field is equal to a given number. Looking in the hash table for the given thread group number returns just one process descriptor, that is, the descriptor of the thread group leader. To quickly retrieve the other processes in the group, the kernel must maintain a list of processes for each thread group. The same situation arises when looking for the processes belonging to a given login session or belonging to a given process group.

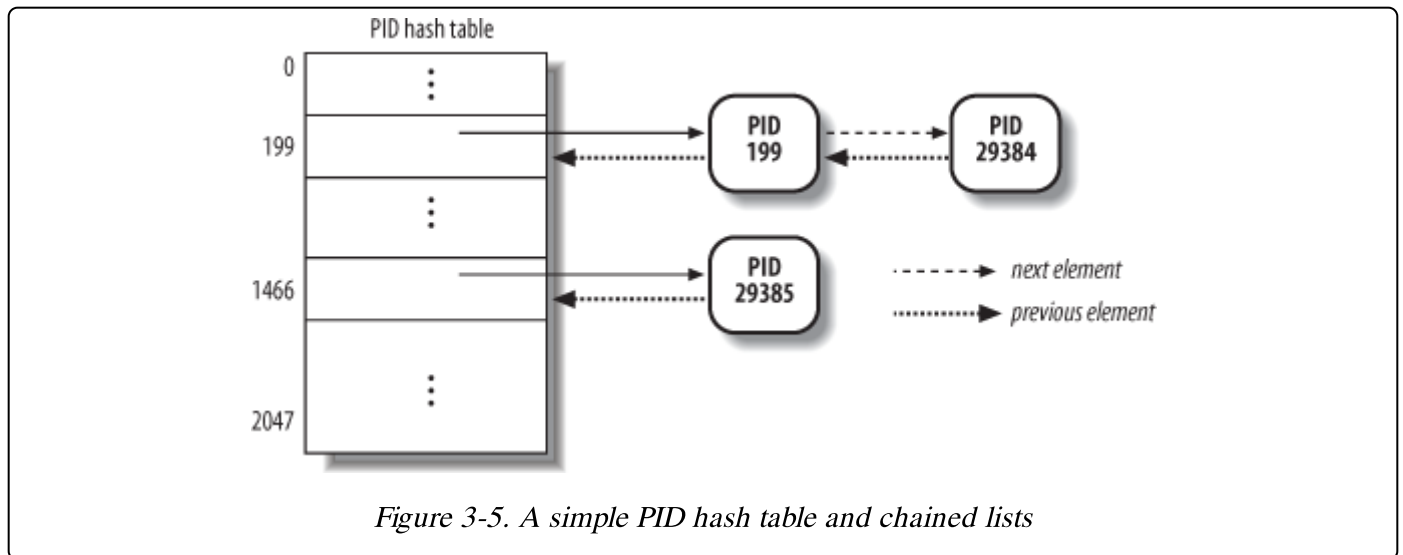


Figure 3-5. A simple PID hash table and chained lists

The PID hash tables' data structures solve all these problems, because they allow the definition of a list of processes for any PID number included in a hash table. The core data structure is an array of four `pid` structures embedded in the `pids` field of the process descriptor; the fields of the `pid` structure are shown in [Table 3-6](#).

Table 3-6. The fields of the `pid` data structures

Type	Name	Description
int	nr	The PID number
struct hlist_node	pid_chain	The links to the next and previous elements in the hash chain list
struct list_head	pid_list	The head of the per-PID list

[Figure 3-6](#) shows an example based on the `PIDTYPE_TGID` hash table. The second entry of the `pid_hash` array stores the address of the hash table, that is, the array of `hlist_head` structures representing the heads of the chain lists. In the chain list rooted at the 71st entry of the hash table, there are two process descriptors corresponding to the PID numbers 246 and 4,351 (double-arrow lines represent a couple of forward and backward pointers). The PID numbers are stored in the `nr` field of the `pid` structure embedded in the process descriptor (by the way, because the thread group number coincides with the PID of its leader, these numbers also are stored in the `pid` field of the process descriptors). Let us consider the per-PID list of the thread group 4,351: the head of the list is stored in the `pid_list` field of the process descriptor included in the hash table, while the links to the next and previous elements of the per-PID list also are stored in the `pid_list` field of each list element.

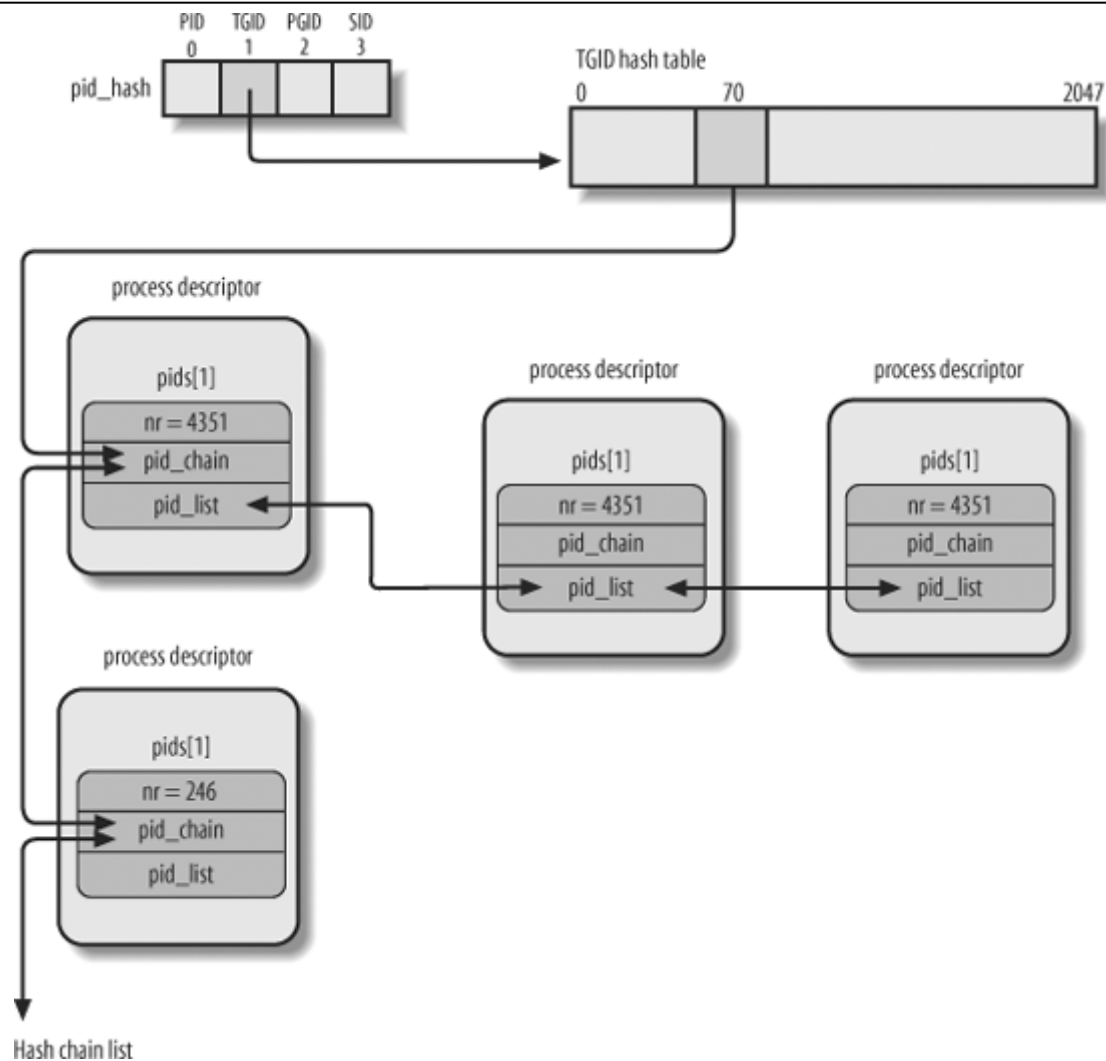


Figure 3-6. The PID hash tables

The following functions and macros are used to handle the PID hash tables:

`do_each_task_pid(nr, type, task)`

`while_each_task_pid(nr, type, task)`

Mark begin and end of a do-while loop that iterates over the per-PID list associated with the PID number `nr` of type `type`; in any iteration, `task` points to the process descriptor of the currently scanned element.

`find_task_by_pid_type(type, nr)`

Looks for the process having PID `nr` in the hash table of type `type`. The function returns a process descriptor pointer if a match is found, otherwise it returns `NULL`.

`find_task_by_pid(nr)`

Same as `find_task_by_pid_type(PIDTYPE_PID, nr)`.

`attach_pid(task, type, nr)`

Inserts the process descriptor pointed to by `task` in the PID hash table of type `type` according to the PID number `nr`; if a process descriptor having PID `nr` is already in the hash table, the function simply inserts `task` in the per-PID list of the already present process.

`detach_pid(task, type)`

Removes the process descriptor pointed to by `task` from the per-PID list of type `type` to which the descriptor belongs. If the per-PID list does not become empty, the function terminates. Otherwise, the function removes the process descriptor from the hash table of type `type`; finally, if the PID number does not occur in any other hash table, the function clears the corresponding bit in the PID bitmap, so that the number can be recycled.

`next_thread(task)`

Returns the process descriptor address of the lightweight process that follows `task` in the hash table list of type `PIDTYPE_TGID`. Because the hash table list is circular, when applied to a conventional process the macro returns the descriptor address of the process itself.

How Processes Are Organized

The runqueue lists group all processes in a `TASK_RUNNING` state. When it comes to grouping processes in other states, the various states call for different types of treatment, with Linux opting for one of the choices shown in the following list.

- Processes in a `TASK_STOPPED`, `EXIT_ZOMBIE`, or `EXIT_DEAD` state are not linked in specific lists. There is no need to group processes in any of these three states, because stopped, zombie, and dead processes are accessed only via PID or via linked lists of the child processes for a particular parent.
- Processes in a `TASK_INTERRUPTIBLE` or `TASK_UNINTERRUPTIBLE` state are subdivided into many classes, each of which corresponds to a specific event. In this case, the process state does not provide enough information to retrieve the process quickly, so it is necessary to introduce additional lists of processes. These are called *wait queues* and are discussed next.

Wait queues

Wait queues have several uses in the kernel, particularly for interrupt handling, process synchronization, and timing. Because these topics are discussed in later chapters, we'll just say here that a process must often wait for some event to occur, such as for a disk operation to terminate, a system resource to be released, or a fixed interval of time to elapse. Wait queues implement conditional waits on events: a process wishing to wait for a specific event places itself in the proper wait queue and relinquishes control. Therefore, a wait queue represents a set of sleeping processes, which are woken up by the kernel when some condition becomes true.

Wait queues are implemented as doubly linked lists whose elements include pointers to process descriptors. Each wait queue is identified by a *wait queue head*, a data structure of type `wait_queue_head_t`:

```
struct __wait_queue_head {
    spinlock_t lock;
    struct list_head task_list;
};
typedef struct __wait_queue_head wait_queue_head_t;
```

Because wait queues are modified by interrupt handlers as well as by major kernel functions, the doubly linked lists must be protected from concurrent accesses, which could induce unpredictable results (see [Chapter 5](#)). Synchronization is achieved by the `lock` spin lock in the wait queue head. The `task_list` field is the head of the list of waiting processes.

Elements of a wait queue list are of type `wait_queue_t`:

```
struct __wait_queue {
    unsigned int flags;
    struct task_struct * task;
    wait_queue_func_t func;
    struct list_head task_list;
};
typedef struct __wait_queue wait_queue_t;
```

Each element in the wait queue list represents a sleeping process, which is waiting for some event to occur; its descriptor address is stored in the `task` field. The `task_list` field contains the pointers that link this element to the list of processes waiting for the same event.

However, it is not always convenient to wake up *all* sleeping processes in a wait queue. For instance, if two or more processes are waiting for exclusive access to some resource to be released, it makes sense to wake up just one process in the wait queue. This process takes the resource, while the other processes continue to sleep. (This avoids a problem known as the "thundering herd," with which multiple processes are

wakened only to race for a resource that can be accessed by one of them, with the result that remaining processes must once more be put back to sleep.)

Thus, there are two kinds of sleeping processes: *exclusive processes* (denoted by the value 1 in the `flags` field of the corresponding wait queue element) are selectively woken up by the kernel, while *nonexclusive processes* (denoted by the value 0 in the `flags` field) are always woken up by the kernel when the event occurs. A process waiting for a resource that can be granted to just one process at a time is a typical exclusive process. Processes waiting for an event that may concern any of them are nonexclusive. Consider, for instance, a group of processes that are waiting for the termination of a group of disk block transfers: as soon as the transfers complete, all waiting processes must be woken up. As we'll see next, the `func` field of a wait queue element is used to specify how the processes sleeping in the wait queue should be woken up.

Handling wait queues

A new wait queue head may be defined by using the `DECLARE_WAIT_QUEUE_HEAD(name)` macro, which statically declares a new wait queue head variable called `name` and initializes its `lock` and `task_list` fields. The `init_waitqueue_head( )` function may be used to initialize a wait queue head variable that was allocated dynamically.

The `init_waitqueue_entry(q,p )` function initializes a `wait_queue_t` structure `q` as follows:

```
q->flags = 0;
q->task = p;
q->func = default_wake_function;
```

The nonexclusive process `p` will be awakened by `default_wake_function( )`, which is a simple wrapper for the `try_to_wake_up( )` function discussed in [Chapter 7](#).

Alternatively, the `DEFINE_WAIT` macro declares a new `wait_queue_t` variable and initializes it with the descriptor of the process currently executing on the CPU and the address of the `autoremove_wake_function( )` wake-up function. This function invokes `default_wake_function( )` to awaken the sleeping process, and then removes the wait queue element from the wait queue list. Finally, a kernel developer can define a custom awakening function by initializing the wait queue element with the `init_waitqueue_func_entry( )` function.

Once an element is defined, it must be inserted into a wait queue. The `add_wait_queue( )` function inserts a nonexclusive process in the first position of a wait queue list. The `add_wait_queue_exclusive( )` function inserts an exclusive process in the last position of a wait queue list. The `remove_wait_queue( )` function removes a process from a wait queue list. The `waitqueue_active( )` function checks whether a given wait queue list is empty.

A process wishing to wait for a specific condition can invoke any of the functions shown in the following list.

- The `sleep_on( )` function operates on the current process:

```
void sleep_on(wait_queue_head_t *wq)
{
    wait_queue_t wait;
    init_waitqueue_entry(&wait, current);
    current->state = TASK_UNINTERRUPTIBLE;
    add_wait_queue(wq,&wait); /* wq points to the wait queue head */
    schedule( );
    remove_wait_queue(wq, &wait);
}
```

The function sets the state of the current process to `TASK_UNINTERRUPTIBLE` and inserts it into the specified wait queue. Then it invokes the scheduler, which resumes the execution of another process. When the sleeping process is awakened, the scheduler resumes execution of the `sleep_on( )` function, which removes the process from the wait queue.

- The `interruptible_sleep_on( )` function is identical to `sleep_on( )`, except that it sets the state of the current process to `TASK_INTERRUPTIBLE` instead of setting it to `TASK_UNINTERRUPTIBLE`, so that the process also can be woken up by receiving a signal.
- The `sleep_on_timeout( )` and `interruptible_sleep_on_timeout( )` functions are similar to the previous ones, but they also allow the caller to define a time interval after which the process will be woken up by the kernel. To do this, they invoke the `schedule_timeout( )` function instead of `schedule( )` (see the section "[An Application of Dynamic Timers: the nanosleep\(.\) System Call](#)" in [Chapter 6](#)).
- The `prepare_to_wait( )`, `prepare_to_wait_exclusive( )`, and `finish_wait( )` functions, introduced in Linux 2.6, offer yet another way to put the current process to sleep in a wait queue. Typically, they are used as follows:

```

DEFINE_WAIT(wait);
prepare_to_wait_exclusive(&wq, &wait, TASK_INTERRUPTIBLE);
/* wq is the head of the wait queue */

...
if (!condition)
    schedule();
finish_wait(&wq, &wait);

```

The `prepare_to_wait( )` and `prepare_to_wait_exclusive( )` functions set the process state to the value passed as the third parameter, then set the exclusive flag in the wait queue element respectively to 0 (nonexclusive) or 1 (exclusive), and finally insert the wait queue element `wait` into the list of the wait queue head `wq`.

As soon as the process is awakened, it executes the `finish_wait( )` function, which sets again the process state to `TASK_RUNNING` (just in case the awaking condition becomes true before invoking `schedule( )`), and removes the wait queue element from the wait queue list (unless this has already been done by the wake-up function).

- The `wait_event` and `wait_event_interruptible` macros put the calling process to sleep on a wait queue until a given condition is verified. For instance, the `wait_event(wq,condition)` macro essentially yields the following fragment:

```

DEFINE_WAIT(_ _wait);
for (;;) {
    prepare_to_wait(&wq, &_ _wait, TASK_UNINTERRUPTIBLE);
    if (condition)
        break;
    schedule( );
}
finish_wait(&wq, &_ _wait);

```

A few comments on the functions mentioned in the above list: the `sleep_on( )`-like functions cannot be used in the common situation where one has to test a condition and atomically put the process to sleep when the condition is not verified; therefore, because they are a well-known source of race conditions, their use is discouraged. Moreover, in order to insert an exclusive process into a wait queue, the kernel must make use of the `prepare_to_wait_exclusive( )` function (or just invoke `add_wait_queue_exclusive( )` directly); any other helper function inserts the process as nonexclusive. Finally, unless `DEFINE_WAIT` or `finish_wait( )` are used, the kernel must remove the wait queue element from the list after the waiting process has been awakened.

The kernel awakens processes in the wait queues, putting them in the `TASK_RUNNING` state, by means of one of the following macros: `wake_up`, `wake_up_nr`, `wake_up_all`,

`wake_up_interruptible`, `wake_up_interruptible_nr`, `wake_up_interruptible_all`, `wake_up_interruptible_sync`, and `wake_up_locked`. One can understand what each of these nine macros does from its name:

- All macros take into consideration sleeping processes in the `TASK_INTERRUPTIBLE` state; if the macro name does not include the string "interruptible," sleeping processes in the `TASK_UNINTERRUPTIBLE` state also are considered.
- All macros wake all nonexclusive processes having the required state (see the previous bullet item).
- The macros whose name include the string "nr" wake a given number of exclusive processes having the required state; this number is a parameter of the macro. The macros whose names include the string "all" wake all exclusive processes having the required state. Finally, the macros whose names don't include "nr" or "all" wake exactly one exclusive process that has the required state.
- The macros whose names don't include the string "sync" check whether the priority of any of the woken processes is higher than that of the processes currently running in the systems and invoke `schedule( )` if necessary. These checks are not made by the macro whose name includes the string "sync"; as a result, execution of a high priority process might be slightly delayed.
- The `wake_up_locked` macro is similar to `wake_up`, except that it is called when the spin lock in `wait_queue_head_t` is already held.

For instance, the `wake_up` macro is essentially equivalent to the following code fragment:

```
void wake_up(wait_queue_head_t *q)
{
    struct list_head *tmp;
    wait_queue_t *curr;

    list_for_each(tmp, &q->task_list) {
        curr = list_entry(tmp, wait_queue_t, task_list);
        if (curr->func(curr, TASK_INTERRUPTIBLE|TASK_UNINTERRUPTIBLE,
                        0, NULL) && curr->flags)
            break;
    }
}
```

The `list_for_each` macro scans all items in the `q->task_list` doubly linked list, that is, all processes in the wait queue. For each item, the `list_entry` macro computes the address of the corresponding `wait_queue_t` variable. The `func` field of

this variable stores the address of the wake-up function, which tries to wake up the process identified by the `task` field of the wait queue element. If a process has been effectively awakened (the function returned 1) and if the process is exclusive (`current->flags` equal to 1), the loop terminates. Because all nonexclusive processes are always at the beginning of the doubly linked list and all exclusive processes are at the end, the function always wakes the nonexclusive processes and then wakes one exclusive process, if any exists.^[*]

Process Resource Limits

Each process has an associated set of *resource limits*, which specify the amount of system resources it can use. These limits keep a user from overwhelming the system (its CPU, disk space, and so on). Linux recognizes the following resource limits illustrated in [Table 3-7](#).

The resource limits for the current process are stored in the `current->signal->rlim` field, that is, in a field of the process's signal descriptor (see the section "[Data Structures Associated with Signals](#)" in [Chapter 11](#)). The field is an array of elements of type `struct rlimit`, one for each resource limit:

```
struct rlimit {
    unsigned long rlim_cur;
    unsigned long rlim_max;
};
```

Table 3-7. Resource limits

Field name	Description
RLIMIT_AS	The maximum size of process address space, in bytes. The kernel checks this value when the process uses <code>malloc()</code> or a related function to enlarge its address space (see the section " The Process's Address Space " in Chapter 9).
RLIMIT_CORE	The maximum core dump file size, in bytes. The kernel checks this value when a process is aborted, before creating a core file in the current directory of the process (see the section " Actions Performed upon Delivering a Signal " in Chapter 11). If the limit is 0, the kernel won't create the file.
RLIMIT_CPU	The maximum CPU time for the process, in seconds. If the process exceeds the limit, the kernel sends it a <code>SIGXCPU</code> signal, and then, if the process doesn't terminate, a <code>SIGKILL</code> signal (see Chapter 11).
RLIMIT_DATA	The maximum heap size, in bytes. The kernel checks this value before expanding the heap of the process (see the section " Managing the Heap " in Chapter 9).
RLIMIT_FSIZE	The maximum file size allowed, in bytes. If the process tries to enlarge a file to a size greater than this value, the kernel sends it a <code>SIGXFSZ</code> signal.
RLIMIT_LOCKS	Maximum number of file locks (currently, not enforced).
RLIMIT_MEMLOCK	The maximum size of nonswappable memory, in bytes. The kernel checks this value when the process tries to lock a page frame in memory using the <code>mlock()</code> or <code>mlockall()</code> system calls (see the section " Allocating a Linear Address Interval " in Chapter 9).
RLIMIT_MSGQUEUE	Maximum number of bytes in POSIX message queues (see the section " POSIX Message Queues " in Chapter 19).
RLIMIT_NOFILE	The maximum number of open file descriptors . The kernel checks this value when opening a new file or duplicating a file descriptor (see Chapter 12).
RLIMIT_NPROC	The maximum number of processes that the user can own (see the section " The clone(), fork(), and vfork() System Calls " later in this chapter).
RLIMIT_RSS	The maximum number of page frames owned by the process (currently, not enforced).
RLIMIT_SIGPENDING	The maximum number of pending signals for the process (see Chapter 11).

Field name	Description
RLIMIT_STACK	The maximum stack size, in bytes. The kernel checks this value before expanding the User Mode stack of the process (see the section " Page Fault Exception Handler " in Chapter 9).

The `rlim_cur` field is the current resource limit for the resource. For example, `current->signal->rlim[RLIMIT_CPU].rlim_cur` represents the current limit on the CPU time of the running process.

The `rlim_max` field is the maximum allowed value for the resource limit. By using the `getrlimit( )` and `setrlimit( )` system calls, a user can always increase the `rlim_cur` limit of some resource up to `rlim_max`. However, only the superuser (or, more precisely, a user who has the `CAP_SYS_RESOURCE` capability) can increase the `rlim_max` field or set the `rlim_cur` field to a value greater than the corresponding `rlim_max` field.

Most resource limits contain the value `RLIM_INFINITY` (`0xffffffff`), which means that no user limit is imposed on the corresponding resource (of course, real limits exist due to kernel design restrictions, available RAM, available space on disk, etc.). However, the system administrator may choose to impose stronger limits on some resources. Whenever a user logs into the system, the kernel creates a process owned by the superuser, which can invoke `setrlimit( )` to decrease the `rlim_max` and `rlim_cur` fields for a resource. The same process later executes a login shell and becomes owned by the user. Each new process created by the user inherits the content of the `rlim` array from its parent, and therefore the user cannot override the limits enforced by the administrator.

[*] The kernel also defines the `task_t` data type to be equivalent to `struct task_struct`.

[*] There are other `wait( )`-like library functions, such as `wait3( )` and `wait( )`, but in Linux they are implemented by means of the `wait4( )` and `waitpid( )` system calls.

[†] As already noted in the section "[Segmentation in Linux](#)" in [Chapter 2](#), although technically these 32 bits are only the offset component of a logical address, they coincide with the linear address.

[*] By the way, it is rather uncommon that a wait queue includes both exclusive and nonexclusive processes.

Process Switch

To control the execution of processes, the kernel must be able to suspend the execution of the process running on the CPU and resume the execution of some other process previously suspended. This activity goes variously by the names *process switch*, *task switch*, or *context switch*. The next sections describe the elements of process switching in Linux.

Hardware Context

While each process can have its own address space, all processes have to share the CPU registers. So before resuming the execution of a process, the kernel must ensure that each such register is loaded with the value it had when the process was suspended.

The set of data that must be loaded into the registers before the process resumes its execution on the CPU is called the *hardware context*. The hardware context is a subset of the process execution context, which includes all information needed for the process execution. In Linux, a part of the hardware context of a process is stored in the process descriptor, while the remaining part is saved in the Kernel Mode stack.

In the description that follows, we will assume the `prev` local variable refers to the process descriptor of the process being switched out and `next` refers to the one being switched in to replace it. We can thus define a *process switch* as the activity consisting of saving the hardware context of `prev` and replacing it with the hardware context of `next`. Because process switches occur quite often, it is important to minimize the time spent in saving and loading hardware contexts.

Old versions of Linux took advantage of the hardware support offered by the 80x86 architecture and performed a process switch through a `far jmp` instruction^[*] to the selector of the Task State Segment Descriptor of the `next` process. While executing the instruction, the CPU performs a *hardware context switch* by automatically saving the old hardware context and loading a new one. But Linux 2.6 uses software to perform a process switch for the following reasons:

- Step-by-step switching performed through a sequence of `mov` instructions allows better control over the validity of the data being loaded. In particular, it is possible to check the values of the `ds` and `es` segmentation registers, which might have been forged by a malicious user. This type of checking is not possible when using a single `far jmp` instruction.

- The amount of time required by the old approach and the new approach is about the same. However, it is not possible to optimize a hardware context switch, while there might be room for improving the current switching code.

Process switching occurs only in Kernel Mode. The contents of all registers used by a process in User Mode have already been saved on the Kernel Mode stack before performing process switching (see [Chapter 4](#)). This includes the contents of the `ss` and `esp` pair that specifies the User Mode stack pointer address.

Task State Segment

The 80×86 architecture includes a specific segment type called the *Task State Segment* (TSS), to store hardware contexts. Although Linux doesn't use hardware context switches, it is nonetheless forced to set up a TSS for each distinct CPU in the system. This is done for two main reasons:

- When an 80×86 CPU switches from User Mode to Kernel Mode, it fetches the address of the Kernel Mode stack from the TSS (see the sections "[Hardware Handling of Interrupts and Exceptions](#)" in [Chapter 4](#) and "[Issuing a System Call via the `sysenter` Instruction](#)" in [Chapter 10](#)).
- When a User Mode process attempts to access an I/O port by means of an `in` or `out` instruction, the CPU may need to access an I/O Permission Bitmap stored in the TSS to verify whether the process is allowed to address the port.

More precisely, when a process executes an `in` or `out` I/O instruction in User Mode, the control unit performs the following operations:

1. It checks the 2-bit IOPL field in the `eflags` register. If it is set to 3, the control unit executes the I/O instructions. Otherwise, it performs the next check.
2. It accesses the `tr` register to determine the current TSS, and thus the proper I/O Permission Bitmap.
3. It checks the bit of the I/O Permission Bitmap corresponding to the I/O port specified in the I/O instruction. If it is cleared, the instruction is executed; otherwise, the control unit raises a "General protection " exception.

The `tss_struct` structure describes the format of the TSS. As already mentioned in [Chapter 2](#), the `init_tss` array stores one TSS for each CPU on the system. At each process switch, the kernel updates some fields of the TSS so that the corresponding

CPU's control unit may safely retrieve the information it needs. Thus, the TSS reflects the privilege of the current process on the CPU, but there is no need to maintain TSSs for processes when they're not running.

Each TSS has its own 8-byte *Task State Segment Descriptor* (TSSD). This descriptor includes a 32-bit **Base** field that points to the TSS starting address and a 20-bit **Limit** field. The **S** flag of a TSSD is cleared to denote the fact that the corresponding TSS is a System Segment (see the section "[Segment Descriptors](#)" in [Chapter 2](#)).

The **Type** field is set to either 9 or 11 to denote that the segment is actually a TSS. In the Intel's original design, each process in the system should refer to its own TSS; the second least significant bit of the **Type** field is called the *Busy bit*; it is set to 1 if the process is being executed by a CPU, and to 0 otherwise. In Linux design, there is just one TSS for each CPU, so the *Busy bit* is always set to 1.

The TSSDs created by Linux are stored in the Global Descriptor Table (GDT), whose base address is stored in the `gdtr` register of each CPU. The `tr` register of each CPU contains the TSSD Selector of the corresponding TSS. The register also includes two hidden, nonprogrammable fields: the **Base** and **Limit** fields of the TSSD. In this way, the processor can address the TSS directly without having to retrieve the TSS address from the GDT.

The thread field

At every process switch, the hardware context of the process being replaced must be saved somewhere. It cannot be saved on the TSS, as in the original Intel design, because Linux uses a single TSS for each processor, instead of one for every process.

Thus, each process descriptor includes a field called `thread` of type `thread_struct`, in which the kernel saves the hardware context whenever the process is being switched out. As we'll see later, this data structure includes fields for most of the CPU registers, except the general-purpose registers such as `eax`, `ebx`, etc., which are stored in the Kernel Mode stack.

Performing the Process Switch

A process switch may occur at just one well-defined point: the `schedule( )` function, which is discussed at length in [Chapter 7](#). Here, we are only concerned with how the kernel performs a process switch.

Essentially, every process switch consists of two steps:

1. Switching the Page Global Directory to install a new address space; we'll describe this step in [Chapter 9](#).
2. Switching the Kernel Mode stack and the hardware context, which provides all the information needed by the kernel to execute the new process, including the CPU registers.

Again, we assume that `prev` points to the descriptor of the process being replaced, and `next` to the descriptor of the process being activated. As we'll see in [Chapter 7](#), `prev` and `next` are local variables of the `schedule( )` function.

The `switch_to` macro

The second step of the process switch is performed by the `switch_to` macro. It is one of the most hardware-dependent routines of the kernel, and it takes some effort to understand what it does.

First of all, the macro has three parameters, called `prev`, `next`, and `last`. You might easily guess the role of `prev` and `next`: they are just placeholders for the local variables `prev` and `next`, that is, they are input parameters that specify the memory locations containing the descriptor address of the process being replaced and the descriptor address of the new process, respectively.

What about the third parameter, `last`? Well, in any process switch three processes are involved, not just two. Suppose the kernel decides to switch off process A and to activate process B. In the `schedule( )` function, `prev` points to A's descriptor and `next` points to B's descriptor. As soon as the `switch_to` macro deactivates A, the execution flow of A freezes.

Later, when the kernel wants to reactivate A, it must switch off another process C (in general, this is different from B) by executing another `switch_to` macro with `prev` pointing to C and `next` pointing to A. When A resumes its execution flow, it finds its old Kernel Mode stack, so the `prev` local variable points to A's descriptor and `next` points to B's descriptor. The scheduler, which is now executing on behalf of process A, has lost any reference to C. This reference, however, turns out to be useful to complete the process switching (see [Chapter 7](#) for more details).

The last parameter of the `switch_to` macro is an output parameter that specifies a memory location in which the macro writes the descriptor address of process C (of course, this is done after A resumes its execution). Before the process switching, the macro saves in the `eax` CPU register the content of the variable identified by the first

input parameter `prev`—that is, the `prev` local variable allocated on the Kernel Mode stack of A. After the process switching, when A has resumed its execution, the macro writes the content of the `eax` CPU register in the memory location of A identified by the third output parameter `last`. Because the CPU register doesn't change across the process switch, this memory location receives the address of C's descriptor. In the current implementation of `schedule( )`, the `last` parameter identifies the `prev` local variable of A, so `prev` is overwritten with the address of C.

The contents of the Kernel Mode stacks of processes A, B, and C are shown in [Figure 3-7](#), together with the values of the `eax` register; be warned that the figure shows the value of the `prev` local variable *before* its value is overwritten with the contents of the `eax` register.

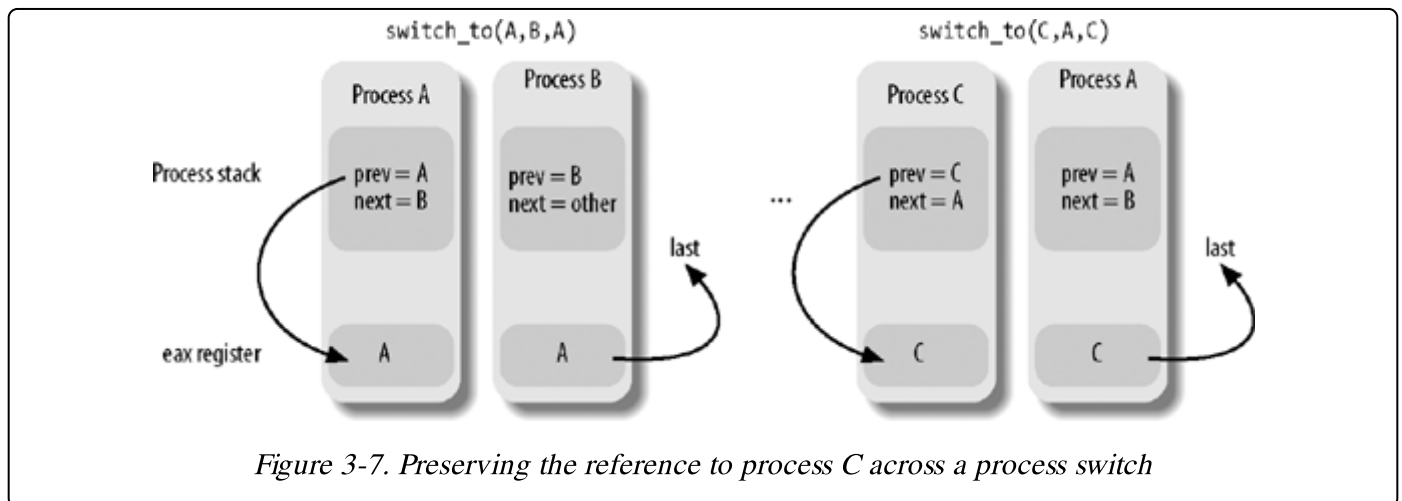


Figure 3-7. Preserving the reference to process C across a process switch

The `switch_to` macro is coded in *extended inline assembly language* that makes for rather complex reading: in fact, the code refers to registers by means of a special positional notation that allows the compiler to freely choose the general-purpose registers to be used. Rather than follow the cumbersome extended inline assembly language, we'll describe what the `switch_to` macro typically does on an 80x86 microprocessor by using standard assembly language:

1. Saves the values of `prev` and `next` in the `eax` and `edx` registers, respectively:

```
movl prev, %eax
movl next, %edx
```

2. Saves the contents of the `eflags` and `ebp` registers in the `prev` Kernel Mode stack. They must be saved because the compiler assumes that they will stay unchanged until the end of `switch_to`:

```
pushfl
pushl %ebp
```

3. Saves the content of `esp` in `prev->thread.esp` so that the field points to the top of the `prev` Kernel Mode stack:

```
movl %esp, 484(%eax)
```

The `484(%eax)` operand identifies the memory cell whose address is the contents of `eax` plus 484.

4. Loads `next->thread.esp` in `esp`. From now on, the kernel operates on the Kernel Mode stack of `next`, so this instruction performs the actual process switch from `prev` to `next`. Because the address of a process descriptor is closely related to that of the Kernel Mode stack (as explained in the section "[Identifying a Process](#)" earlier in this chapter), changing the kernel stack means changing the current process:

```
movl 484(%edx), %esp
```

5. Saves the address labeled 1 (shown later in this section) in `prev->thread.eip`. When the process being replaced resumes its execution, the process executes the instruction labeled as 1:

```
movl $1f, 480(%eax)
```

6. On the Kernel Mode stack of `next`, the macro pushes the `next->thread.eip` value, which, in most cases, is the address labeled as 1:

```
pushl 480(%edx)
```

7. Jumps to the `_ _switch_to( )` C function (see next):

```
jmp _ _switch_to
```

8. Here process A that was replaced by B gets the CPU again: it executes a few instructions that restore the contents of the `eflags` and `ebp` registers. The first of these two instructions is labeled as 1:

```
1:
    popl %ebp
    popfl
```

Notice how these `pop` instructions refer to the kernel stack of the `prev` process. They will be executed when the scheduler selects `prev` as the new process to be executed on the CPU, thus invoking `switch_to` with `prev` as the second parameter. Therefore, the `esp` register points to the `prev`'s Kernel Mode stack.

9. Copies the content of the `eax` register (loaded in step 1 above) into the memory location identified by the third parameter `last` of the `switch_to` macro:

```
movl %eax, last
```

As discussed earlier, the `eax` register points to the descriptor of the process that has just been replaced.^[*]

The `__switch_to( )` function

The `__switch_to( )` function does the bulk of the process switch started by the `switch_to( )` macro. It acts on the `prev_p` and `next_p` parameters that denote the former process and the new process. This function call is different from the average function call, though, because `__switch_to( )` takes the `prev_p` and `next_p` parameters from the `eax` and `edx` registers (where we saw they were stored), not from the stack like most functions. To force the function to go to the registers for its parameters, the kernel uses the `__attribute__((regparm(3)))` keyword, which are nonstandard extensions of the C language implemented by the `gcc` compiler. The `__switch_to( )` function is declared in the `include/asm-i386/system.h` header file as follows:

```
__switch_to(struct task_struct *prev_p,  
            struct task_struct *next_p)  
__attribute__((regparm(3)));
```

The steps performed by the function are the following:

1. Executes the code yielded by the `__unlazy_fpu( )` macro (see the section "[Saving and Loading the FPU](#)", MMX, and XMM Registers" later in this chapter) to optionally save the contents of the FPU, MMX, and XMM registers of the `prev_p` process.

```
__unlazy_fpu(prev_p);
```

2. Executes the `smp_processor_id( )` macro to get the index of the *local CPU*, namely the CPU that executes the code. The macro gets the index from the `cpu` field of the `thread_info` structure of the current process and stores it into the `cpu` local variable.
3. Loads `next_p->thread.esp0` in the `esp0` field of the TSS relative to the local CPU; as we'll see in the section "[Issuing a System Call via the sysenter Instruction](#)" in [Chapter 10](#), any future privilege level change from User Mode to Kernel Mode raised by a `sysenter` assembly instruction will copy this address in the `esp` register:

```
init_tss[cpu].esp0 = next_p->thread.esp0;
```

4. Loads in the Global Descriptor Table of the local CPU the Thread-Local Storage (TLS) segments used by the `next_p` process; the three Segment Selectors are stored in the `tls_array` array inside the process descriptor (see the section "[Segmentation in Linux](#)" in [Chapter 2](#)).

```
cpu_gdt_table[cpu][6] = next_p->thread.tls_array[0];
cpu_gdt_table[cpu][7] = next_p->thread.tls_array[1];
cpu_gdt_table[cpu][8] = next_p->thread.tls_array[2];
```

5. Stores the contents of the `fs` and `gs` segmentation registers in `prev_p->thread.fs` and `prev_p->thread.gs`, respectively; the corresponding assembly language instructions are:

```
movl %fs, 40(%esi)
movl %gs, 44(%esi)
```

The `esi` register points to the `prev_p->thread` structure.

6. If the `fs` or the `gs` segmentation register have been used either by the `prev_p` or by the `next_p` process (i.e., if they have a nonzero value), loads into these registers the values stored in the `thread_struct` descriptor of the `next_p` process. This step logically complements the actions performed in the previous step. The main assembly language instructions are:

```
movl 40(%ebx),%fs
movl 44(%ebx),%gs
```

The `ebx` register points to the `next_p->thread` structure. The code is actually more intricate, as an exception might be raised by the CPU when it detects an invalid segment register value. The code takes this possibility into account by adopting a "fix-up" approach (see the section "[Dynamic Address Checking: The Fix-up Code](#)" in [Chapter 10](#)).

7. Loads six of the `dr0`, ..., `dr7` debug registers ^[*] with the contents of the `next_p->thread.debugreg` array. This is done only if `next_p` was using the debug registers when it was suspended (that is, field `next_p->thread.debugreg[7]` is not 0). These registers need not be saved, because the `prev_p->thread.debugreg` array is modified only when a debugger wants to monitor `prev`:

```
if (next_p->thread.debugreg[7]){
    loaddebug(&next_p->thread, 0);
    loaddebug(&next_p->thread, 1);
    loaddebug(&next_p->thread, 2);
    loaddebug(&next_p->thread, 3);
    /* no 4 and 5 */
    loaddebug(&next_p->thread, 6);
}
```



```

        loaddebug(&next_p->thread, 7);
    }

```

8. Updates the I/O bitmap in the TSS, if necessary. This must be done when either `next_p` or `prev_p` has its own customized I/O Permission Bitmap:

```

    if (prev_p->thread.io_bitmap_ptr || next_p->thread.io_bitmap_ptr)
        handle_io_bitmap(&next_p->thread, &init_tss[cpu]);

```

Because processes seldom modify the I/O Permission Bitmap, this bitmap is handled in a "lazy" mode: the actual bitmap is copied into the TSS of the local CPU only if a process actually accesses an I/O port in the current timeslice. The customized I/O Permission Bitmap of a process is stored in a buffer pointed to by the `io_bitmap_ptr` field of the `thread_info` structure. The `handle_io_bitmap( )` function sets up the `io_bitmap` field of the TSS used by the local CPU for the `next_p` process as follows:

- If the `next_p` process does not have its own customized I/O Permission Bitmap, the `io_bitmap` field of the TSS is set to the value `0x8000`.
- If the `next_p` process has its own customized I/O Permission Bitmap, the `io_bitmap` field of the TSS is set to the value `0x9000`.

The `io_bitmap` field of the TSS should contain an offset inside the TSS where the actual bitmap is stored. The `0x8000` and `0x9000` values point outside of the TSS limit and will thus cause a "General protection " exception whenever the User Mode process attempts to access an I/O port (see the section "[Exceptions](#)" in [Chapter 4](#)). The `do_general_protection( )` exception handler will check the value stored in the `io_bitmap` field: if it is `0x8000`, the function sends a SIGSEGV signal to the User Mode process; otherwise, if it is `0x9000`, the function copies the process bitmap (pointed to by the `io_bitmap_ptr` field in the `thread_info` structure) in the TSS of the local CPU, sets the `io_bitmap` field to the actual bitmap offset (104), and forces a new execution of the faulty assembly language instruction.

9. Terminates. The `_ _switch_to( )` C function ends by means of the statement:

```

    return prev_p;

```

The corresponding assembly language instructions generated by the compiler are:

```

    movl %edi,%eax
    ret

```

The `prev_p` parameter (now in `edi`) is copied into `eax`, because by default the return value of any C function is passed in the `eax` register. Notice that the value of `eax` is thus preserved across the invocation of `_ _switch_to( )`; this is quite important, because the invoking `switch_to` macro assumes that `eax` always stores the address of the process descriptor being replaced.

The `ret` assembly language instruction loads the `eip` program counter with the return address stored on top of the stack. However, the `_ _switch_to( )` function has been invoked simply by jumping into it. Therefore, the `ret` instruction finds on the stack the address of the instruction labeled as 1, which was pushed by the `switch_to` macro. If `next_p` was never suspended before because it is being executed for the first time, the function finds the starting address of the `ret_from_fork( )` function (see the section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)" later in this chapter).

Saving and Loading the FPU, MMX, and XMM Registers

Starting with the Intel 80486DX, the arithmetic floating-point unit (FPU) has been integrated into the CPU. The name *mathematical coprocessor* continues to be used in memory of the days when floating-point computations were executed by an expensive special-purpose chip. To maintain compatibility with older models, however, floating-point arithmetic functions are performed with *ESCAPE instructions*, which are instructions with a prefix byte ranging between `0xd8` and `0xdf`. These instructions act on the set of floating-point registers included in the CPU. Clearly, if a process is using ESCAPE instructions, the contents of the floating-point registers belong to its hardware context and should be saved.

In later Pentium models, Intel introduced a new set of assembly language instructions into its microprocessors. They are called *MMX instructions* and are supposed to speed up the execution of multimedia applications. MMX instructions act on the floating-point registers of the FPU. The obvious disadvantage of this architectural choice is that programmers cannot mix floating-point instructions and MMX instructions. The advantage is that operating system designers can ignore the new instruction set, because the same facility of the task-switching code for saving the state of the floating-point unit can also be relied upon to save the MMX state.

MMX instructions speed up multimedia applications, because they introduce a single-instruction multiple-data (SIMD) pipeline inside the processor. The Pentium III

model extends that SIMD capability: it introduces the *SSE extensions* (Streaming SIMD Extensions), which adds facilities for handling floating-point values contained in eight 128-bit registers called the XMM registers . Such registers do not overlap with the FPU and MMX registers , so SSE and FPU/MMX instructions may be freely mixed. The Pentium 4 model introduces yet another feature: the *SSE2 extensions*, which is basically an extension of SSE supporting higher-precision floating-point values. SSE2 uses the same set of XMM registers as SSE.

The 80×86 microprocessors do not automatically save the FPU, MMX, and XMM registers in the TSS. However, they include some hardware support that enables kernels to save these registers only when needed. The hardware support consists of a TS (Task-Switching) flag in the `cr0` register, which obeys the following rules:

- Every time a hardware context switch is performed, the TS flag is set.
- Every time an ESCAPE, MMX, SSE, or SSE2 instruction is executed when the TS flag is set, the control unit raises a "Device not available " exception (see [Chapter 4](#)).

The TS flag allows the kernel to save and restore the FPU, MMX, and XMM registers only when really needed. To illustrate how it works, suppose that a process A is using the mathematical coprocessor. When a context switch occurs from A to B, the kernel sets the TS flag and saves the floating-point registers into the TSS of process A. If the new process B does not use the mathematical coprocessor, the kernel won't need to restore the contents of the floating-point registers. But as soon as B tries to execute an ESCAPE or MMX instruction, the CPU raises a "Device not available" exception, and the corresponding handler loads the floating-point registers with the values saved in the TSS of process B.

Let's now describe the data structures introduced to handle selective loading of the FPU, MMX, and XMM registers. They are stored in the `thread.i387` subfield of the process descriptor, whose format is described by the `i387_union` union:

```
union i387_union {
    struct i387_fsave_struct    fsave;
    struct i387_fxsave_struct  fxsave;
    struct i387_soft_struct     soft;
};
```

As you see, the field may store just one of three different types of data structures. The `i387_soft_struct` type is used by CPU models without a mathematical coprocessor; the Linux kernel still supports these old chips by emulating the coprocessor via

software. We don't discuss this legacy case further, however. The `i387_fsave_struct` type is used by CPU models with a mathematical coprocessor and, optionally, an MMX unit. Finally, the `i387_fxsave_struct` type is used by CPU models featuring SSE and SSE2 extensions.

The process descriptor includes two additional flags:

- The `TS_USED_FPU` flag, which is included in the `status` field of the `thread_info` descriptor. It specifies whether the process used the FPU, MMX, or XMM registers in the current execution run.
- The `PF_USED_MATH` flag, which is included in the `flags` field of the `task_struct` descriptor. This flag specifies whether the contents of the `thread.i387` subfield are significant. The flag is cleared (not significant) in two cases, shown in the following list.

— When the process starts executing a new program by invoking an `execve( )` system call (see [Chapter 20](#)). Because control will never return to the former program, the data currently stored in `thread.i387` is never used again.

— When a process that was executing a program in User Mode starts executing a signal handler procedure (see [Chapter 11](#)). Because signal handlers are asynchronous with respect to the program execution flow, the floating-point registers could be meaningless to the signal handler. However, the kernel saves the floating-point registers in `thread.i387` before starting the handler and restores them after the handler terminates. Therefore, a signal handler is allowed to use the mathematical coprocessor.

Saving the FPU registers

As stated earlier, the `_ _switch_to( )` function executes the `_ _unlazy_fpu` macro, passing the process descriptor of the `prev` process being replaced as an argument. The macro checks the value of the `TS_USED_FPU` flags of `prev`. If the flag is set, `prev` has used an FPU, MMX, SSE, or SSE2 instructions; therefore, the kernel must save the relative hardware context:

```
if (prev->thread_info->status & TS_USED_FPU)
    save_init_fpu(prev);
```

The `save_init_fpu( )` function, in turn, executes essentially the following operations:

1. Dumps the contents of the FPU registers in the process descriptor of `prev` and then reinitializes the FPU. If the CPU uses SSE/SSE2 extensions, it also dumps the contents of the XMM registers and reinitializes the SSE/SSE2 unit. A couple of powerful extended inline assembly language instructions take care of everything, either:

```
asm volatile( "fxsave
%0 ; fnclex"
: "=m" (prev->thread.i387.fxsave) );
```

if the CPU uses SSE/SSE2 extensions, or otherwise:

```
asm volatile( "fnsave
%0 ; fwait"
: "=m" (prev->thread.i387.fsave) );
```

2. Resets the `TS_USED_FPU` flag of `prev`:

```
prev->thread_info->status &= ~TS_USED_FPU;
```

3. Sets the `CW` flag of `cr0` by means of the `stts( )` macro, which in practice yields assembly language instructions like the following:

```
movl %cr0, %eax
orl $8,%eax
movl %eax, %cr0
```

Loading the FPU registers

The contents of the floating-point registers are not restored right after the next process resumes execution. However, the `TS` flag of `cr0` has been set by `_unlazy_fpu( )`. Thus, the first time the next process tries to execute an `ESCAPE`, `MMX`, or `SSE/SSE2` instruction, the control unit raises a "Device not available" exception, and the kernel (more precisely, the exception handler involved by the exception) runs the `math_state_restore( )` function. The next process is identified by this handler as `current`.

```
void math_state_restore( )
{
    asm volatile ("clts"); /* clear the TS flag of cr0 */
    if (!(current->flags & PF_USED_MATH))
        init_fpu(current);
    restore_fpu(current);
    current->thread.status |= TS_USED_FPU;
}
```

The function clears the `CW` flags of `cr0`, so that further FPU, `MMX`, or `SSE/SSE2` instructions executed by the process won't trigger the "Device not available" exception. If the contents of the `thread.i387` subfield are not significant, i.e., if the

PF_USED_MATH flag is equal to 0, `init_fpu()` is invoked to reset the `thread.i387` subfield and to set the PF_USED_MATH flag of `current` to 1. The `restore_fpu()` function is then invoked to load the FPU registers with the proper values stored in the `thread.i387` subfield. To do this, either the `fxrstor` or the `frstor` assembly language instructions are used, depending on whether the CPU supports SSE/SSE2 extensions. Finally, `math_state_restore()` sets the TS_USED_FPU flag.

Using the FPU, MMX, and SSE/SSE2 units in Kernel Mode

Even the kernel can make use of the FPU, MMX, or SSE/SSE2 units. In doing so, of course, it should avoid interfering with any computation carried on by the current User Mode process. Therefore:

- Before using the coprocessor, the kernel must invoke `kernel_fpu_begin()`, which essentially calls `save_init_fpu()` to save the contents of the registers if the User Mode process used the FPU (TS_USED_FPU flag), and then resets the TS flag of the `cr0` register.
- After using the coprocessor, the kernel must invoke `kernel_fpu_end()`, which sets the TS flag of the `cr0` register.

Later, when the User Mode process executes a coprocessor instruction, the `math_state_restore()` function will restore the contents of the registers, just as in process switch handling.

It should be noted, however, that the execution time of `kernel_fpu_begin()` is rather large when the current User Mode process is using the coprocessor, so much as to nullify the speedup obtained by using the FPU, MMX, or SSE/SSE2 units. As a matter of fact, the kernel uses them only in a few places, typically when moving or clearing large memory areas or when computing checksum functions.

[*] `far jmp` instructions modify both the `cs` and `eip` registers, while simple `jmp` instructions modify only `eip`.

[*] As stated earlier in this section, the current implementation of the `schedule()` function reuses the `prev` local variable, so that the assembly language instruction looks like `movl %eax,prev`.

[*] The 80x86 debug registers allow a process to be monitored by the hardware. Up to four breakpoint areas may be defined. Whenever a monitored process issues a linear address included in one of the breakpoint areas, an exception occurs.

Creating Processes

Unix operating systems rely heavily on process creation to satisfy user requests. For example, the shell creates a new process that executes another copy of the shell whenever the user enters a command.

Traditional Unix systems treat all processes in the same way: resources owned by the parent process are duplicated in the child process. This approach makes process creation very slow and inefficient, because it requires copying the entire address space of the parent process. The child process rarely needs to read or modify all the resources inherited from the parent; in many cases, it issues an immediate `execve( )` and wipes out the address space that was so carefully copied.

Modern Unix kernels solve this problem by introducing three different mechanisms:

- The Copy On Write technique allows both the parent and the child to read the same physical pages. Whenever either one tries to write on a physical page, the kernel copies its contents into a new physical page that is assigned to the writing process. The implementation of this technique in Linux is fully explained in [Chapter 9](#).
- Lightweight processes allow both the parent and the child to share many per-process kernel data structures, such as the paging tables (and therefore the entire User Mode address space), the open file tables, and the signal dispositions.
- The `vfork( )` system call creates a process that shares the memory address space of its parent. To prevent the parent from overwriting data needed by the child, the parent's execution is blocked until the child exits or executes a new program. We'll learn more about the `vfork( )` system call in the following section.

The `clone( )`, `fork( )`, and `vfork( )` System Calls

Lightweight processes are created in Linux by using a function named `clone( )`, which uses the following parameters:

fn

Specifies a function to be executed by the new process; when the function returns, the child terminates. The function returns an integer, which represents the exit code for the child process.

arg

Points to data passed to the `fn( )` function.

flags

Miscellaneous information. The low byte specifies the signal number to be sent to the parent process when the child terminates; the SIGCHLD signal is generally selected. The remaining three bytes encode a group of clone flags, which are shown in [Table 3-8](#).

child_stack

Specifies the User Mode stack pointer to be assigned to the `esp` register of the child process. The invoking process (the parent) should always allocate a new stack for the child.

tls

Specifies the address of a data structure that defines a Thread Local Storage segment for the new lightweight process (see the section "[The Linux GDT](#)" in [Chapter 2](#)). Meaningful only if the CLONE_SETTLS flag is set.

ptid

Specifies the address of a User Mode variable of the parent process that will hold the PID of the new lightweight process. Meaningful only if the CLONE_PARENT_SETTID flag is set.

ctid

Specifies the address of a User Mode variable of the new lightweight process that will hold the PID of such process. Meaningful only if the CLONE_CHILD_SETTID flag is set.

Table 3-8. Clone flags

Flag name	Description
CLONE_VM	Shares the memory descriptor and all Page Tables (see Chapter 9).
CLONE_FS	Shares the table that identifies the root directory and the current working directory, as well as the value of the bitmask used to mask the initial file permissions of a new file (the so-called file umask).
CLONE_FILES	Shares the table that identifies the open files (see Chapter 12).
CLONE_SIGHAND	Shares the tables that identify the signal handlers and the blocked and pending signals (see Chapter 11). If this flag is true, the CLONE_VM flag must also be set.
CLONE_PTRACE	If traced, the parent wants the child to be traced too. Furthermore, the debugger may want to trace the child on its own; in this case, the kernel forces the flag to 1.
CLONE_VFORK	Set when the system call issued is a <code>vfork()</code> (see later in this section).
CLONE_PARENT	Sets the parent of the child (<code>parent</code> and <code>real_parent</code> fields in the process descriptor) to the parent of the calling process.
CLONE_THREAD	Inserts the child into the same thread group of the parent, and forces the child to share the signal descriptor of the parent. The child's <code>tgid</code> and <code>group_leader</code> fields are set accordingly. If this flag is true, the CLONE_SIGHAND flag must also be set.
CLONE_NEWNS	Set if the clone needs its own namespace, that is, its own view of the mounted filesystems (see Chapter 12); it is not possible to specify both CLONE_NEWNS and CLONE_FS.
CLONE_SYSVSEM	Shares the System V IPC undoable semaphore operations (see the section " IPC Semaphores " in Chapter 19).
CLONE_SETTLS	Creates a new Thread Local Storage (TLS) segment for the lightweight process; the segment is described in the structure pointed to by the <code>tls</code> parameter.
CLONE_PARENT_SETTID	Writes the PID of the child into the User Mode variable of the parent pointed to by the <code>ptid</code> parameter.
CLONE_CHILD_CLEARTID	When set, the kernel sets up a mechanism to be triggered when the child process will exit or when it will start executing a new program. In these cases, the kernel will clear the User Mode variable pointed to by the <code>ctid</code> parameter and will awaken any process waiting for this event.

Flag name	Description
CLONE_DETACHED	A legacy flag ignored by the kernel.
CLONE_UNTRACED	Set by the kernel to override the value of the CLONE_PTRACE flag (used for disabling tracing of kernel threads ; see the section " Kernel Threads " later in this chapter).
CLONE_CHILD_SETTID	Writes the PID of the child into the User Mode variable of the child pointed to by the ctid parameter.
CLONE_STOPPED	Forces the child to start in the TASK_STOPPED state.

`clone( )` is actually a wrapper function defined in the C library (see the section "[POSIX APIs and System Calls](#)" in [Chapter 10](#)), which sets up the stack of the new lightweight process and invokes a `clone( )` system call hidden to the programmer. The `sys_clone( )` service routine that implements the `clone( )` system call does not have the `fn` and `arg` parameters. In fact, the wrapper function saves the pointer `fn` into the child's stack position corresponding to the return address of the wrapper function itself; the pointer `arg` is saved on the child's stack right below `fn`. When the wrapper function terminates, the CPU fetches the return address from the stack and executes the `fn(arg)` function.

The traditional `fork( )` system call is implemented by Linux as a `clone( )` system call whose `flags` parameter specifies both a `SIGCHLD` signal and all the clone flags cleared, and whose `child_stack` parameter is the current parent stack pointer. Therefore, the parent and child temporarily share the same User Mode stack. But thanks to the Copy On Write mechanism, they usually get separate copies of the User Mode stack as soon as one tries to change the stack.

The `vfork( )` system call, introduced in the previous section, is implemented by Linux as a `clone( )` system call whose `flags` parameter specifies both a `SIGCHLD` signal and the flags `CLONE_VM` and `CLONE_VFORK`, and whose `child_stack` parameter is equal to the current parent stack pointer.

The `do_fork( )` function

The `do_fork( )` function, which handles the `clone( )`, `fork( )`, and `vfork( )` system calls, acts on the following parameters:

`clone_flags`

Same as the `flags` parameter of `clone( )`

`stack_start`

Same as the `child_stack` parameter of `clone( )`

`regs`

Pointer to the values of the general purpose registers saved into the Kernel Mode stack when switching from User Mode to Kernel Mode (see the section "[The `do\_IRQ\(.\)` function](#)" in [Chapter 4](#))

`stack_size`

Unused (always set to 0)

`parent_tidptr, child_tidptr`

Same as the corresponding `ptid` and `ctid` parameters of `clone( )`

`do_fork( )` makes use of an auxiliary function called `copy_process( )` to set up the process descriptor and any other kernel data structure required for child's execution.

Here are the main steps performed by `do_fork( )`:

1. Allocates a new PID for the child by looking in the `pidmap_array` bitmap (see the earlier section "[Identifying a Process](#)").
2. Checks the `ptrace` field of the parent (`current->ptrace`): if it is not zero, the parent process is being traced by another process, thus `do_fork( )` checks whether the debugger wants to trace the child on its own (independently of the value of the `CLONE_PTRACE` flag specified by the parent); in this case, if the child is not a kernel thread (`CLONE_UNTRACED` flag cleared), the function sets the `CLONE_PTRACE` flag.
3. Invokes `copy_process( )` to make a copy of the process descriptor. If all needed resources are available, this function returns the address of the `task_struct` descriptor just created. This is the workhorse of the forking procedure, and we will describe it right after `do_fork( )`.
4. If either the `CLONE_STOPPED` flag is set or the child process must be traced, that is, the `PT_PTRACED` flag is set in `p->ptrace`, it sets the state of the child to `TASK_STOPPED` and adds a pending `SIGSTOP` signal to it (see the section "[The Role of Signals](#)" in [Chapter 11](#)). The state of the child will remain `TASK_STOPPED` until another process (presumably the tracing process or the parent) will revert its state to `TASK_RUNNING`, usually by means of a `SIGCONT` signal.

5. If the `CLONE_STOPPED` flag is not set, it invokes the `wake_up_new_task( )` function, which performs the following operations:
 - a. Adjusts the scheduling parameters of both the parent and the child (see "[The Scheduling Algorithm](#)" in [Chapter 7](#)).
 - b. If the child will run on the same CPU as the parent,^[*] and parent and child do not share the same set of page tables (`CLONE_VM` flag cleared), it then forces the child to run before the parent by inserting it into the parent's runqueue right before the parent. This simple step yields better performance if the child flushes its address space and executes a new program right after the forking. If we let the parent run first, the Copy On Write mechanism would give rise to a series of unnecessary page duplications.
 - c. Otherwise, if the child will not be run on the same CPU as the parent, or if parent and child share the same set of page tables (`CLONE_VM` flag set), it inserts the child in the last position of the parent's runqueue.
6. If the `CLONE_STOPPED` flag is set, it puts the child in the `TASK_STOPPED` state.
7. If the parent process is being traced, it stores the PID of the child in the `ptrace_message` field of `current` and invokes `ptrace_notify( )`, which essentially stops the current process and sends a `SIGCHLD` signal to its parent. The "grandparent" of the child is the debugger that is tracing the parent; the `SIGCHLD` signal notifies the debugger that `current` has forked a child, whose PID can be retrieved by looking into the `current->ptrace_message` field.
8. If the `CLONE_VFORK` flag is specified, it inserts the parent process in a wait queue and suspends it until the child releases its memory address space (that is, until the child either terminates or executes a new program).
9. Terminates by returning the PID of the child.

The `copy_process( )` function

The `copy_process( )` function sets up the process descriptor and any other kernel data structure required for a child's execution. Its parameters are the same as `do_fork( )`, plus the PID of the child. Here is a description of its most significant steps:

1. Checks whether the flags passed in the `clone_flags` parameter are compatible. In particular, it returns an error code in the following cases:

- a. Both the flags `CLONE_NEWNS` and `CLONE_FS` are set.
 - b. The `CLONE_THREAD` flag is set, but the `CLONE_SIGHAND` flag is cleared (lightweight processes in the same thread group must share signals).
 - c. The `CLONE_SIGHAND` flag is set, but the `CLONE_VM` flag is cleared (lightweight processes sharing the signal handlers must also share the memory descriptor).
2. Performs any additional security checks by invoking `security_task_create( )` and, later, `security_task_alloc( )`. The Linux kernel 2.6 offers hooks for security extensions that enforce a security model stronger than the one adopted by traditional Unix. See [Chapter 20](#) for details.
3. Invokes `dup_task_struct( )` to get the process descriptor for the child. This function performs the following actions:
- a. Invokes `__unlazy_fpu( )` on the current process to save, if necessary, the contents of the FPU, MMX, and SSE/SSE2 registers in the `thread_info` structure of the parent. Later, `dup_task_struct( )` will copy these values in the `thread_info` structure of the child.
 - b. Executes the `alloc_task_struct( )` macro to get a process descriptor (`task_struct` structure) for the new process, and stores its address in the `tsk` local variable.
 - c. Executes the `alloc_thread_info` macro to get a free memory area to store the `thread_info` structure and the Kernel Mode stack of the new process, and saves its address in the `ti` local variable. As explained in the earlier section "[Identifying a Process](#)," the size of this memory area is either 8 KB or 4 KB.
 - d. Copies the contents of the current's process descriptor into the `task_struct` structure pointed to by `tsk`, then sets `tsk->thread_info` to `ti`.
 - e. Copies the contents of the current's `thread_info` descriptor into the structure pointed to by `ti`, then sets `ti->task` to `tsk`.
 - f. Sets the usage counter of the new process descriptor (`tsk->usage`) to 2 to specify that the process descriptor is in use and that the corresponding process is alive (its state is not `EXIT_ZOMBIE` or `EXIT_DEAD`).
 - g. Returns the process descriptor pointer of the new process (`tsk`).

4. Checks whether the value stored in `current->signal->rlim[RLIMIT_NPROC].rlim_cur` is smaller than or equal to the current number of processes owned by the user. If so, an error code is returned, unless the process has root privileges. The function gets the current number of processes owned by the user from a per-user data structure named `user_struct`. This data structure can be found through a pointer in the `user` field of the process descriptor.
5. Increases the usage counter of the `user_struct` structure (`tsk->user->_count` field) and the counter of the processes owned by the user (`tsk->user->processes`).
6. Checks that the number of processes in the system (stored in the `nr_threads` variable) does not exceed the value of the `max_threads` variable. The default value of this variable depends on the amount of RAM in the system. The general rule is that the space taken by all `thread_info` descriptors and Kernel Mode stacks cannot exceed 1/8 of the physical memory. However, the system administrator may change this value by writing in the `/proc/sys/kernel/threads-max` file.
7. If the kernel functions implementing the execution domain and the executable format (see [Chapter 20](#)) of the new process are included in kernel modules, it increases their usage counters (see Appendix B).
8. Sets a few crucial fields related to the process state:
 - a. Initializes the big kernel lock counter `tsk->lock_depth` to -1 (see the section "[The Big Kernel Lock](#)" in [Chapter 5](#)).
 - b. Initializes the `tsk->did_exec` field to 0: it counts the number of `execve( )` system calls issued by the process.
 - c. Updates some of the flags included in the `tsk->flags` field that have been copied from the parent process: first clears the `PF_SUPERPRIV` flag, which indicates whether the process has used any of its superuser privileges, then sets the `PF_FORKNOEXEC` flag, which indicates that the child has not yet issued an `execve( )` system call.
9. Stores the PID of the new process in the `tsk->pid` field.
10. If the `CLONE_PARENT_SETTID` flag in the `clone_flags` parameter is set, it copies the child's PID into the User Mode variable addressed by the `parent_tidptr`

parameter.

11. Initializes the `list_head` data structures and the spin locks included in the child's process descriptor, and sets up several other fields related to pending signals, timers, and time statistics.
12. Invokes `copy_semundo( )`, `copy_files( )`, `copy_fs( )`, `copy_sighand( )`, `copy_signal( )`, `copy_mm( )`, and `copy_namespace( )` to create new data structures and copy into them the values of the corresponding parent process data structures, unless specified differently by the `clone_flags` parameter.
13. Invokes `copy_thread( )` to initialize the Kernel Mode stack of the child process with the values contained in the CPU registers when the `clone( )` system call was issued (these values have been saved in the Kernel Mode stack of the parent, as described in [Chapter 10](#)). However, the function forces the value 0 into the field corresponding to the `eax` register (this is the child's return value of the `fork( )` or `clone( )` system call). The `thread.esp` field in the descriptor of the child process is initialized with the base address of the child's Kernel Mode stack, and the address of an assembly language function (`ret_from_fork( )`) is stored in the `thread.eip` field. If the parent process makes use of an I/O Permission Bitmap, the child gets a copy of such bitmap. Finally, if the `CLONE_SETTLS` flag is set, the child gets the TLS segment specified by the User Mode data structure pointed to by the `tls` parameter of the `clone( )` system call.^[*]
14. If either `CLONE_CHILD_SETTID` or `CLONE_CHILD_CLEARTID` is set in the `clone_flags` parameter, it copies the value of the `child_tidptr` parameter in the `tsk->set_child_tid` or `tsk->clear_child_tid` field, respectively. These flags specify that the value of the variable pointed to by `child_tidptr` in the User Mode address space of the child has to be changed, although the actual write operations will be done later.
15. Turns off the `TIF_SYSCALL_TRACE` flag in the `thread_info` structure of the child, so that the `ret_from_fork( )` function will not notify the debugging process about the system call termination (see the section "[Entering and Exiting a System Call](#)" in [Chapter 10](#)). (The system call tracing of the child is not disabled, because it is controlled by the `PTRACE_SYSCALL` flag in `tsk->ptrace`.)
16. Initializes the `tsk->exit_signal` field with the signal number encoded in the low bits of the `clone_flags` parameter, unless the `CLONE_THREAD` flag is set, in

which case initializes the field to -1. As we'll see in the section "[Process Termination](#)" later in this chapter, only the death of the last member of a thread group (usually, the thread group leader) causes a signal notifying the parent of the thread group leader.

17. Invokes `sched_fork( )` to complete the initialization of the scheduler data structure of the new process. The function also sets the state of the new process to `TASK_RUNNING` and sets the `preempt_count` field of the `thread_info` structure to 1, thus disabling kernel preemption (see the section "[Kernel Preemption](#)" in [Chapter 5](#)). Moreover, in order to keep process scheduling fair, the function shares the remaining timeslice of the parent between the parent and the child (see "[The scheduler_tick\(.\) Function](#)" in [Chapter 7](#)).
18. Sets the `cpu` field in the `thread_info` structure of the new process to the number of the local CPU returned by `smp_processor_id( )`.
19. Initializes the fields that specify the parenthood relationships. In particular, if `CLONE_PARENT` or `CLONE_THREAD` are set, it initializes `tsk->real_parent` and `tsk->parent` to the value in `current->real_parent`; the parent of the child thus appears as the parent of the current process. Otherwise, it sets the same fields to `current`.
20. If the child does not need to be traced (`CLONE_PTRACE` flag not set), it sets the `tsk->ptrace` field to 0. This field stores a few flags used when a process is being traced by another process. In such a way, even if the current process is being traced, the child will not.
21. Executes the `SET_LINKS` macro to insert the new process descriptor in the process list.
22. If the child must be traced (`PT_PTRACED` flag in the `tsk->ptrace` field set), it sets `tsk->parent` to `current->parent` and inserts the child into the trace list of the debugger.
23. Invokes `attach_pid( )` to insert the PID of the new process descriptor in the `pidhash[PIDTYPE_PID]` hash table.
24. If the child is a thread group leader (flag `CLONE_THREAD` cleared):
 - a. Initializes `tsk->tgid` to `tsk->pid`.
 - b. Initializes `tsk->group_leader` to `tsk`.

- c. Invokes three times `attach_pid( )` to insert the child in the PID hash tables of type `PIDTYPE_TGID`, `PIDTYPE_PGID`, and `PIDTYPE_SID`.
25. Otherwise, if the child belongs to the thread group of its parent (`CLONE_THREAD` flag set):
- a. Initializes `tsk->tgid` to `tsk->current->tgid`.
 - b. Initializes `tsk->group_leader` to the value in `current->group_leader`.
 - c. Invokes `attach_pid( )` to insert the child in the `PIDTYPE_TGID` hash table (more specifically, in the per-PID list of the `current->group_leader` process).
26. A new process has now been added to the set of processes: increases the value of the `nr_threads` variable.
27. Increases the `total_forks` variable to keep track of the number of forked processes.
28. Terminates by returning the child's process descriptor pointer (`tsk`).

Let's go back to what happens after `do_fork( )` terminates. Now we have a complete child process in the runnable state. But it isn't actually running. It is up to the scheduler to decide when to give the CPU to this child. At some future process switch, the scheduler bestows this favor on the child process by loading a few CPU registers with the values of the `thread` field of the child's process descriptor. In particular, `esp` is loaded with `thread.esp` (that is, with the address of child's Kernel Mode stack), and `eip` is loaded with the address of `ret_from_fork( )`. This assembly language function invokes the `schedule_tail( )` function (which in turn invokes the `finish_task_switch( )` function to complete the process switch; see the section "[The schedule\(.\) Function](#)" in [Chapter 7](#)), reloads all other registers with the values stored in the stack, and forces the CPU back to User Mode. The new process then starts its execution right at the end of the `fork( )`, `vfork( )`, or `clone( )` system call. The value returned by the system call is contained in `eax`: the value is 0 for the child and equal to the PID for the child's parent. To understand how this is done, look back at what `copy_thread( )` does on the `eax` register of the child's process (step 13 of `copy_process( )`).

The child process executes the same code as the parent, except that the `fork` returns a 0 (see step 13 of `copy_process( )`). The developer of the application can exploit this fact, in a manner familiar to Unix programmers, by inserting a conditional statement

in the program based on the PID value that forces the child to behave differently from the parent process.

Kernel Threads

Traditional Unix systems delegate some critical tasks to intermittently running processes, including flushing disk caches, swapping out unused pages, servicing network connections, and so on. Indeed, it is not efficient to perform these tasks in strict linear fashion; both their functions and the end user processes get better response if they are scheduled in the background. Because some of the system processes run only in Kernel Mode, modern operating systems delegate their functions to *kernel threads*, which are not encumbered with the unnecessary User Mode context. In Linux, kernel threads differ from regular processes in the following ways:

- Kernel threads run only in Kernel Mode, while regular processes run alternatively in Kernel Mode and in User Mode.
- Because kernel threads run only in Kernel Mode, they use only linear addresses greater than `PAGE_OFFSET`. Regular processes, on the other hand, use all four gigabytes of linear addresses, in either User Mode or Kernel Mode.

Creating a kernel thread

The `kernel_thread( )` function creates a new kernel thread. It receives as parameters the address of the kernel function to be executed (`fn`), the argument to be passed to that function (`arg`), and a set of clone flags (`flags`). The function essentially invokes `do_fork( )` as follows:

```
do_fork(flags|CLONE_VM|CLONE_UNTRACED, 0, pregs, 0, NULL, NULL);
```

The `CLONE_VM` flag avoids the duplication of the page tables of the calling process: this duplication would be a waste of time and memory, because the new kernel thread will not access the User Mode address space anyway. The `CLONE_UNTRACED` flag ensures that no process will be able to trace the new kernel thread, even if the calling process is being traced.

The `pregs` parameter passed to `do_fork( )` corresponds to the address in the Kernel Mode stack where the `copy_thread( )` function will find the initial values of the CPU registers for the new thread. The `kernel_thread( )` function builds up this stack area so that:

- The `ebx` and `edx` registers will be set by `copy_thread()` to the values of the parameters `fn` and `arg`, respectively.
- The `eip` register will be set to the address of the following assembly language fragment:

```
movl %edx,%eax
pushl %edx
call *%ebx
pushl %eax
call do_exit
```

Therefore, the new kernel thread starts by executing the `fn(arg)` function. If this function terminates, the kernel thread executes the `_exit( )` system call passing to it the return value of `fn( )` (see the section "[Destroying Processes](#)" later in this chapter).

Process 0

The ancestor of all processes, called *process 0*, the *idle process*, or, for historical reasons, the *swapper process*, is a kernel thread created from scratch during the initialization phase of Linux (see Appendix A). This ancestor process uses the following statically allocated data structures (data structures for all other processes are dynamically allocated):

- A process descriptor stored in the `init_task` variable, which is initialized by the `INIT_TASK` macro.
- A `thread_info` descriptor and a Kernel Mode stack stored in the `init_thread_union` variable and initialized by the `INIT_THREAD_INFO` macro.
- The following tables, which the process descriptor points to:

- `init_mm`
- `init_fs`
- `init_files`
- `init_signals`
- `init_sighand`

The tables are initialized, respectively, by the following macros:

- `INIT_MM`
- `INIT_FS`

- INIT_FILES
- INIT_SIGNALS
- INIT_SIGHAND

- The master kernel Page Global Directory stored in `swapper_pg_dir` (see the section "[Kernel Page Tables](#)" in [Chapter 2](#)).

The `start_kernel( )` function initializes all the data structures needed by the kernel, enables interrupts, and creates another kernel thread, named *process 1* (more commonly referred to as the *init process*):

```
kernel_thread(init, NULL, CLONE_FS|CLONE_SIGHAND);
```

The newly created kernel thread has PID 1 and shares all per-process kernel data structures with process 0. When selected by the scheduler, the *init* process starts executing the `init( )` function.

After having created the *init* process, process 0 executes the `cpu_idle( )` function, which essentially consists of repeatedly executing the `hlt` assembly language instruction with the interrupts enabled (see [Chapter 4](#)). Process 0 is selected by the scheduler only when there are no other processes in the `TASK_RUNNING` state.

In multiprocessor systems there is a process 0 for each CPU. Right after the power-on, the BIOS of the computer starts a single CPU while disabling the others. The *swapper* process running on CPU 0 initializes the kernel data structures, then enables the other CPUs and creates the additional *swapper* processes by means of the `copy_process( )` function passing to it the value 0 as the new PID. Moreover, the kernel sets the `cpu` field of the `thread_info` descriptor of each forked process to the proper CPU index.

Process 1

The kernel thread created by process 0 executes the `init( )` function, which in turn completes the initialization of the kernel. Then `init( )` invokes the `execve( )` system call to load the executable program *init*. As a result, the *init* kernel thread becomes a regular process having its own per-process kernel data structure (see [Chapter 20](#)). The *init* process stays alive until the system is shut down, because it creates and monitors the activity of all processes that implement the outer layers of the operating system.

Other kernel threads

Linux uses many other kernel threads. Some of them are created in the initialization phase and run until shutdown; others are created "on demand," when the kernel must execute a task that is better performed in its own execution context.

A few examples of kernel threads (besides process 0 and process 1) are:

keventd (also called events)

Executes the functions in the `keventd_wq` workqueue (see [Chapter 4](#)).

kapmd

Handles the events related to the Advanced Power Management (APM).

kswapd

Reclaims memory, as described in the section "[Periodic Reclaiming](#)" in [Chapter 17](#).

pdflush

Flushes "dirty" buffers to disk to reclaim memory, as described in the section "[The pdflush Kernel Threads](#)" in [Chapter 15](#).

kblockd

Executes the functions in the `kblockd_workqueue` workqueue. Essentially, it periodically activates the block device drivers, as described in the section "[Activating the Block Device Driver](#)" in [Chapter 14](#).

ksoftirqd

Runs the tasklets (see section "[Softirqs and Tasklets](#)" in [Chapter 4](#)); there is one of these kernel threads for each CPU in the system.

[*] The parent process might be moved on to another CPU while the kernel forks the new process.

[*] A careful reader might wonder how `copy_thread( )` gets the value of the `tls` parameter of `clone( )`, because `tls` is not passed to `do_fork( )` and nested functions. As we'll see in [Chapter 10](#), the parameters of the system calls are usually passed to the kernel by copying their values into some CPU register; thus, these values are saved in the Kernel Mode stack together with the other registers. The `copy_thread( )` function just looks at the address saved in the Kernel Mode stack location corresponding to the value of `esi`.

Destroying Processes

Most processes "die" in the sense that they terminate the execution of the code they were supposed to run. When this occurs, the kernel must be notified so that it can release the resources owned by the process; this includes memory, open files, and any other odds and ends that we will encounter in this book, such as semaphores.

The usual way for a process to terminate is to invoke the `exit( )` library function, which releases the resources allocated by the C library, executes each function registered by the programmer, and ends up invoking a system call that evicts the process from the system. The `exit( )` library function may be inserted by the programmer explicitly. Additionally, the C compiler always inserts an `exit( )` function call right after the last statement of the `main( )` function.

Alternatively, the kernel may force a whole thread group to die. This typically occurs when a process in the group has received a signal that it cannot handle or ignore (see [Chapter 11](#)) or when an unrecoverable CPU exception has been raised in Kernel Mode while the kernel was running on behalf of the process (see [Chapter 4](#)).

Process Termination

In Linux 2.6 there are two system calls that terminate a User Mode application:

- The `exit_group( )` system call, which terminates a full thread group, that is, a whole multithreaded application. The main kernel function that implements this system call is called `do_group_exit( )`. This is the system call that should be invoked by the `exit( )` C library function.
- The `_exit( )` system call, which terminates a single process, regardless of any other process in the thread group of the victim. The main kernel function that implements this system call is called `do_exit( )`. This is the system call invoked, for instance, by the `pthread_exit( )` function of the LinuxThreads library.

The `do_group_exit( )` function

The `do_group_exit( )` function kills all processes belonging to the thread group of current. It receives as a parameter the process termination code, which is either a value specified in the `exit_group( )` system call (normal termination) or an error code supplied by the kernel (abnormal termination). The function executes the following operations:

1. Checks whether the `SIGNAL_GROUP_EXIT` flag of the exiting process is not zero, which means that the kernel already started an exit procedure for this thread group. In this case, it considers as exit code the value stored in `current->signal->group_exit_code`, and jumps to step 4.
2. Otherwise, it sets the `SIGNAL_GROUP_EXIT` flag of the process and stores the termination code in the `current->signal->group_exit_code` field.
3. Invokes the `zap_other_threads( )` function to kill the other processes in the thread group of `current`, if any. In order to do this, the function scans the per-PID list in the `PIDTYPE_TGID` hash table corresponding to `current->tgid`; for each process in the list different from `current`, it sends a `SIGKILL` signal to it (see [Chapter 11](#)). As a result, all such processes will eventually execute the `do_exit( )` function, and thus they will be killed.
4. Invokes the `do_exit( )` function passing to it the process termination code. As we'll see below, `do_exit( )` kills the process and never returns.

The `do_exit( )` function

All process terminations are handled by the `do_exit( )` function, which removes most references to the terminating process from kernel data structures. The `do_exit( )` function receives as a parameter the process termination code and essentially executes the following actions:

1. Sets the `PF_EXITING` flag in the `flag` field of the process descriptor to indicate that the process is being eliminated.
2. Removes, if necessary, the process descriptor from a dynamic timer queue via the `del_timer_sync( )` function (see [Chapter 6](#)).
3. Detaches from the process descriptor the data structures related to paging, semaphores, filesystem, open file descriptors, namespaces, and I/O Permission Bitmap, respectively, with the `exit_mm( )`, `exit_sem( )`, `_exit_files( )`, `_exit_fs( )`, `exit_namespace( )`, and `exit_thread( )` functions. These functions also remove each of these data structures if no other processes are sharing them.
4. If the kernel functions implementing the execution domain and the executable format (see [Chapter 20](#)) of the process being killed are included in kernel modules, the function decreases their usage counters.

5. Sets the `exit_code` field of the process descriptor to the process termination code. This value is either the `_exit( )` or `exit_group( )` system call parameter (normal termination), or an error code supplied by the kernel (abnormal termination).
6. Invokes the `exit_notify( )` function to perform the following operations:
 - a. Updates the parenthood relationships of both the parent process and the child processes. All child processes created by the terminating process become children of another process in the same thread group, if any is running, or otherwise of the *init* process.
 - b. Checks whether the `exit_signal` process descriptor field of the process being terminated is different from -1, and whether the process is the last member of its thread group (notice that these conditions always hold for any normal process; see step 16 in the description of `copy_process( )` in the earlier section "[The clone\(\), fork\(\), and vfork\(\) System Calls](#)"). In this case, the function sends a signal (usually SIGCHLD) to the parent of the process being terminated to notify the parent about a child's death.
 - c. Otherwise, if the `exit_signal` field is equal to -1 or the thread group includes other processes, the function sends a SIGCHLD signal to the parent only if the process is being traced (in this case the parent is the debugger, which is thus informed of the death of the lightweight process).
 - d. If the `exit_signal` process descriptor field is equal to -1 and the process is not being traced, it sets the `exit_state` field of the process descriptor to EXIT_DEAD, and invokes `release_task( )` to reclaim the memory of the remaining process data structures and to decrease the usage counter of the process descriptor (see the following section). The usage counter becomes equal to 1 (see step 3f in the `copy_process( )` function), so that the process descriptor itself is not released right away.
 - e. Otherwise, if the `exit_signal` process descriptor field is not equal to -1 or the process is being traced, it sets the `exit_state` field to EXIT_ZOMBIE. We'll see what happens to zombie processes in the following section.
 - f. Sets the PF_DEAD flag in the `flags` field of the process descriptor (see the section "[The schedule\(\) Function](#)" in [Chapter 7](#)).

7. Invokes the `schedule( )` function (see [Chapter 7](#)) to select a new process to run. Because a process in an `EXIT_ZOMBIE` state is ignored by the scheduler, the process stops executing right after the `switch_to` macro in `schedule( )` is invoked. As we'll see in [Chapter 7](#), the scheduler will check the `PF_DEAD` flag and will decrease the usage counter in the descriptor of the zombie process being replaced to denote the fact that the process is no longer alive.

Process Removal

The Unix operating system allows a process to query the kernel to obtain the PID of its parent process or the execution state of any of its children. A process may, for instance, create a child process to perform a specific task and then invoke some `wait( )`-like library function to check whether the child has terminated. If the child has terminated, its termination code will tell the parent process if the task has been carried out successfully.

To comply with these design choices, Unix kernels are not allowed to discard data included in a process descriptor field right after the process terminates. They are allowed to do so only after the parent process has issued a `wait( )`-like system call that refers to the terminated process. This is why the `EXIT_ZOMBIE` state has been introduced: although the process is technically dead, its descriptor must be saved until the parent process is notified.

What happens if parent processes terminate before their children? In such a case, the system could be flooded with zombie processes whose process descriptors would stay forever in RAM. As mentioned earlier, this problem is solved by forcing all orphan processes to become children of the *init* process. In this way, the *init* process will destroy the zombies while checking for the termination of one of its legitimate children through a `wait( )`-like system call.

The `release_task( )` function detaches the last data structures from the descriptor of a zombie process; it is applied on a zombie process in two possible ways: by the `do_exit( )` function if the parent is not interested in receiving signals from the child, or by the `wait4( )` or `waitpid( )` system calls after a signal has been sent to the parent. In the latter case, the function also will reclaim the memory used by the process descriptor, while in the former case the memory reclaiming will be done by the scheduler (see [Chapter 7](#)). This function executes the following steps:

1. Decreases the number of processes belonging to the user owner of the terminated process. This value is stored in the `user_struct` structure mentioned earlier in

the chapter (see step 4 of `copy_process( )`).

2. If the process is being traced, the function removes it from the debugger's `ptrace_children` list and assigns the process back to its original parent.
3. Invokes `_exit_signal()` to cancel any pending signal and to release the `signal_struct` descriptor of the process. If the descriptor is no longer used by other lightweight processes, the function also removes this data structure. Moreover, the function invokes `exit_itimers( )` to detach any POSIX interval timer from the process.
4. Invokes `_exit_sighand()` to get rid of the signal handlers.
5. Invokes `_unhash_process( )`, which in turn:
 - a. Decreases by 1 the `nr_threads` variable.
 - b. Invokes `detach_pid( )` twice to remove the process descriptor from the `pidhash` hash tables of type `PIDTYPE_PID` and `PIDTYPE_TGID`.
 - c. If the process is a thread group leader, invokes again `detach_pid( )` twice to remove the process descriptor from the `PIDTYPE_PGID` and `PIDTYPE_SID` hash tables.
 - d. Uses the `REMOVE_LINKS` macro to unlink the process descriptor from the process list.
6. If the process is not a thread group leader, the leader is a zombie, and the process is the last member of the thread group, the function sends a signal to the parent of the leader to notify it of the death of the process.
7. Invokes the `sched_exit( )` function to adjust the timeslice of the parent process (this step logically complements step 17 in the description of `copy_process( )`).
8. Invokes `put_task_struct()` to decrease the process descriptor's usage counter; if the counter becomes zero, the function drops any remaining reference to the process:
 - a. Decreases the usage counter (`_count` field) of the `user_struct` data structure of the user that owns the process (see step 5 of `copy_process( )`), and releases that data structure if the usage counter becomes zero.
 - b. Releases the process descriptor and the memory area used to contain the `thread_info` descriptor and the Kernel Mode stack.

Chapter 5. Kernel Synchronization

You could think of the kernel as a server that answers requests; these requests can come either from a process running on a CPU or an external device issuing an interrupt request. We make this analogy to underscore that parts of the kernel are not run serially, but in an interleaved way. Thus, they can give rise to race conditions, which must be controlled through proper synchronization techniques. A general introduction to these topics can be found in the section "[An Overview of Unix Kernels](#)" in [Chapter 1](#).

We start this chapter by reviewing when, and to what extent, kernel requests are executed in an interleaved fashion. We then introduce the basic synchronization primitives implemented by the kernel and describe how they are applied in the most common cases. Finally, we illustrate a few practical examples.

How the Kernel Services Requests

To get a better grasp of how kernel's code is executed, we will look at the kernel as a waiter who must satisfy two types of requests: those issued by customers and those issued by a limited number of different bosses. The policy adopted by the waiter is the following:

1. If a boss calls while the waiter is idle, the waiter starts servicing the boss.
2. If a boss calls while the waiter is servicing a customer, the waiter stops servicing the customer and starts servicing the boss.
3. If a boss calls while the waiter is servicing another boss, the waiter stops servicing the first boss and starts servicing the second one. When he finishes servicing the new boss, he resumes servicing the former one.
4. One of the bosses may induce the waiter to leave the customer being currently serviced. After servicing the last request of the bosses, the waiter may decide to drop temporarily his customer and to pick up a new one.

The services performed by the waiter correspond to the code executed when the CPU is in Kernel Mode. If the CPU is executing in User Mode, the waiter is considered idle.

Boss requests correspond to interrupts, while customer requests correspond to system calls or exceptions raised by User Mode processes. As we shall see in detail in [Chapter 10](#), User Mode processes that want to request a service from the kernel must issue an appropriate instruction (on the 80x86, an `int $0x80` or a `sysenter` instruction). Such instructions raise an exception that forces the CPU to switch from User Mode to Kernel Mode. In the rest of this chapter, we will generally denote as "exceptions" both the system calls and the usual exceptions.

The careful reader has already associated the first three rules with the nesting of kernel control paths described in "[Nested Execution of Exception and Interrupt Handlers](#)" in [Chapter 4](#). The fourth rule corresponds to one of the most interesting new features included in the Linux 2.6 kernel, namely *kernel preemption*.

Kernel Preemption

It is surprisingly hard to give a good definition of kernel preemption. As a first try, we could say that a kernel is *preemptive* if a process switch may occur while the replaced process is executing a kernel function, that is, while it runs in Kernel Mode.

Unfortunately, in Linux (as well as in any other real operating system) things are much more complicated:

- Both in preemptive and nonpreemptive kernels, a process running in Kernel Mode can voluntarily relinquish the CPU, for instance because it has to sleep waiting for some resource. We will call this kind of process switch a *planned process switch*. However, a preemptive kernel differs from a nonpreemptive kernel on the way a process running in Kernel Mode reacts to asynchronous events that could induce a process switch—for instance, an interrupt handler that awakes a higher priority process. We will call this kind of process switch a *forced process switch*.
- All process switches are performed by the `switch_to` macro. In both preemptive and nonpreemptive kernels, a process switch occurs when a process has finished some thread of kernel activity and the scheduler is invoked. However, in nonpreemptive kernels, the current process cannot be replaced unless it is about to switch to User Mode (see the section "[Performing the Process Switch](#)" in [Chapter 3](#)).

Therefore, the main characteristic of a preemptive kernel is that a process running in Kernel Mode can be replaced by another process while in the middle of a kernel function.

Let's give a couple of examples to illustrate the difference between preemptive and nonpreemptive kernels.

While process A executes an exception handler (necessarily in Kernel Mode), a higher priority process B becomes runnable. This could happen, for instance, if an IRQ occurs and the corresponding handler awakens process B. If the kernel is preemptive, a forced process switch replaces process A with B. The exception handler is left unfinished and will be resumed only when the scheduler selects again process A for execution. Conversely, if the kernel is nonpreemptive, no process switch occurs until process A either finishes handling the exception handler or voluntarily relinquishes the CPU.

For another example, consider a process that executes an exception handler and whose time quantum expires (see the section "[The scheduler tick\(.\) Function](#)" in [Chapter 7](#)). If the kernel is preemptive, the process may be replaced immediately; however, if the kernel is nonpreemptive, the process continues to run until it finishes handling the exception handler or voluntarily relinquishes the CPU.

The main motivation for making a kernel preemptive is to reduce the *dispatch latency* of the User Mode processes, that is, the delay between the time they become runnable and the time they actually begin running. Processes performing timely scheduled tasks (such as external hardware controllers, environmental monitors, movie players, and so on) really benefit from kernel preemption, because it reduces the risk of being delayed by another process running in Kernel Mode.

Making the Linux 2.6 kernel preemptive did not require a drastic change in the kernel design with respect to the older nonpreemptive kernel versions. As described in the section "[Returning from Interrupts and Exceptions](#)" in [Chapter 4](#), kernel preemption is disabled when the `preempt_count` field in the `thread_info` descriptor referenced by the `current_thread_info( )` macro is greater than zero. The field encodes three different counters, as shown in [Table 4-10](#) in [Chapter 4](#), so it is greater than zero when any of the following cases occurs:

1. The kernel is executing an interrupt service routine.
2. The deferrable functions are disabled (always true when the kernel is executing a softirq or tasklet).
3. The kernel preemption has been explicitly disabled by setting the preemption counter to a positive value.

The above rules tell us that the kernel can be preempted only when it is executing an exception handler (in particular a system call) and the kernel preemption has not been explicitly disabled. Furthermore, as described in the section "[Returning from Interrupts and Exceptions](#)" in [Chapter 4](#), the local CPU must have local interrupts enabled, otherwise kernel preemption is not performed.

A few simple macros listed in [Table 5-1](#) deal with the preemption counter in the `preempt_count` field.

Table 5-1. Macros dealing with the preemption counter subfield

Macro	Description
<code>preempt_count()</code>	Selects the <code>preempt_count</code> field in the <code>thread_info</code> descriptor
<code>preempt_disable()</code>	Increases by one the value of the preemption counter
<code>preempt_enable_no_resched()</code>	Decreases by one the value of the preemption counter
<code>preempt_enable()</code>	Decreases by one the value of the preemption counter, and invokes <code>preempt_schedule()</code> if the <code>TIF_NEED_RESCHED</code> flag in the <code>thread_info</code> descriptor is set
<code>get_cpu()</code>	Similar to <code>preempt_disable()</code> , but also returns the number of the local CPU
<code>put_cpu()</code>	Same as <code>preempt_enable()</code>
<code>put_cpu_no_resched()</code>	Same as <code>preempt_enable_no_resched()</code>

The `preempt_enable( )` macro decreases the preemption counter, then checks whether the `TIF_NEED_RESCHED` flag is set (see [Table 4-15](#) in [Chapter 4](#)). In this case, a process switch request is pending, so the macro invokes the `preempt_schedule( )` function, which essentially executes the following code:

```
if (!current_thread_info->preempt_count && !irqs_disabled()) {
    current_thread_info->preempt_count = PREEMPT_ACTIVE;
    schedule();
    current_thread_info->preempt_count = 0;
}
```

The function checks whether local interrupts are enabled and the `preempt_count` field of `current` is zero; if both conditions are true, it invokes `schedule( )` to select another process to run. Therefore, kernel preemption may happen either when a kernel control path (usually, an interrupt handler) is terminated, or when an exception handler reenables kernel preemption by means of `preempt_enable( )`. As we'll see in the section "[Disabling and Enabling Deferrable Functions](#)" later in this chapter, kernel preemption may also happen when deferrable functions are enabled.

We'll conclude this section by noticing that kernel preemption introduces a nonnegligible overhead. For that reason, Linux 2.6 features a kernel configuration option that allows users to enable or disable kernel preemption when compiling the kernel.

When Synchronization Is Necessary

[Chapter 1](#) introduced the concepts of race condition and critical region for processes. The same definitions apply to kernel control paths. In this chapter, a race condition can occur when the outcome of a computation depends on how two or more interleaved kernel control paths are nested. A *critical region* is a section of code that must be completely executed by the kernel control path that enters it before another kernel control path can enter it.

Interleaving kernel control paths complicates the life of kernel developers: they must apply special care in order to identify the critical regions in exception handlers, interrupt handlers, deferrable functions, and kernel threads. Once a critical region has been identified, it must be suitably protected to ensure that any time at most one kernel control path is inside that region.

Suppose, for instance, that two different interrupt handlers need to access the same data structure that contains several related member variables — for instance, a buffer and an integer indicating its length. All statements affecting the data structure must be put into a single critical region. If the system includes a single CPU, the critical region can be implemented by disabling interrupts while accessing the shared data structure, because nesting of kernel control paths can only occur when interrupts are enabled.

On the other hand, if the same data structure is accessed only by the service routines of system calls, and if the system includes a single CPU, the critical region can be implemented quite simply by disabling kernel preemption while accessing the shared data structure.

As you would expect, things are more complicated in multiprocessor systems. Many CPUs may execute kernel code at the same time, so kernel developers cannot assume that a data structure can be safely accessed just because kernel preemption is disabled and the data structure is never addressed by an interrupt, exception, or softirq handler.

We'll see in the following sections that the kernel offers a wide range of different synchronization techniques. It is up to kernel designers to solve each synchronization problem by selecting the most efficient technique.

When Synchronization Is Not Necessary

Some design choices already discussed in the previous chapter simplify somewhat the synchronization of kernel control paths. Let us recall them briefly:

- All interrupt handlers acknowledge the interrupt on the PIC and also disable the IRQ line. Further occurrences of the same interrupt cannot occur until the handler terminates.
- Interrupt handlers, softirqs, and tasklets are both nonpreemptable and non-blocking, so they cannot be suspended for a long time interval. In the worst case, their execution will be slightly delayed, because other interrupts occur during their execution (nested execution of kernel control paths).
- A kernel control path performing interrupt handling cannot be interrupted by a kernel control path executing a deferrable function or a system call service routine.
- Softirqs and tasklets cannot be interleaved on a given CPU.
- The same tasklet cannot be executed simultaneously on several CPUs.

Each of the above design choices can be viewed as a constraint that can be exploited to code some kernel functions more easily. Here are a few examples of possible simplifications:

- Interrupt handlers and tasklets need not to be coded as reentrant functions.
- Per-CPU variables accessed by softirqs and tasklets only do not require synchronization.
- A data structure accessed by only one kind of tasklet does not require synchronization.

The rest of this chapter describes what to do when synchronization *is* necessary — i.e., how to prevent data corruption due to unsafe accesses to shared data structures.

Synchronization Primitives

We now examine how kernel control paths can be interleaved while avoiding race conditions among shared data. [Table 5-2](#) lists the synchronization techniques used by the Linux kernel. The "Scope" column indicates whether the synchronization technique applies to all CPUs in the system or to a single CPU. For instance, local interrupt disabling applies to just one CPU (other CPUs in the system are not affected); conversely, an atomic operation affects all CPUs in the system (atomic operations on several CPUs cannot interleave while accessing the same data structure).

Table 5-2. Various types of synchronization techniques used by the kernel

Technique	Description	Scope
Per-CPU variables	Duplicate a data structure among the CPUs	All CPUs
Atomic operation	Atomic read-modify-write instruction to a counter	All CPUs
Memory barrier	Avoid instruction reordering	Local CPU or All CPUs
Spin lock	Lock with busy wait	All CPUs
Semaphore	Lock with blocking wait (sleep)	All CPUs
Seqlocks	Lock based on an access counter	All CPUs
Local interrupt disabling	Forbid interrupt handling on a single CPU	Local CPU
Local softirq disabling	Forbid deferrable function handling on a single CPU	Local CPU
Read-copy-update (RCU)	Lock-free access to shared data structures through pointers	All CPUs

Let's now briefly discuss each synchronization technique. In the later section "[Synchronizing Accesses to Kernel Data Structures](#)," we show how these synchronization techniques can be combined to effectively protect kernel data structures.

Per-CPU Variables

The best synchronization technique consists in designing the kernel so as to avoid the need for synchronization in the first place. As we'll see, in fact, every explicit synchronization primitive has a significant performance cost.

The simplest and most efficient synchronization technique consists of declaring kernel variables as *per-CPU variables*. Basically, a per-CPU variable is an array of data structures, one element per each CPU in the system.

A CPU should not access the elements of the array corresponding to the other CPUs; on the other hand, it can freely read and modify its own element without fear of race conditions, because it is the only CPU entitled to do so. This also means, however, that the per-CPU variables can be used only in particular cases—basically, when it makes sense to logically split the data across the CPUs of the system.

The elements of the per-CPU array are aligned in main memory so that each data structure falls on a different line of the hardware cache (see the section "[Hardware Cache](#)" in [Chapter 2](#)). Therefore, concurrent accesses to the per-CPU array do not result in cache line snooping and invalidation, which are costly operations in terms of system performance.

While per-CPU variables provide protection against concurrent accesses from several CPUs, they do not provide protection against accesses from asynchronous functions (interrupt handlers and deferrable functions). In these cases, additional synchronization primitives are required.

Furthermore, per-CPU variables are prone to race conditions caused by kernel preemption, both in uniprocessor and multiprocessor systems. As a general rule, a kernel control path should access a per-CPU variable with kernel preemption disabled. Just consider, for instance, what would happen if a kernel control path gets the address of its local copy of a per-CPU variable, and then it is preempted and moved to another CPU: the address still refers to the element of the previous CPU.

[Table 5-3](#) lists the main functions and macros offered by the kernel to use per-CPU variables.

Table 5-3. Functions and macros for the per-CPU variables

Macro or function name	Description
DEFINE_PER_CPU(type, name)	Statically allocates a per-CPU array called <code>name</code> of <code>type</code> data structures
per_cpu(name, cpu)	Selects the element for CPU <code>cpu</code> of the per-CPU array <code>name</code>
__get_cpu_var(name)	Selects the local CPU's element of the per-CPU array <code>name</code>
get_cpu_var(name)	Disables kernel preemption, then selects the local CPU's element of the per-CPU array <code>name</code>
put_cpu_var(name)	Enables kernel preemption (<code>name</code> is not used)
alloc_percpu(type)	Dynamically allocates a per-CPU array of <code>type</code> data structures and returns its address
free_percpu(pointer)	Releases a dynamically allocated per-CPU array at address <code>pointer</code>
per_cpu_ptr(pointer, cpu)	Returns the address of the element for CPU <code>cpu</code> of the per-CPU array at address <code>pointer</code>

Atomic Operations

Several assembly language instructions are of type "read-modify-write" — that is, they access a memory location twice, the first time to read the old value and the second time to write a new value.

Suppose that two kernel control paths running on two CPUs try to "read-modify-write" the same memory location at the same time by executing nonatomic operations. At first, both CPUs try to read the same location, but the *memory arbiter* (a hardware circuit that serializes accesses to the RAM chips) steps in to grant access to one of them and delay the other. However, when the first read operation has completed, the delayed CPU reads exactly the same (old) value from the memory location. Both CPUs then try to write the same (new) value to the memory location; again, the bus memory access is serialized by the memory arbiter, and eventually both write operations succeed. However, the global result is incorrect because both CPUs write the same (new) value. Thus, the two interleaving "read-modify-write" operations act as a single one.

The easiest way to prevent race conditions due to "read-modify-write" instructions is by ensuring that such operations are atomic at the chip level. Every such operation must be executed in a single instruction without being interrupted in the middle and avoiding accesses to the same memory location by other CPUs. These very small *atomic operations* can be found at the base of other, more flexible mechanisms to create critical regions.

Let's review 80x86 Instructions According To That classification:

- Assembly language instructions that make zero or one aligned memory access are atomic. [\[*\]](#)
- Read-modify-write assembly language instructions (such as `inc` or `dec`) that read data from memory, update it, and write the updated value back to memory are atomic if no other processor has taken the memory bus after the read and before the write. Memory bus stealing never happens in a uniprocessor system.
- Read-modify-write assembly language instructions whose opcode is prefixed by the `lock` byte (`0xf0`) are atomic even on a multiprocessor system. When the control unit detects the prefix, it "locks" the memory bus until the instruction is finished. Therefore, other processors cannot access the memory location while the locked instruction is being executed.
- Assembly language instructions whose opcode is prefixed by a `rep` byte (`0xf2`, `0xf3`, which forces the control unit to repeat the same instruction several times) are not atomic. The control unit checks for pending interrupts before executing a new iteration.

When you write C code, you cannot guarantee that the compiler will use an atomic instruction for an operation like `a=a+1` or even for `a++`. Thus, the Linux kernel provides a special `atomic_t` type (an atomically accessible counter) and some special functions and macros (see [Table 5-4](#)) that act on `atomic_t` variables and are implemented as single, atomic assembly language instructions. On multiprocessor systems, each such instruction is prefixed by a `lock` byte.

Table 5-4. Atomic operations in Linux

Function	Description
<code>atomic_read(v)</code>	Return <code>*v</code>
<code>atomic_set(v,i)</code>	Set <code>*v</code> to <code>i</code>
<code>atomic_add(i,v)</code>	Add <code>i</code> to <code>*v</code>
<code>atomic_sub(i,v)</code>	Subtract <code>i</code> from <code>*v</code>
<code>atomic_sub_and_test(i, v)</code>	Subtract <code>i</code> from <code>*v</code> and return 1 if the result is zero; 0 otherwise
<code>atomic_inc(v)</code>	Add 1 to <code>*v</code>
<code>atomic_dec(v)</code>	Subtract 1 from <code>*v</code>
<code>atomic_dec_and_test(v)</code>	Subtract 1 from <code>*v</code> and return 1 if the result is zero; 0 otherwise
<code>atomic_inc_and_test(v)</code>	Add 1 to <code>*v</code> and return 1 if the result is zero; 0 otherwise
<code>atomic_add_negative(i, v)</code>	Add <code>i</code> to <code>*v</code> and return 1 if the result is negative; 0 otherwise
<code>atomic_inc_return(v)</code>	Add 1 to <code>*v</code> and return the new value of <code>*v</code>
<code>atomic_dec_return(v)</code>	Subtract 1 from <code>*v</code> and return the new value of <code>*v</code>
<code>atomic_add_return(i, v)</code>	Add <code>i</code> to <code>*v</code> and return the new value of <code>*v</code>
<code>atomic_sub_return(i, v)</code>	Subtract <code>i</code> from <code>*v</code> and return the new value of <code>*v</code>

Another class of atomic functions operate on bit masks (see [Table 5-5](#)).

Table 5-5. Atomic bit handling functions in Linux

Function	Description
<code>test_bit(nr, addr)</code>	Return the value of the nr^{th} bit of <code>*addr</code>
<code>set_bit(nr, addr)</code>	Set the nr^{th} bit of <code>*addr</code>
<code>clear_bit(nr, addr)</code>	Clear the nr^{th} bit of <code>*addr</code>
<code>change_bit(nr, addr)</code>	Invert the nr^{th} bit of <code>*addr</code>
<code>test_and_set_bit(nr, addr)</code>	Set the nr^{th} bit of <code>*addr</code> and return its old value
<code>test_and_clear_bit(nr, addr)</code>	Clear the nr^{th} bit of <code>*addr</code> and return its old value
<code>test_and_change_bit(nr, addr)</code>	Invert the nr^{th} bit of <code>*addr</code> and return its old value
<code>atomic_clear_mask(mask, addr)</code>	Clear all bits of <code>*addr</code> specified by <code>mask</code>
<code>atomic_set_mask(mask, addr)</code>	Set all bits of <code>*addr</code> specified by <code>mask</code>

Optimization and Memory Barriers

When using optimizing compilers, you should never take for granted that instructions will be performed in the exact order in which they appear in the source code. For example, a compiler might reorder the assembly language instructions in such a way to optimize how registers are used. Moreover, modern CPUs usually execute several instructions in parallel and might reorder memory accesses. These kinds of reordering can greatly speed up the program.

When dealing with synchronization, however, reordering instructions must be avoided. Things would quickly become hairy if an instruction placed after a synchronization primitive is executed before the synchronization primitive itself. Therefore, all synchronization primitives act as optimization and memory barriers .

An *optimization barrier* primitive ensures that the assembly language instructions corresponding to C statements placed before the primitive are not mixed by the compiler with assembly language instructions corresponding to C statements placed after the primitive. In Linux the `barrier( )` macro, which expands into `asm volatile("":::"memory")`, acts as an optimization barrier. The `asm` instruction tells the compiler to insert an assembly language fragment (empty, in this case). The `volatile` keyword forbids the compiler to reshuffle the `asm` instruction with the other

instructions of the program. The `memory` keyword forces the compiler to assume that all memory locations in RAM have been changed by the assembly language instruction; therefore, the compiler cannot optimize the code by using the values of memory locations stored in CPU registers before the `asm` instruction. Notice that the optimization barrier does not ensure that the executions of the assembly language instructions are not mixed by the CPU—this is a job for a memory barrier.

A *memory barrier* primitive ensures that the operations placed before the primitive are finished before starting the operations placed after the primitive. Thus, a memory barrier is like a firewall that cannot be passed by an assembly language instruction.

In the 80×86 processors, the following kinds of assembly language instructions are said to be "serializing" because they act as memory barriers:

- All instructions that operate on I/O ports
- All instructions prefixed by the `lock` byte (see the section "[Atomic Operations](#)")
- All instructions that write into control registers, system registers, or debug registers (for instance, `cli` and `sti`, which change the status of the IF flag in the `eflags` register)
- The `lfence`, `sfence`, and `mfence` assembly language instructions, which have been introduced in the Pentium 4 microprocessor to efficiently implement read memory barriers, write memory barriers, and read-write memory barriers, respectively.
- A few special assembly language instructions; among them, the `iret` instruction that terminates an interrupt or exception handler

Linux uses a few memory barrier primitives, which are shown in [Table 5-6](#). These primitives act also as optimization barriers, because we must make sure the compiler does not move the assembly language instructions around the barrier. "Read memory barriers" act only on instructions that read from memory, while "write memory barriers" act only on instructions that write to memory. Memory barriers can be useful in both multiprocessor and uniprocessor systems. The `smp_XXX()` primitives are used whenever the memory barrier should prevent race conditions that might occur only in multiprocessor systems; in uniprocessor systems, they do nothing. The other memory barriers are used to prevent race conditions occurring both in uniprocessor and multiprocessor systems.

Table 5-6. Memory barriers in Linux

Macro	Description
<code>mb()</code>	Memory barrier for MP and UP
<code>rmb()</code>	Read memory barrier for MP and UP
<code>wmb()</code>	Write memory barrier for MP and UP
<code>smp_mb()</code>	Memory barrier for MP only
<code>smp_rmb()</code>	Read memory barrier for MP only
<code>smp_wmb()</code>	Write memory barrier for MP only

The implementations of the memory barrier primitives depend on the architecture of the system. On an 80×86 microprocessor, the `rmb( )` macro usually expands into `asm volatile("lfence")` if the CPU supports the `lfence` assembly language instruction, or into `asm volatile("lock;addl $0,0(%%esp)"::"memory")` otherwise. The `asm` statement inserts an assembly language fragment in the code generated by the compiler and acts as an optimization barrier. The `lock; addl $0,0(%%esp)` assembly language instruction adds zero to the memory location on top of the stack; the instruction is useless by itself, but the `lock` prefix makes the instruction a memory barrier for the CPU.

The `wmb( )` macro is actually simpler because it expands into `barrier( )`. This is because existing Intel microprocessors never reorder write memory accesses, so there is no need to insert a serializing assembly language instruction in the code. The macro, however, forbids the compiler from shuffling the instructions.

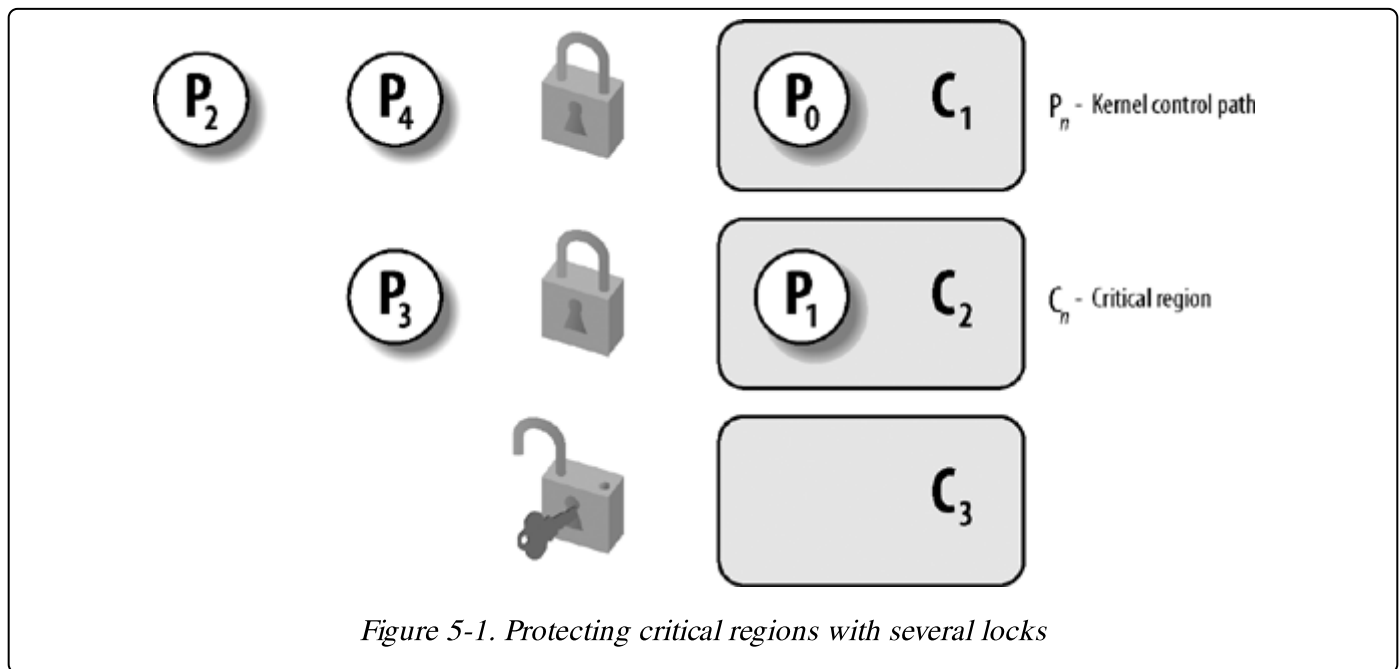
Notice that in multiprocessor systems, all atomic operations described in the earlier section "[Atomic Operations](#)" act as memory barriers because they use the `lock` byte.

Spin Locks

A widely used synchronization technique is *locking*. When a kernel control path must access a shared data structure or enter a critical region, it needs to acquire a "lock" for it. A resource protected by a locking mechanism is quite similar to a resource confined in a room whose door is locked when someone is inside. If a kernel control path wishes to access the resource, it tries to "open the door" by acquiring the lock. It succeeds only if the resource is free. Then, as long as it wants to use the resource, the

door remains locked. When the kernel control path releases the lock, the door is unlocked and another kernel control path may enter the room.

[Figure 5-1](#) illustrates the use of locks. Five kernel control paths (P_0 , P_1 , P_2 , P_3 , and P_4) are trying to access two critical regions (C_1 and C_2). Kernel control path P_0 is inside C_1 , while P_2 and P_4 are waiting to enter it. At the same time, P_1 is inside C_2 , while P_3 is waiting to enter it. Notice that P_0 and P_1 could run concurrently. The lock for critical region C_3 is open because no kernel control path needs to enter it.



Spin locks are a special kind of lock designed to work in a multiprocessor environment. If the kernel control path finds the spin lock "open," it acquires the lock and continues its execution. Conversely, if the kernel control path finds the lock "closed" by a kernel control path running on another CPU, it "spins" around, repeatedly executing a tight instruction loop, until the lock is released.

The instruction loop of spin locks represents a "busy wait." The waiting kernel control path keeps running on the CPU, even if it has nothing to do besides waste time. Nevertheless, spin locks are usually convenient, because many kernel resources are locked for a fraction of a millisecond only; therefore, it would be far more time-consuming to release the CPU and reacquire it later.

As a general rule, kernel preemption is disabled in every critical region protected by spin locks. In the case of a uniprocessor system, the locks themselves are useless, and the spin lock primitives just disable or enable the kernel preemption. Please notice

that kernel preemption is still enabled during the busy wait phase, thus a process waiting for a spin lock to be released could be replaced by a higher priority process. In Linux, each spin lock is represented by a `spinlock_t` structure consisting of two fields:

slock

Encodes the spin lock state: the value 1 corresponds to the unlocked state, while every negative value and 0 denote the locked state

break_lock

Flag signaling that a process is busy waiting for the lock (present only if the kernel supports both SMP and kernel preemption)

Six macros shown in [Table 5-7](#) are used to initialize, test, and set spin locks. All these macros are based on atomic operations; this ensures that the spin lock will be updated properly even when multiple processes running on different CPUs try to modify the lock at the same time.^[*]

Table 5-7. Spin lock macros

Macro	Description
<code>spin_lock_init()</code>	Set the spin lock to 1 (unlocked)
<code>spin_lock()</code>	Cycle until spin lock becomes 1 (unlocked), then set it to 0 (locked)
<code>spin_unlock()</code>	Set the spin lock to 1 (unlocked)
<code>spin_unlock_wait()</code>	Wait until the spin lock becomes 1 (unlocked)
<code>spin_is_locked()</code>	Return 0 if the spin lock is set to 1 (unlocked); 1 otherwise
<code>spin_trylock()</code>	Set the spin lock to 0 (locked), and return 1 if the previous value of the lock was 1; 0 otherwise

The `spin_lock` macro with kernel preemption

Let's discuss in detail the `spin_lock` macro, which is used to acquire a spin lock. The following description refers to a preemptive kernel for an SMP system. The macro takes the address `slp` of the spin lock as its parameter and executes the following actions:

1. Invokes `preempt_disable( )` to disable kernel preemption.

2. Invokes the `_raw_spin_trylock( )` function, which does an atomic test-and-set operation on the spin lock's `slock` field; this function executes first some instructions equivalent to the following assembly language fragment:

```
movb $0, %al
xchgb %al, slp->slock
```

The `xchg` assembly language instruction exchanges atomically the content of the 8-bit `%al` register (storing zero) with the content of the memory location pointed to by `slp->slock`. The function then returns the value 1 if the old value stored in the spin lock (in `%al` after the `xchg` instruction) was positive, the value 0 otherwise.

3. If the old value of the spin lock was positive, the macro terminates: the kernel control path has acquired the spin lock.
4. Otherwise, the kernel control path failed in acquiring the spin lock, thus the macro must cycle until the spin lock is released by a kernel control path running on some other CPU. Invokes `preempt_enable( )` to undo the increase of the preemption counter done in step 1. If kernel preemption was enabled before executing the `spin_lock` macro, another process can now replace this process while it waits for the spin lock.
5. If the `break_lock` field is equal to zero, sets it to one. By checking this field, the process owning the lock and running on another CPU can learn whether there are other processes waiting for the lock. If a process holds a spin lock for a long time, it may decide to release it prematurely to allow another process waiting for the same spin lock to progress.
6. Executes the wait cycle:

```
while (spin_is_locked(slp) && slp->break_lock)
    cpu_relax();
```

The `cpu_relax( )` macro reduces to a `pause` assembly language instruction. This instruction has been introduced in the Pentium 4 model to optimize the execution of spin lock loops. By introducing a short delay, it speeds up the execution of code following the lock and reduces power consumption. The `pause` instruction is backward compatible with earlier models of 80x86 microprocessors because it corresponds to the instruction `rep;nop`, that is, to a no-operation.

7. Jumps back to step 1 to try once more to get the spin lock.

The spin_lock macro without kernel preemption

If the kernel preemption option has not been selected when the kernel was compiled, the `spin_lock` macro is quite different from the one described above. In this case, the macro yields an assembly language fragment that is essentially equivalent to the following tight busy wait:^[*]

```
1: lock; decb slp->slock
   jns 3f
2: pause
   cmpb $0,slp->slock
   jle 2b
   jmp 1b
3:
```

The `decb` assembly language instruction decreases the spin lock value; the instruction is atomic because it is prefixed by the `lock` byte. A test is then performed on the sign flag. If it is clear, it means that the spin lock was set to 1 (unlocked), so normal execution continues at label 3 (the `f` suffix denotes the fact that the label is a "forward" one; it appears in a later line of the program). Otherwise, the tight loop at label 2 (the `b` suffix denotes a "backward" label) is executed until the spin lock assumes a positive value. Then execution restarts from label 1, since it is unsafe to proceed without checking whether another processor has grabbed the lock.

The spin_unlock macro

The `spin_unlock` macro releases a previously acquired spin lock; it essentially executes the assembly language instruction:

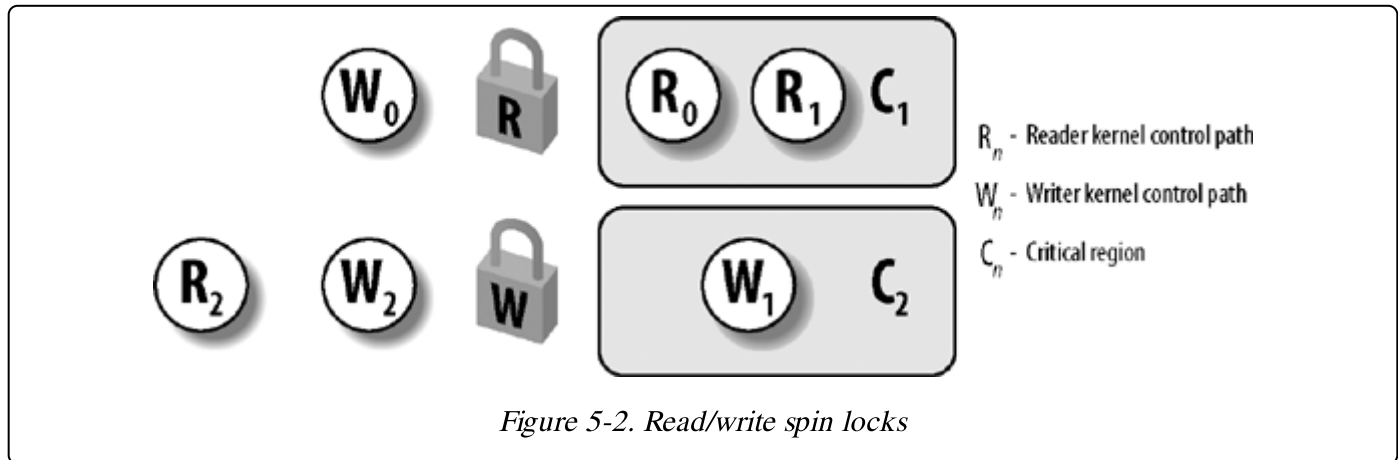
```
movb $1, slp->slock
```

and then invokes `preempt_enable( )` (if kernel preemption is not supported, `preempt_enable( )` does nothing). Notice that the `lock` byte is not used because write-only accesses in memory are always atomically executed by the current 80x86 microprocessors.

Read/Write Spin Locks

Read/write spin locks have been introduced to increase the amount of concurrency inside the kernel. They allow several kernel control paths to simultaneously read the same data structure, as long as no kernel control path modifies it. If a kernel control path wishes to write to the structure, it must acquire the write version of the read/write lock, which grants exclusive access to the resource. Of course, allowing concurrent reads on data structures improves system performance.

[Figure 5-2](#) illustrates two critical regions (C1 and C2) protected by read/write locks. Kernel control paths R0 and R1 are reading the data structures in C1 at the same time, while W0 is waiting to acquire the lock for writing. Kernel control path W1 is writing the data structures in C2, while both R2 and W2 are waiting to acquire the lock for reading and writing, respectively.



Each read/write spin lock is a `rwlock_t` structure; its `lock` field is a 32-bit field that encodes two distinct pieces of information:

- A 24-bit counter denoting the number of kernel control paths currently reading the protected data structure. The two's complement value of this counter is stored in bits 0–23 of the field.
- An unlock flag that is set when no kernel control path is reading or writing, and clear otherwise. This unlock flag is stored in bit 24 of the field.

Notice that the `lock` field stores the number `0x01000000` if the spin lock is idle (unlock flag set and no readers), the number `0x00000000` if it has been acquired for writing (unlock flag clear and no readers), and any number in the sequence `0x00ffffff`, `0x00fffffe`, and so on, if it has been acquired for reading by one, two, or more processes (unlock flag clear and the two's complement on 24 bits of the number of readers). As the `spinlock_t` structure, the `rwlock_t` structure also includes a `break_lock` field.

The `rwlock_init` macro initializes the `lock` field of a read/write spin lock to `0x01000000` (unlocked) and the `break_lock` field to zero.

Getting and releasing a lock for reading

The `read_lock` macro, applied to the address `rwlp` of a read/write spin lock, is similar to the `spin_lock` macro described in the previous section. If the kernel preemption

option has been selected when the kernel was compiled, the macro performs the very same actions as those of `spin_lock( )`, with just one exception: to effectively acquire the read/write spin lock in step 2, the macro executes the `_raw_read_trylock( )` function:

```
int _raw_read_trylock(rwlock_t *lock)
{
    atomic_t *count = (atomic_t *)lock->lock;
    atomic_dec(count);
    if (atomic_read(count) >= 0)
        return 1;
    atomic_inc(count);
    return 0;
}
```

The `lock` field—the read/write lock counter—is accessed by means of atomic operations. Notice, however, that the whole function does not act atomically on the counter: for instance, the counter might change after having tested its value with the `if` statement and before returning 1. Nevertheless, the function works properly: in fact, the function returns 1 only if the counter was not zero or negative before the decrement, because the counter is equal to `0x01000000` for no owner, `0x00ffffff` for one reader, and `0x00000000` for one writer.

If the kernel preemption option has not been selected when the kernel was compiled, the `read_lock` macro yields the following assembly language code:

```
    movl $rwlp->lock,%eax
    lock; subl $1,(%eax)
    jns 1f
    call __read_lock_failed
1:
```

where `__read_lock_failed( )` is the following assembly language function:

```
__read_lock_failed:
    lock; incl (%eax)
1:  pause
    cmpl $1,(%eax)
    js 1b
    lock; decl (%eax)
    js __read_lock_failed
    ret
```

The `read_lock` macro atomically decreases the spin lock value by 1, thus increasing the number of readers. The spin lock is acquired if the decrement operation yields a nonnegative value; otherwise, the `__read_lock_failed( )` function is invoked. The function atomically increases the `lock` field to undo the decrement operation performed by the `read_lock` macro, and then loops until the field becomes positive

(greater than or equal to 1). Next, `__read_lock_failed( )` tries to get the spin lock again (another kernel control path could acquire the spin lock for writing right after the `cmpl` instruction).

Releasing the read lock is quite simple, because the `read_unlock` macro must simply increase the counter in the `lock` field with the assembly language instruction:

```
lock; incl rwp->lock
```

to decrease the number of readers, and then invoke `preempt_enable( )` to reenables kernel preemption.

Getting and releasing a lock for writing

The `write_lock` macro is implemented in the same way as `spin_lock( )` and `read_lock( )`. For instance, if kernel preemption is supported, the function disables kernel preemption and tries to grab the lock right away by invoking `_raw_write_trylock( )`. If this function returns 0, the lock was already taken, thus the macro reenables kernel preemption and starts a busy wait loop, as explained in the description of `spin_lock( )` in the previous section.

The `_raw_write_trylock( )` function is shown below:

```
int _raw_write_trylock(rwlock_t *lock)
{
    atomic_t *count = (atomic_t *)lock->lock;
    if (atomic_sub_and_test(0x01000000, count))
        return 1;
    atomic_add(0x01000000, count);
    return 0;
}
```

The `_raw_write_trylock( )` function subtracts `0x01000000` from the read/write spin lock value, thus clearing the unlock flag (bit 24). If the subtraction operation yields zero (no readers), the lock is acquired and the function returns 1; otherwise, the function atomically adds `0x01000000` to the spin lock value to undo the subtraction operation.

Once again, releasing the write lock is much simpler because the `write_unlock` macro must simply set the unlock flag in the `lock` field with the assembly language instruction:

```
lock; addl $0x01000000,rwp
```

and then invoke `preempt_enable( )`.

Seqlocks

When using read/write spin locks, requests issued by kernel control paths to perform a `read_lock` or a `write_lock` operation have the same priority: readers must wait until the writer has finished and, similarly, a writer must wait until all readers have finished.

Seqlocks introduced in Linux 2.6 are similar to read/write spin locks, except that they give a much higher priority to writers: in fact a writer is allowed to proceed even when readers are active. The good part of this strategy is that a writer never waits (unless another writer is active); the bad part is that a reader may sometimes be forced to read the same data several times until it gets a valid copy.

Each seqlock is a `seqlock_t` structure consisting of two fields: a `lock` field of type `spinlock_t` and an integer `sequence` field. This second field plays the role of a sequence counter. Each reader must read this sequence counter twice, before and after reading the data, and check whether the two values coincide. In the opposite case, a new writer has become active and has increased the sequence counter, thus implicitly telling the reader that the data just read is not valid.

A `seqlock_t` variable is initialized to "unlocked" either by assigning to it the value `SEQLOCK_UNLOCKED`, or by executing the `seqlock_init` macro. Writers acquire and release a seqlock by invoking `write_seqlock( )` and `write_sequnlock( )`. The first function acquires the spin lock in the `seqlock_t` data structure, then increases the sequence counter by one; the second function increases the sequence counter once more, then releases the spin lock. This ensures that when the writer is in the middle of writing, the counter is odd, and that when no writer is altering data, the counter is even. Readers implement a critical region as follows:

```
unsigned int seq;
do {
    seq = read_seqbegin(&seqlock);
    /* ... CRITICAL REGION ... */
} while (read_seqretry(&seqlock, seq));
```

`read_seqbegin( )` returns the current sequence number of the seqlock;

`read_seqretry( )` returns 1 if either the value of the `seq` local variable is odd (a writer was updating the data structure when the `read_seqbegin( )` function has been invoked), or if the value of `seq` does not match the current value of the seqlock's sequence counter (a writer started working while the reader was still executing the code in the critical region).

Notice that when a reader enters a critical region, it does not need to disable kernel preemption; on the other hand, the writer automatically disables kernel preemption when entering the critical region, because it acquires the spin lock.

Not every kind of data structure can be protected by a seqlock. As a general rule, the following conditions must hold:

- The data structure to be protected does not include pointers that are modified by the writers and dereferenced by the readers (otherwise, a writer could change the pointer under the nose of the readers)
- The code in the critical regions of the readers does not have side effects (otherwise, multiple reads would have different effects from a single read)

Furthermore, the critical regions of the readers should be short and writers should seldom acquire the seqlock, otherwise repeated read accesses would cause a severe overhead. A typical usage of seqlocks in Linux 2.6 consists of protecting some data structures related to the system time handling (see [Chapter 6](#)).

Read-Copy Update (RCU)

Read-copy update (RCU) is yet another synchronization technique designed to protect data structures that are mostly accessed for reading by several CPUs. RCU allows many readers and many writers to proceed concurrently (an improvement over seqlocks, which allow only one writer to proceed). Moreover, RCU is lock-free, that is, it uses no lock or counter shared by all CPUs; this is a great advantage over read/write spin locks and seqlocks, which have a high overhead due to cache line-snooping and invalidation.

How does RCU obtain the surprising result of synchronizing several CPUs without shared data structures? The key idea consists of limiting the scope of RCU as follows:

1. Only data structures that are dynamically allocated and referenced by means of pointers can be protected by RCU.
2. No kernel control path can sleep inside a critical region protected by RCU.

When a kernel control path wants to read an RCU-protected data structure, it executes the `rcu_read_lock( )` macro, which is equivalent to `preempt_disable( )`. Next, the reader dereferences the pointer to the data structure and starts reading it. As stated above, the reader cannot sleep until it finishes reading the data structure; the end of the critical region is marked by the `rcu_read_unlock( )` macro, which is equivalent to `preempt_enable( )`.

Because the reader does very little to prevent race conditions, we could expect that the writer has to work a bit more. In fact, when a writer wants to update the data

structure, it dereferences the pointer and makes a copy of the whole data structure. Next, the writer modifies the copy. Once finished, the writer changes the pointer to the data structure so as to make it point to the updated copy. Because changing the value of the pointer is an atomic operation, each reader or writer sees either the old copy or the new one: no corruption in the data structure may occur. However, a memory barrier is required to ensure that the updated pointer is seen by the other CPUs only after the data structure has been modified. Such a memory barrier is implicitly introduced if a spin lock is coupled with RCU to forbid the concurrent execution of writers.

The real problem with the RCU technique, however, is that the old copy of the data structure cannot be freed right away when the writer updates the pointer. In fact, the readers that were accessing the data structure when the writer started its update could still be reading the old copy. The old copy can be freed only after all (potential) readers on the CPUs have executed the `rcu_read_unlock( )` macro. The kernel requires every potential reader to execute that macro before:

- The CPU performs a process switch (see restriction 2 earlier).
- The CPU starts executing in User Mode.
- The CPU executes the idle loop (see the section "[Kernel Threads](#)" in [Chapter 3](#)).

In each of these cases, we say that the CPU has gone through a *quiescent state*.

The `call_rcu( )` function is invoked by the writer to get rid of the old copy of the data structure. It receives as its parameters the address of an `rcu_head` descriptor (usually embedded inside the data structure to be freed) and the address of a *callback* function to be invoked when all CPUs have gone through a quiescent state. Once executed, the callback function usually frees the old copy of the data structure.

The `call_rcu( )` function stores in the `rcu_head` descriptor the address of the callback and its parameter, then inserts the descriptor in a per-CPU list of callbacks. Periodically, once every tick (see the section "[Updating Local CPU Statistics](#)" in [Chapter 6](#)), the kernel checks whether the local CPU has gone through a quiescent state. When all CPUs have gone through a quiescent state, a local tasklet—whose descriptor is stored in the `rcu_tasklet` per-CPU variable—executes all callbacks in the list.

RCU is a new addition in Linux 2.6; it is used in the networking layer and in the Virtual Filesystem.

Semaphores

We have already introduced semaphores in the section "[Synchronization and Critical Regions](#)" in [Chapter 1](#). Essentially, they implement a locking primitive that allows waiters to sleep until the desired resource becomes free.

Actually, Linux offers two kinds of semaphores:

- Kernel semaphores, which are used by kernel control paths
- System V IPC semaphores, which are used by User Mode processes

In this section, we focus on kernel semaphores, while IPC semaphores are described in [Chapter 19](#).

A kernel semaphore is similar to a spin lock, in that it doesn't allow a kernel control path to proceed unless the lock is open. However, whenever a kernel control path tries to acquire a busy resource protected by a kernel semaphore, the corresponding process is suspended. It becomes runnable again when the resource is released. Therefore, kernel semaphores can be acquired only by functions that are allowed to sleep; interrupt handlers and deferrable functions cannot use them.

A kernel semaphore is an object of type `struct semaphore`, containing the fields shown in the following list.

`count`

Stores an `atomic_t` value. If it is greater than 0, the resource is free — that is, it is currently available. If `count` is equal to 0, the semaphore is busy but no other process is waiting for the protected resource. Finally, if `count` is negative, the resource is unavailable and at least one process is waiting for it.

`wait`

Stores the address of a wait queue list that includes all sleeping processes that are currently waiting for the resource. Of course, if `count` is greater than or equal to 0, the wait queue is empty.

`sleepers`

Stores a flag that indicates whether some processes are sleeping on the semaphore. We'll see this field in operation soon.

The `init_MUTEX( )` and `init_MUTEX_LOCKED( )` functions may be used to initialize a semaphore for exclusive access: they set the `count` field to 1 (free resource with exclusive access) and 0 (busy resource with exclusive access currently granted to the

process that initializes the semaphore), respectively. The `DECLARE_MUTEX` and `DECLARE_MUTEX_LOCKED` macros do the same, but they also statically allocate the `struct semaphore` variable. Note that a semaphore could also be initialized with an arbitrary positive value n for `count`. In this case, at most n processes are allowed to concurrently access the resource.

Getting and releasing semaphores

Let's start by discussing how to release a semaphore, which is much simpler than getting one. When a process wishes to release a kernel semaphore lock, it invokes the `up( )` function. This function is essentially equivalent to the following assembly language fragment:

```
    movl $sem->count,%ecx
    lock; incl (%ecx)
    jg 1f
    lea %ecx,%eax
    pushl %edx
    pushl %ecx
    call __up
    popl %ecx
    popl %edx
1:
```

where `__up( )` is the following C function:

```
__attribute__((regparm(3))) void __up(struct semaphore *sem)
{
    wake_up(&sem->wait);
}
```

The `up( )` function increases the `count` field of the `*sem` semaphore, and then it checks whether its value is greater than 0. The increment of `count` and the setting of the flag tested by the following jump instruction must be atomically executed, or else another kernel control path could concurrently access the field value, with disastrous results. If `count` is greater than 0, there was no process sleeping in the wait queue, so nothing has to be done. Otherwise, the `__up( )` function is invoked so that one sleeping process is woken up. Notice that `__up( )` receives its parameter from the `eax` register (see the description of the `__switch_to( )` function in the section "[Performing the Process Switch](#)" in [Chapter 3](#)).

Conversely, when a process wishes to acquire a kernel semaphore lock, it invokes the `down( )` function. The implementation of `down( )` is quite involved, but it is essentially equivalent to the following:

```
down:
    movl $sem->count,%ecx
```

```

lock; decl (%ecx);
jns 1f
lea %ecx, %eax
pushl %edx
pushl %ecx
call __down
popl %ecx
popl %edx
1:

```

where `__down( )` is the following C function:

```

__attribute__((regparm(3))) void __down(struct semaphore * sem)
{
    DECLARE_WAITQUEUE(wait, current);
    unsigned long flags;
    current->state = TASK_UNINTERRUPTIBLE;
    spin_lock_irqsave(&sem->wait.lock, flags);
    add_wait_queue_exclusive_locked(&sem->wait, &wait);
    sem->sleepers++;
    for (;;) {
        if (!atomic_add_negative(sem->sleepers-1, &sem->count)) {
            sem->sleepers = 0;
            break;
        }
        sem->sleepers = 1;
        spin_unlock_irqrestore(&sem->wait.lock, flags);
        schedule( );
        spin_lock_irqsave(&sem->wait.lock, flags);
        current->state = TASK_UNINTERRUPTIBLE;
    }
    remove_wait_queue_locked(&sem->wait, &wait);
    wake_up_locked(&sem->wait);
    spin_unlock_irqrestore(&sem->wait.lock, flags);
    current->state = TASK_RUNNING;
}

```

The `down( )` function decreases the `count` field of the `*sem` semaphore, and then checks whether its value is negative. Again, the decrement and the test must be atomically executed. If `count` is greater than or equal to 0, the current process acquires the resource and the execution continues normally. Otherwise, `count` is negative, and the current process must be suspended. The contents of some registers are saved on the stack, and then `__down( )` is invoked.

Essentially, the `__down( )` function changes the state of the current process from `TASK_RUNNING` to `TASK_UNINTERRUPTIBLE`, and it puts the process in the semaphore wait queue. Before accessing the fields of the `semaphore` structure, the function also gets the `sem->wait.lock` spin lock that protects the semaphore wait queue (see "[How Processes Are Organized](#)" in [Chapter 3](#)) and disables local interrupts. Usually, wait queue functions get and release the wait queue spin lock as necessary when inserting

and deleting an element. The `_down()` function, however, uses the wait queue spin lock also to protect the other fields of the `semaphore` data structure, so that no process running on another CPU is able to read or modify them. To that end, `_down()` uses the "`_locked`" versions of the wait queue functions, which assume that the spin lock has been already acquired before their invocations.

The main task of the `_down()` function is to suspend the current process until the semaphore is released. However, the way in which this is done is quite involved. To easily understand the code, keep in mind that the `sleepers` field of the semaphore is usually set to 0 if no process is sleeping in the wait queue of the semaphore, and it is set to 1 otherwise. Let's try to explain the code by considering a few typical cases.

MUTEX semaphore open (count equal to 1, sleepers equal to 0)

The `down` macro just sets the `count` field to 0 and jumps to the next instruction of the main program; therefore, the `_down()` function is not executed at all.

MUTEX semaphore closed, no sleeping processes (count equal to 0, sleepers equal to 0)

The `down` macro decreases `count` and invokes the `_down()` function with the `count` field set to -1 and the `sleepers` field set to 0. In each iteration of the loop, the function checks whether the `count` field is negative. (Observe that the `count` field is not changed by `atomic_add_negative()` because `sleepers` is equal to 0 when the function is invoked.)

- If the `count` field is negative, the function invokes `schedule()` to suspend the current process. The `count` field is still set to -1, and the `sleepers` field to 1. The process picks up its run subsequently inside this loop and issues the test again.
- If the `count` field is not negative, the function sets `sleepers` to 0 and exits from the loop. It tries to wake up another process in the semaphore wait queue (but in our scenario, the queue is now empty) and terminates holding the semaphore. On exit, both the `count` field and the `sleepers` field are set to 0, as required when the semaphore is closed but no process is waiting for it.

MUTEX semaphore closed, other sleeping processes (count equal to -1, sleepers equal to 1)

The `down` macro decreases `count` and invokes the `_down()` function with `count` set to -2 and `sleepers` set to 1. The function temporarily sets `sleepers` to

2, and then undoes the decrement performed by the `down` macro by adding the value `sleepers-1` to `count`. At the same time, the function checks whether `count` is still negative (the semaphore could have been released by the holding process right before `__down( )` entered the critical region).

- If the `count` field is negative, the function resets `sleepers` to 1 and invokes `schedule( )` to suspend the current process. The `count` field is still set to -1, and the `sleepers` field to 1.
- If the `count` field is not negative, the function sets `sleepers` to 0, tries to wake up another process in the semaphore wait queue, and exits holding the semaphore. On exit, the `count` field is set to 0 and the `sleepers` field to 0. The values of both fields look wrong, because there are other sleeping processes. However, consider that another process in the wait queue has been woken up. This process does another iteration of the loop; the `atomic_add_negative( )` function subtracts 1 from `count`, restoring it to -1; moreover, before returning to sleep, the woken-up process resets `sleepers` to 1.

So, the code properly works in all cases. Consider that the `wake_up( )` function in `__down( )` wakes up at most one process, because the sleeping processes in the wait queue are exclusive (see the section "[How Processes Are Organized](#)" in [Chapter 3](#)).

Only exception handlers, and particularly system call service routines, can use the `down( )` function. Interrupt handlers or deferrable functions must not invoke `down( )`, because this function suspends the process when the semaphore is busy. For this reason, Linux provides the `down_trylock( )` function, which may be safely used by one of the previously mentioned asynchronous functions. It is identical to `down( )` except when the resource is busy. In this case, the function returns immediately instead of putting the process to sleep.

A slightly different function called `down_interruptible( )` is also defined. It is widely used by device drivers, because it allows processes that receive a signal while being blocked on a semaphore to give up the "down" operation. If the sleeping process is woken up by a signal before getting the needed resource, the function increases the `count` field of the semaphore and returns the value `-EINTR`. On the other hand, if `down_interruptible( )` runs to normal completion and gets the resource, it returns 0. The device driver may thus abort the I/O operation when the return value is `-EINTR`.

Finally, because processes usually find semaphores in an open state, the semaphore functions are optimized for this case. In particular, the `up( )` function does not

execute jump instructions if the semaphore wait queue is empty; similarly, the `down()` function does not execute jump instructions if the semaphore is open. Much of the complexity of the semaphore implementation is precisely due to the effort of avoiding costly instructions in the main branch of the execution flow.

Read/Write Semaphores

Read/write semaphores are similar to the read/write spin locks described earlier in the section "[Read/Write Spin Locks](#)," except that waiting processes are suspended instead of spinning until the semaphore becomes open again.

Many kernel control paths may concurrently acquire a read/write semaphore for reading; however, every writer kernel control path must have exclusive access to the protected resource. Therefore, the semaphore can be acquired for writing only if no other kernel control path is holding it for either read or write access. Read/write semaphores improve the amount of concurrency inside the kernel and improve overall system performance.

The kernel handles all processes waiting for a read/write semaphore in strict FIFO order. Each reader or writer that finds the semaphore closed is inserted in the last position of a semaphore's wait queue list. When the semaphore is released, the process in the first position of the wait queue list are checked. The first process is always awoken. If it is a writer, the other processes in the wait queue continue to sleep. If it is a reader, all readers at the start of the queue, up to the first writer, are also woken up and get the lock. However, readers that have been queued after a writer continue to sleep.

Each read/write semaphore is described by a `rw_semaphore` structure that includes the following fields:

`count`

Stores two 16-bit counters. The counter in the most significant word encodes in two's complement form the sum of the number of nonwaiting writers (either 0 or 1) and the number of waiting kernel control paths. The counter in the less significant word encodes the total number of nonwaiting readers and writers.

`wait_list`

Points to a list of waiting processes. Each element in this list is a `rwsem_waiter` structure, including a pointer to the descriptor of the sleeping process and a flag indicating whether the process wants the semaphore for reading or for writing.

`wait_lock`

A spin lock used to protect the wait queue list and the `rw_semaphore` structure itself.

The `init_rwsem( )` function initializes an `rw_semaphore` structure by setting the `count` field to 0, the `wait_lock` spin lock to unlocked, and `wait_list` to the empty list.

The `down_read( )` and `down_write( )` functions acquire the read/write semaphore for reading and writing, respectively. Similarly, the `up_read( )` and `up_write( )` functions release a read/write semaphore previously acquired for reading and for writing. The `down_read_trylock( )` and `down_write_trylock( )` functions are similar to `down_read( )` and `down_write( )`, respectively, but they do not block the process if the semaphore is busy. Finally, the `downgrade_write( )` function atomically transforms a write lock into a read lock. The implementation of these five functions is long, but easy to follow because it resembles the implementation of normal semaphores; therefore, we avoid describing them.

Completions

Linux 2.6 also makes use of another synchronization primitive similar to semaphores: *completions*. They have been introduced to solve a subtle race condition that occurs in multiprocessor systems when process A allocates a temporary semaphore variable, initializes it as closed MUTEX, passes its address to process B, and then invokes `down( )` on it. Process A plans to destroy the semaphore as soon as it awakens. Later on, process B running on a different CPU invokes `up( )` on the semaphore. However, in the current implementation `up( )` and `down( )` can execute concurrently on the same semaphore. Thus, process A can be woken up and destroy the temporary semaphore while process B is still executing the `up( )` function. As a result, `up( )` might attempt to access a data structure that no longer exists.

Of course, it is possible to change the implementation of `down( )` and `up( )` to forbid concurrent executions on the same semaphore. However, this change would require additional instructions, which is a bad thing to do for functions that are so heavily used.

The completion is a synchronization primitive that is specifically designed to solve this problem. The `completion` data structure includes a wait queue head and a flag:


```

struct completion {
    unsigned int done;
    wait_queue_head_t wait;
};

```

The function corresponding to `up( )` is called `complete( )`. It receives as an argument the address of a `completion` data structure, invokes `spin_lock_irqsave( )` on the spin lock of the completion's wait queue, increases the `done` field, wakes up the exclusive process sleeping in the `wait` wait queue, and finally invokes `spin_unlock_irqrestore( )`.

The function corresponding to `down( )` is called `wait_for_completion( )`. It receives as an argument the address of a `completion` data structure and checks the value of the `done` flag. If it is greater than zero, `wait_for_completion( )` terminates, because `complete( )` has already been executed on another CPU. Otherwise, the function adds `current` to the tail of the wait queue as an exclusive process and puts `current` to sleep in the `TASK_UNINTERRUPTIBLE` state. Once woken up, the function removes `current` from the wait queue. Then, the function checks the value of the `done` flag: if it is equal to zero the function terminates, otherwise, the current process is suspended again. As in the case of the `complete( )` function, `wait_for_completion( )` makes use of the spin lock in the completion's wait queue.

The real difference between completions and semaphores is how the spin lock included in the wait queue is used. In completions, the spin lock is used to ensure that `complete( )` and `wait_for_completion( )` cannot execute concurrently. In semaphores, the spin lock is used to avoid letting concurrent `down( )`'s functions mess up the semaphore data structure.

Local Interrupt Disabling

Interrupt disabling is one of the key mechanisms used to ensure that a sequence of kernel statements is treated as a critical section. It allows a kernel control path to continue executing even when hardware devices issue IRQ signals, thus providing an effective way to protect data structures that are also accessed by interrupt handlers. By itself, however, local interrupt disabling does not protect against concurrent accesses to data structures by interrupt handlers running on other CPUs, so in multiprocessor systems, local interrupt disabling is often coupled with spin locks (see the later section "[Synchronizing Accesses to Kernel Data Structures](#)").

The `local_irq_disable( )` macro, which makes use of the `cli` assembly language instruction, disables interrupts on the local CPU. The `local_irq_enable( )` macro,

which makes use of the `sti` assembly language instruction, enables them. As stated in the section "[IRQs and Interrupts](#)" in [Chapter 4](#), the `cli` and `sti` assembly language instructions, respectively, clear and set the IF flag of the `eflags` control register. The `irqs_disabled( )` macro yields the value one if the IF flag of the `eflags` register is clear, the value zero if the flag is set.

When the kernel enters a critical section, it disables interrupts by clearing the IF flag of the `eflags` register. But at the end of the critical section, often the kernel can't simply set the flag again. Interrupts can execute in nested fashion, so the kernel does not necessarily know what the IF flag was before the current control path executed. In these cases, the control path must save the old setting of the flag and restore that setting at the end.

Saving and restoring the `eflags` content is achieved by means of the `local_irq_save` and `local_irq_restore` macros, respectively. The `local_irq_save` macro copies the content of the `eflags` register into a local variable; the IF flag is then cleared by a `cli` assembly language instruction. At the end of the critical region, the macro `local_irq_restore` restores the original content of `eflags`; therefore, interrupts are enabled only if they were enabled before this control path issued the `cli` assembly language instruction.

Disabling and Enabling Deferrable Functions

In the section "[Softirqs](#)" in [Chapter 4](#), we explained that deferrable functions can be executed at unpredictable times (essentially, on termination of hardware interrupt handlers). Therefore, data structures accessed by deferrable functions must be protected against race conditions.

A trivial way to forbid deferrable functions execution on a CPU is to disable interrupts on that CPU. Because no interrupt handler can be activated, softirq actions cannot be started asynchronously.

As we'll see in the next section, however, the kernel sometimes needs to disable deferrable functions without disabling interrupts. Local deferrable functions can be enabled or disabled on the local CPU by acting on the softirq counter stored in the `preempt_count` field of the current's `thread_info` descriptor.

Recall that the `do_softirq( )` function never executes the softirqs if the softirq counter is positive. Moreover, tasklets are implemented on top of softirqs, so setting

this counter to a positive value disables the execution of all deferrable functions on a given CPU, not just softirqs.

The `local_bh_disable` macro adds one to the softirq counter of the local CPU, while the `local_bh_enable( )` function subtracts one from it. The kernel can thus use several nested invocations of `local_bh_disable`; deferrable functions will be enabled again only by the `local_bh_enable` macro matching the first `local_bh_disable` invocation.

After having decreased the softirq counter, `local_bh_enable( )` performs two important operations that help to ensure timely execution of long-waiting threads:

1. Checks the hardirq counter and the softirq counter in the `preempt_count` field of the local CPU; if both of them are zero and there are pending softirqs to be executed, invokes `do_softirq( )` to activate them (see the section "[Softirqs](#)" in [Chapter 4](#)).
2. Checks whether the `TIF_NEED_RESCHED` flag of the local CPU is set; if so, a process switch request is pending, thus invokes the `preempt_schedule( )` function (see the section "[Kernel Preemption](#)" earlier in this chapter).

[*] A data item is aligned in memory when its address is a multiple of its size in bytes. For instance, the address of an aligned short integer must be a multiple of two, while the address of an aligned integer must be a multiple of four. Generally speaking, an unaligned memory access is not atomic.

[*] Spin locks, ironically enough, are global and therefore must themselves be protected against concurrent accesses.

[*] The actual implementation of the tight busy wait loop is slightly more complicated. The code at label 2, which is executed only if the spin lock is busy, is included in an auxiliary section so that in the most frequent case (when the spin lock is already free) the hardware cache is not filled with code that won't be executed. In our discussion, we omit these optimization details.

Synchronizing Accesses to Kernel Data Structures

A shared data structure can be protected against race conditions by using some of the synchronization primitives shown in the previous section. Of course, system performance may vary considerably, depending on the kind of synchronization primitive selected. Usually, the following rule of thumb is adopted by kernel developers: *always keep the concurrency level as high as possible in the system.*

In turn, the concurrency level in the system depends on two main factors:

- The number of I/O devices that operate concurrently
- The number of CPUs that do productive work

To maximize I/O throughput, interrupts should be disabled for very short periods of time. As described in the section "[IRQs and Interrupts](#)" in [Chapter 4](#), when interrupts are disabled, IRQs issued by I/O devices are temporarily ignored by the PIC, and no new activity can start on such devices.

To use CPUs efficiently, synchronization primitives based on spin locks should be avoided whenever possible. When a CPU is executing a tight instruction loop waiting for the spin lock to open, it is wasting precious machine cycles. Even worse, as we have already said, spin locks have negative effects on the overall performance of the system because of their impact on the hardware caches.

Let's illustrate a couple of cases in which synchronization can be achieved while still maintaining a high concurrency level:

- A shared data structure consisting of a single integer value can be updated by declaring it as an `atomic_t` type and by using atomic operations. An atomic operation is faster than spin locks and interrupt disabling, and it slows down only kernel control paths that concurrently access the data structure.
- Inserting an element into a shared linked list is never atomic, because it consists of at least two pointer assignments. Nevertheless, the kernel can sometimes perform this insertion operation without using locks or disabling interrupts. As an example of why this works, we'll consider the case where a system call service routine (see "[System Call Handler and Service Routines](#)" in [Chapter 10](#)) inserts new elements in a singly linked list, while an interrupt handler or deferrable function asynchronously looks up the list.

In the C language, insertion is implemented by means of the following pointer assignments:

```
new->next = list_element->next;  
list_element->next = new;
```

In assembly language, insertion reduces to two consecutive atomic instructions. The first instruction sets up the `next` pointer of the new element, but it does not modify the list. Thus, if the interrupt handler sees the list between the execution of the first and second instructions, it sees the list without the new element. If the handler sees the list after the execution of the second instruction, it sees the list with the new element. The important point is that in either case, the list is consistent and in an uncorrupted state. However, this integrity is assured only if the interrupt handler does not modify the list. If it does, the `next` pointer that was just set within the new element might become invalid.

However, developers must ensure that the order of the two assignment operations cannot be subverted by the compiler or the CPU's control unit; otherwise, if the system call service routine is interrupted by the interrupt handler between the two assignments, the handler finds a corrupted list. Therefore, a write memory barrier primitive is required:

```
new->next = list_element->next;  
wmb( );  
list_element->next = new;
```

Choosing Among Spin Locks, Semaphores, and Interrupt Disabling

Unfortunately, access patterns to most kernel data structures are a lot more complex than the simple examples just shown, and kernel developers are forced to use semaphores, spin locks, interrupts, and softirq disabling. Generally speaking, choosing the synchronization primitives depends on what kinds of kernel control paths access the data structure, as shown in [Table 5-8](#). Remember that whenever a kernel control path acquires a spin lock (as well as a read/write lock, a seqlock, or a RCU "read lock"), disables the local interrupts, or disables the local softirqs, kernel preemption is automatically disabled.

Table 5-8. Protection required by data structures accessed by kernel control paths

Kernel control paths accessing the data structure	UP protection	MP further protection
Exceptions	Semaphore	None
Interrupts	Local interrupt disabling	Spin lock
Deferrable functions	None	None or spin lock (see Table 5-9)
Exceptions + Interrupts	Local interrupt disabling	Spin lock
Exceptions + Deferrable functions	Local softirq disabling	Spin lock
Interrupts + Deferrable functions	Local interrupt disabling	Spin lock
Exceptions + Interrupts + Deferrable functions	Local interrupt disabling	Spin lock

Protecting a data structure accessed by exceptions

When a data structure is accessed only by exception handlers, race conditions are usually easy to understand and prevent. The most common exceptions that give rise to synchronization problems are the system call service routines (see the section "[System Call Handler and Service Routines](#)" in [Chapter 10](#)) in which the CPU operates in Kernel Mode to offer a service to a User Mode program. Thus, a data structure accessed only by an exception usually represents a resource that can be assigned to one or more processes.

Race conditions are avoided through semaphores, because these primitives allow the process to sleep until the resource becomes available. Notice that semaphores work equally well both in uniprocessor and multiprocessor systems.

Kernel preemption does not create problems either. If a process that owns a semaphore is preempted, a new process running on the same CPU could try to get the semaphore. When this occurs, the new process is put to sleep, and eventually the old process will release the semaphore. The only case in which kernel preemption must be explicitly disabled is when accessing per-CPU variables, as explained in the section "[Per-CPU Variables](#)" earlier in this chapter.

Protecting a data structure accessed by interrupts

Suppose that a data structure is accessed by only the "top half" of an interrupt handler. We learned in the section "[Interrupt Handling](#)" in [Chapter 4](#) that each interrupt handler is serialized with respect to itself — that is, it cannot execute more than once concurrently. Thus, accessing the data structure does not require synchronization primitives.

Things are different, however, if the data structure is accessed by several interrupt handlers. A handler may interrupt another handler, and different interrupt handlers may run concurrently in multiprocessor systems. Without synchronization, the shared data structure might easily become corrupted.

In uniprocessor systems, race conditions must be avoided by disabling interrupts in all critical regions of the interrupt handler. Nothing less will do because no other synchronization primitives accomplish the job. A semaphore can block the process, so it cannot be used in an interrupt handler. A spin lock, on the other hand, can freeze the system: if the handler accessing the data structure is interrupted, it cannot release the lock; therefore, the new interrupt handler keeps waiting on the tight loop of the spin lock.

Multiprocessor systems, as usual, are even more demanding. Race conditions cannot be avoided by simply disabling local interrupts. In fact, even if interrupts are disabled on a CPU, interrupt handlers can still be executed on the other CPUs. The most convenient method to prevent the race conditions is to disable local interrupts (so that other interrupt handlers running on the same CPU won't interfere) *and* to acquire a spin lock or a read/write spin lock that protects the data structure. Notice that these additional spin locks cannot freeze the system because even if an interrupt handler finds the lock closed, eventually the interrupt handler on the other CPU that owns the lock will release it.

The Linux kernel uses several macros that couple the enabling and disabling of local interrupts with spin lock handling. [Table 5-9](#) describes all of them. In uniprocessor systems, these macros just enable or disable local interrupts and kernel preemption.

Table 5-9. Interrupt-aware spin lock macros

Macro	Description
<code>spin_lock_irq(l)</code>	<code>local_irq_disable(); spin_lock(l)</code>
<code>spin_unlock_irq(l)</code>	<code>spin_unlock(l); local_irq_enable()</code>
<code>spin_lock_bh(l)</code>	<code>local_bh_disable(); spin_lock(l)</code>
<code>spin_unlock_bh(l)</code>	<code>spin_unlock(l); local_bh_enable()</code>
<code>spin_lock_irqsave(l,f)</code>	<code>local_irq_save(f); spin_lock(l)</code>
<code>spin_unlock_irqrestore(l,f)</code>	<code>spin_unlock(l); local_irq_restore(f)</code>
<code>read_lock_irq(l)</code>	<code>local_irq_disable(); read_lock(l)</code>
<code>read_unlock_irq(l)</code>	<code>read_unlock(l); local_irq_enable()</code>
<code>read_lock_bh(l)</code>	<code>local_bh_disable(); read_lock(l)</code>
<code>read_unlock_bh(l)</code>	<code>read_unlock(l); local_bh_enable()</code>
<code>write_lock_irq(l)</code>	<code>local_irq_disable(); write_lock(l)</code>
<code>write_unlock_irq(l)</code>	<code>write_unlock(l); local_irq_enable()</code>
<code>write_lock_bh(l)</code>	<code>local_bh_disable(); write_lock(l)</code>
<code>write_unlock_bh(l)</code>	<code>write_unlock(l); local_bh_enable()</code>
<code>read_lock_irqsave(l,f)</code>	<code>local_irq_save(f); read_lock(l)</code>
<code>read_unlock_irqrestore(l,f)</code>	<code>read_unlock(l); local_irq_restore(f)</code>
<code>write_lock_irqsave(l,f)</code>	<code>local_irq_save(f); write_lock(l)</code>
<code>write_unlock_irqrestore(l,f)</code>	<code>write_unlock(l); local_irq_restore(f)</code>
<code>read_seqbegin_irqsave(l,f)</code>	<code>local_irq_save(f); read_seqbegin(l)</code>
<code>read_seqretry_irqrestore(l,v,f)</code>	<code>read_seqretry(l,v); local_irq_restore(f)</code>
<code>write_seqlock_irqsave(l,f)</code>	<code>local_irq_save(f); write_seqlock(l)</code>
<code>write_sequnlock_irqrestore(l,f)</code>	<code>write_sequnlock(l); local_irq_restore(f)</code>
<code>write_seqlock_irq(l)</code>	<code>local_irq_disable(); write_seqlock(l)</code>
<code>write_sequnlock_irq(l)</code>	<code>write_sequnlock(l); local_irq_enable()</code>

Macro	Description
write_seqlock_bh(l)	local_bh_disable(); write_seqlock(l);
write_sequnlock_bh(l)	write_sequnlock(l); local_bh_enable()

Protecting a data structure accessed by deferrable functions

What kind of protection is required for a data structure accessed only by deferrable functions? Well, it mostly depends on the kind of deferrable function. In the section "[Softirqs and Tasklets](#)" in [Chapter 4](#), we explained that softirqs and tasklets essentially differ in their degree of concurrency.

First of all, no race condition may exist in uniprocessor systems. This is because execution of deferrable functions is always serialized on a CPU — that is, a deferrable function cannot be interrupted by another deferrable function. Therefore, no synchronization primitive is ever required.

Conversely, in multiprocessor systems, race conditions do exist because several deferrable functions may run concurrently. [Table 5-10](#) lists all possible cases.

Table 5-10. Protection required by data structures accessed by deferrable functions in SMP

Deferrable functions accessing the data structure	Protection
Softirqs	Spin lock
One tasklet	None
Many tasklets	Spin lock

A data structure accessed by a softirq must always be protected, usually by means of a spin lock, because the same softirq may run concurrently on two or more CPUs. Conversely, a data structure accessed by just one kind of tasklet need not be protected, because tasklets of the same kind cannot run concurrently. However, if the data structure is accessed by several kinds of tasklets, then it must be protected.

Protecting a data structure accessed by exceptions and interrupts

Let's consider now a data structure that is accessed both by exceptions (for instance, system call service routines) and interrupt handlers.

On uniprocessor systems, race condition prevention is quite simple, because interrupt handlers are not reentrant and cannot be interrupted by exceptions. As long as the kernel accesses the data structure with local interrupts disabled, the kernel cannot be interrupted when accessing the data structure. However, if the data structure is accessed by just one kind of interrupt handler, the interrupt handler can freely access the data structure without disabling local interrupts.

On multiprocessor systems, we have to take care of concurrent executions of exceptions and interrupts on other CPUs. Local interrupt disabling must be coupled with a spin lock, which forces the concurrent kernel control paths to wait until the handler accessing the data structure finishes its work.

Sometimes it might be preferable to replace the spin lock with a semaphore. Because interrupt handlers cannot be suspended, they must acquire the semaphore using a tight loop and the `down_trylock( )` function; for them, the semaphore acts essentially as a spin lock. System call service routines, on the other hand, may suspend the calling processes when the semaphore is busy. For most system calls, this is the expected behavior. In this case, semaphores are preferable to spin locks, because they lead to a higher degree of concurrency of the system.

Protecting a data structure accessed by exceptions and deferrable functions

A data structure accessed both by exception handlers and deferrable functions can be treated like a data structure accessed by exception and interrupt handlers. In fact, deferrable functions are essentially activated by interrupt occurrences, and no exception can be raised while a deferrable function is running. Coupling local interrupt disabling with a spin lock is therefore sufficient.

Actually, this is much more than sufficient: the exception handler can simply disable deferrable functions instead of local interrupts by using the `local_bh_disable( )` macro (see the section "[Softirqs](#)" in [Chapter 4](#)). Disabling only the deferrable functions is preferable to disabling interrupts, because interrupts continue to be serviced by the CPU. Execution of deferrable functions on each CPU is serialized, so no race condition exists.

As usual, in multiprocessor systems, spin locks are required to ensure that the data structure is accessed at any time by just one kernel control.

Protecting a data structure accessed by interrupts and deferrable functions

This case is similar to that of a data structure accessed by interrupt and exception handlers. An interrupt might be raised while a deferrable function is running, but no deferrable function can stop an interrupt handler. Therefore, race conditions must be avoided by disabling local interrupts during the deferrable function. However, an interrupt handler can freely touch the data structure accessed by the deferrable function without disabling interrupts, provided that no other interrupt handler accesses that data structure.

Again, in multiprocessor systems, a spin lock is always required to forbid concurrent accesses to the data structure on several CPUs.

Protecting a data structure accessed by exceptions, interrupts, and deferrable functions

Similarly to previous cases, disabling local interrupts and acquiring a spin lock is almost always necessary to avoid race conditions. Notice that there is no need to explicitly disable deferrable functions, because they are essentially activated when terminating the execution of interrupt handlers; disabling local interrupts is therefore sufficient.

Examples of Race Condition Prevention

Kernel developers are expected to identify and solve the synchronization problems raised by interleaving kernel control paths. However, avoiding race conditions is a hard task because it requires a clear understanding of how the various components of the kernel interact. To give a feeling of what's really inside the kernel code, let's mention a few typical usages of the synchronization primitives defined in this chapter.

Reference Counters

Reference counters are widely used inside the kernel to avoid race conditions due to the concurrent allocation and releasing of a resource. A *reference counter* is just an `atomic_t` counter associated with a specific resource such as a memory page, a module, or a file. The counter is atomically increased when a kernel control path starts using the resource, and it is decreased when a kernel control path finishes using the resource. When the reference counter becomes zero, the resource is not being used, and it can be released if necessary.

The Big Kernel Lock

In earlier Linux kernel versions, a *big kernel lock* (also known as *global kernel lock*, or *BKL*) was widely used. In Linux 2.0, this lock was a relatively crude spin lock, ensuring that only one processor at a time could run in Kernel Mode. The 2.2 and 2.4 kernels were considerably more flexible and no longer relied on a single spin lock; rather, a large number of kernel data structures were protected by many different spin locks. In Linux kernel version 2.6, the big kernel lock is used to protect old code (mostly functions related to the VFS and to several filesystems).

Starting from kernel version 2.6.11, the big kernel lock is implemented by a semaphore named `kernel_sem` (in earlier 2.6 versions, the big kernel lock was implemented by means of a spin lock). The big kernel lock is slightly more sophisticated than a simple semaphore, however.

Every process descriptor includes a `lock_depth` field, which allows the same process to acquire the big kernel lock several times. Therefore, two consecutive requests for it will not hang the processor (as for normal locks). If the process has not acquired the lock, the field has the value -1; otherwise, the field value plus 1 specifies how many times the lock has been taken. The `lock_depth` field is crucial for allowing interrupt handlers, exception handlers, and deferrable functions to take the big kernel lock:

without it, every asynchronous function that tries to get the big kernel lock could generate a deadlock if the current process already owns the lock.

The `lock_kernel( )` and `unlock_kernel( )` functions are used to get and release the big kernel lock. The former function is equivalent to:

```
depth = current->lock_depth + 1;
if (depth == 0)
    down(&kernel_sem);
current->lock_depth = depth;
```

while the latter is equivalent to:

```
if (--current->lock_depth < 0)
    up(&kernel_sem);
```

Notice that the `if` statements of the `lock_kernel( )` and `unlock_kernel( )` functions need not be executed atomically because `lock_depth` is not a global variable — each CPU addresses a field of its own current process descriptor. Local interrupts inside the `if` statements do not induce race conditions either. Even if the new kernel control path invokes `lock_kernel( )`, it must release the big kernel lock before terminating.

Surprisingly enough, a process holding the big kernel lock is allowed to invoke `schedule( )`, thus relinquishing the CPU. The `schedule( )` function, however, checks the `lock_depth` field of the process being replaced and, if its value is zero or positive, automatically releases the `kernel_sem` semaphore (see the section "[The `schedule\( \)` Function](#)" in [Chapter 7](#)). Thus, no process that explicitly invokes `schedule( )` can keep the big kernel lock across the process switch. The `schedule( )` function, however, will reacquire the big kernel lock for the process when it will be selected again for execution.

Things are different, however, if a process that holds the big kernel lock is preempted by another process. Up to kernel version 2.6.10 this case could not occur, because acquiring a spin lock automatically disables kernel preemption. The current implementation of the big kernel lock, however, is based on a semaphore, and acquiring it does not automatically disable kernel preemption. Actually, allowing kernel preemption inside critical regions protected by the big kernel lock has been the main reason for changing its implementation. This, in turn, has beneficial effects on the response time of the system.

When a process holding the big kernel lock is preempted, `schedule( )` must not release the semaphore because the process executing the code in the critical region has

not voluntarily triggered a process switch, thus if the big kernel lock would be released, another process might take it and corrupt the data structures accessed by the preempted process.

To avoid the preempted process losing the big kernel lock, the `preempt_schedule_irq( )` function temporarily sets the `lock_depth` field of the process to -1 (see the section "[Returning from Interrupts and Exceptions](#)" in [Chapter 4](#)). Looking at the value of this field, `schedule( )` assumes that the process being replaced does not hold the `kernel_sem` semaphore and thus does not release it. As a result, the `kernel_sem` semaphore continues to be owned by the preempted process. Once this process is selected again by the scheduler, the `preempt_schedule_irq( )` function restores the original value of the `lock_depth` field and lets the process resume execution in the critical section protected by the big kernel lock.

Memory Descriptor Read/Write Semaphore

Each memory descriptor of type `mm_struct` includes its own semaphore in the `mmap_sem` field (see the section "[The Memory Descriptor](#)" in [Chapter 9](#)). The semaphore protects the descriptor against race conditions that could arise because a memory descriptor can be shared among several lightweight processes.

For instance, let's suppose that the kernel must create or extend a memory region for some process; to do this, it invokes the `do_mmap( )` function, which allocates a new `vm_area_struct` data structure. In doing so, the current process could be suspended if no free memory is available, and another process sharing the same memory descriptor could run. Without the semaphore, every operation of the second process that requires access to the memory descriptor (for instance, a Page Fault due to a Copy on Write) could lead to severe data corruption.

The semaphore is implemented as a read/write semaphore, because some kernel functions, such as the Page Fault exception handler (see the section "[Page Fault Exception Handler](#)" in [Chapter 9](#)), need only to scan the memory descriptors.

Slab Cache List Semaphore

The list of slab cache descriptors (see the section "[Cache Descriptor](#)" in [Chapter 8](#)) is protected by the `cache_chain_sem` semaphore, which grants an exclusive right to access and modify the list.

A race condition is possible when `kmem_cache_create( )` adds a new element in the list, while `kmem_cache_shrink( )` and `kmem_cache_reap( )` sequentially scan the list. However, these functions are never invoked while handling an interrupt, and they can never block while accessing the list. The semaphore plays an active role both in multiprocessor systems and in uniprocessor systems with kernel preemption supported.

Inode Semaphore

As we'll see in "[Inode Objects](#)" in [Chapter 12](#), Linux stores the information on a disk file in a memory object called an inode. The corresponding data structure includes its own semaphore in the `i_sem` field.

A huge number of race conditions can occur during filesystem handling. Indeed, each file on disk is a resource held in common for all users, because all processes may (potentially) access the file content, change its name or location, destroy or duplicate it, and so on. For example, let's suppose that a process lists the files contained in some directory. Each disk operation is potentially blocking, and therefore even in uniprocessor systems, other processes could access the same directory and modify its content while the first process is in the middle of the listing operation. Or, again, two different processes could modify the same directory at the same time. All these race conditions are avoided by protecting the directory file with the inode semaphore.

Whenever a program uses two or more semaphores, the potential for deadlock is present, because two different paths could end up waiting for each other to release a semaphore. Generally speaking, Linux has few problems with deadlocks on semaphore requests, because each kernel control path usually needs to acquire just one semaphore at a time. However, in some cases, the kernel must get two or more locks. Inode semaphores are prone to this scenario; for instance, this occurs in the service routine in the `rename( )` system call. In this case, two different inodes are involved in the operation, so both semaphores must be taken. To avoid such deadlocks, semaphore requests are performed in predefined address order.

Chapter 6. Timing Measurements

Countless computerized activities are driven by timing measurements, often behind the user's back. For instance, if the screen is automatically switched off after you have stopped using the computer's console, it is due to a timer that allows the kernel to keep track of how much time has elapsed since you pushed a key or moved the mouse. If you receive a warning from the system asking you to remove a set of unused files, it is the outcome of a program that identifies all user files that have not been accessed for a long time. To do these things, programs must be able to retrieve a timestamp identifying its last access time from each file. Such a timestamp must be automatically written by the kernel. More significantly, timing drives process switches along with even more visible kernel activities such as checking for time-outs.

We can distinguish two main kinds of timing measurement that must be performed by the Linux kernel:

- Keeping the current time and date so they can be returned to user programs through the `time()`, `ftime()`, and `gettimeofday()` APIs (see the section "[The `time\(\)`, `ftime\(\)`, and `gettimeofday\(\)` System Calls](#)" later in this chapter) and used by the kernel itself as timestamps for files and network packets
- Maintaining timers — mechanisms that are able to notify the kernel (see the later section "[Software Timers and Delay Functions](#)") or a user program (see the later sections "[The `setitimer\(\)` and `alarm\(\)` System Calls](#)" and "System Calls for POSIX Timers") that a certain interval of time has elapsed

Timing measurements are performed by several hardware circuits based on fixed-frequency oscillators and counters. This chapter consists of four different parts. The first two sections describe the hardware devices that underly timing and give an overall picture of Linux timekeeping architecture. The following sections describe the main time-related duties of the kernel: implementing CPU time sharing, updating system time and resource usage statistics, and maintaining software timers. The last section discusses the system calls related to timing measurements and the corresponding service routines.

Clock and Timer Circuits

On the 80×86 architecture, the kernel must explicitly interact with several kinds of clocks and timer circuits. The *clock circuits* are used both to keep track of the current time of day and to make precise time measurements. The *timer circuits* are programmed by the kernel, so that they issue interrupts at a fixed, predefined frequency; such periodic interrupts are crucial for implementing the software timers used by the kernel and the user programs. We'll now briefly describe the clock and hardware circuits that can be found in IBM-compatible PCs.

Real Time Clock (RTC)

All PCs include a clock called *Real Time Clock (RTC)*, which is independent of the CPU and all other chips.

The RTC continues to tick even when the PC is switched off, because it is energized by a small battery. The CMOS RAM and RTC are integrated in a single chip (the Motorola 146818 or an equivalent).

The RTC is capable of issuing periodic interrupts on IRQ 8 at frequencies ranging between 2 Hz and 8,192 Hz. It can also be programmed to activate the IRQ 8 line when the RTC reaches a specific value, thus working as an alarm clock.

Linux uses the RTC only to derive the time and date; however, it allows processes to program the RTC by acting on the `/dev/rtc` device file (see [Chapter 13](#)). The kernel accesses the RTC through the 0x70 and 0x71 I/O ports. The system administrator can read and write the RTC by executing the *clock* Unix system program that acts directly on these two I/O ports.

Time Stamp Counter (TSC)

All 80×86 microprocessors include a CLK input pin, which receives the clock signal of an external oscillator. Starting with the Pentium, 80×86 microprocessors sport a counter that is increased at each clock signal. The counter is accessible through the 64-bit *Time Stamp Counter (TSC)* register, which can be read by means of the `rdtsc` assembly language instruction. When using this register, the kernel has to take into consideration the frequency of the clock signal: if, for instance, the clock ticks at 1 GHz, the Time Stamp Counter is increased once every nanosecond.

Linux may take advantage of this register to get much more accurate time measurements than those delivered by the Programmable Interval Timer. To do this, Linux must determine the clock signal frequency while initializing the system. In fact, because this frequency is not declared when compiling the kernel, the same kernel image may run on CPUs whose clocks may tick at any frequency.

The task of figuring out the actual frequency of a CPU is accomplished during the system's boot. The `calibrate_tsc( )` function computes the frequency by counting the number of clock signals that occur in a time interval of approximately 5 milliseconds. This time constant is produced by properly setting up one of the channels of the Programmable Interval Timer (see the next section).^[*]

Programmable Interval Timer (PIT)

Besides the Real Time Clock and the Time Stamp Counter, IBM-compatible PCs include another type of time-measuring device called *Programmable Interval Timer (PIT)*. The role of a PIT is similar to the alarm clock of a microwave oven: it makes the user aware that the cooking time interval has elapsed. Instead of ringing a bell, this device issues a special interrupt called *timer interrupt*, which notifies the kernel that one more time interval has elapsed.^[†] Another difference from the alarm clock is that the PIT goes on issuing interrupts forever at some fixed frequency established by the kernel. Each IBM-compatible PC includes at least one PIT, which is usually implemented by an 8254 CMOS chip using the 0x40-0x43 I/O ports.

As we'll see in detail in the next paragraphs, Linux programs the PIT of IBM-compatible PCs to issue timer interrupts on the IRQ 0 at a (roughly) 1000-Hz frequency — that is, once every 1 millisecond. This time interval is called a *tick*, and its length in nanoseconds is stored in the `tick_nsec` variable. On a PC, `tick_nsec` is initialized to 999,848 nanoseconds (yielding a clock signal frequency of about 1000.15 Hz), but its value may be automatically adjusted by the kernel if the computer is synchronized with an external clock (see the later section "[The adjtimex\(.\) System Call](#)"). The ticks beat time for all activities in the system; in some sense, they are like the ticks sounded by a metronome while a musician is rehearsing.

Generally speaking, shorter ticks result in higher resolution timers, which help with smoother multimedia playback and faster response time when performing synchronous I/O multiplexing (`poll( )` and `select( )` system calls). This is a trade-off however: shorter ticks require the CPU to spend a larger fraction of its time in Kernel Mode —

that is, a smaller fraction of time in User Mode. As a consequence, user programs run slower.

The frequency of timer interrupts depends on the hardware architecture. The slower machines have a tick of roughly 10 milliseconds (100 timer interrupts per second), while the faster ones have a tick of roughly 1 millisecond (1000 or 1024 timer interrupts per second).

A few macros in the Linux code yield some constants that determine the frequency of timer interrupts. These are discussed in the following list.

- HZ yields the approximate number of timer interrupts per second — that is, their frequency. This value is set to 1000 for IBM PCs.
- CLOCK_TICK_RATE yields the value 1,193,182, which is the 8254 chip's internal oscillator frequency.
- LATCH yields the ratio between CLOCK_TICK_RATE and HZ, rounded to the nearest integer. It is used to program the PIT.

The PIT is initialized by `setup_pit_timer( )` as follows:

```
spin_lock_irqsave(&i8253_lock, flags);
outb_p(0x34, 0x43);
udelay(10);
outb_p(LATCH & 0xff, 0x40);
udelay(10);
outb
(LATCH >> 8, 0x40);
spin_unlock_irqrestore(&i8253_lock, flags);
```

The `outb( )` C function is equivalent to the `outb` assembly language instruction: it copies the first operand into the I/O port specified as the second operand. The `outb_p( )` function is similar to `outb( )`, except that it introduces a pause by executing a no-op instruction to keep the hardware from getting confused. The `udelay( )` macro introduces a further small delay (see the later section "[Delay Functions](#)"). The first `outb_p( )` invocation is a command to the PIT to issue interrupts at a new rate. The next two `outb_p( )` and `outb( )` invocations supply the new interrupt rate to the device. The 16-bit LATCH constant is sent to the 8-bit 0x40 I/O port of the device as two consecutive bytes. As a result, the PIT issues timer interrupts at a (roughly) 1000-Hz frequency (that is, once every 1 ms).

CPU Local Timer

The local APIC present in recent 80 × 86 microprocessors (see the section "[Interrupts and Exceptions](#)" in [Chapter 4](#)) provides yet another time-measuring device: the *CPU local timer*.

The CPU local timer is a device similar to the Programmable Interval Timer just described that can issue one-shot or periodic interrupts. There are, however, a few differences:

- The APIC's timer counter is 32 bits long, while the PIT's timer counter is 16 bits long; therefore, the local timer can be programmed to issue interrupts at very low frequencies (the counter stores the number of ticks that must elapse before the interrupt is issued).
- The local APIC timer sends an interrupt only to its processor, while the PIT raises a global interrupt, which may be handled by any CPU in the system.
- The APIC's timer is based on the bus clock signal (or the APIC bus signal, in older machines). It can be programmed in such a way to decrease the timer counter every 1, 2, 4, 8, 16, 32, 64, or 128 bus clock signals. Conversely, the PIT, which makes use of its own clock signals, can be programmed in a more flexible way.

High Precision Event Timer (HPET)

The *High Precision Event Timer (HPET)* is a new timer chip developed jointly by Intel and Microsoft. Although HPETs are not yet very common in end-user machines, Linux 2.6 already supports them, so we'll spend a few words describing their characteristics.

The HPET provides a number of hardware timers that can be exploited by the kernel. Basically, the chip includes up to eight 32-bit or 64-bit independent *counters*. Each counter is driven by its own clock signal, whose frequency must be at least 10 MHz; therefore, the counter is increased at least once in 100 nanoseconds. Any counter is associated with at most 32 *timers*, each of which is composed by a *comparator* and a *match register*. The comparator is a circuit that checks the value in the counter against the value in the match register, and raises a hardware interrupt if a match is found. Some of the timers can be enabled to generate a periodic interrupt.

The HPET chip can be programmed through registers mapped into memory space (much like the I/O APIC). The BIOS establishes the mapping during the bootstrapping phase and reports to the operating system kernel its initial memory address. The HPET registers allow the kernel to read and write the values of the counters and of the

match registers , to program one-shot interrupts, and to enable or disable periodic interrupts on the timers that support them.

The next generation of motherboards will likely sport both the HPET and the 8254 PIT; in some future time, however, the HPET is expected to completely replace the PIT.

ACPI Power Management Timer

The *ACPI Power Management Timer* (or *ACPI PMT*) is yet another clock device included in almost all ACPI-based motherboards. Its clock signal has a fixed frequency of roughly 3.58 MHz. The device is actually a simple counter increased at each clock tick; to read the current value of the counter, the kernel accesses an I/O port whose address is determined by the BIOS during the initialization phase (see Appendix A).

The ACPI Power Management Timer is preferable to the TSC if the operating system or the BIOS may dynamically lower the frequency or voltage of the CPU to save battery power. When this happens, the frequency of the TSC changes—thus causing time warps and others unpleasant effects—while the frequency of the ACPI PMT does not. On the other hand, the high-frequency of the TSC counter is quite handy for measuring very small time intervals.

However, if an HPET device is present, it should always be preferred to the other circuits because of its richer architecture. [Table 6-2](#) later in this chapter illustrates how Linux takes advantage of the available timing circuits.

Now that we understand what the hardware timers are, we may discuss how the Linux kernel exploits them to conduct all activities of the system.

[*] To avoid losing significant digits in the integer divisions, `calibrate_tsc( )` returns the duration, in microseconds, of a clock tick multiplied by 232.

[†] The PIT is also used to drive an audio amplifier connected to the computer's internal speaker.

The Linux Timekeeping Architecture

Linux must carry on several time-related activities. For instance, the kernel periodically:

- Updates the time elapsed since system startup.
- Updates the time and date.
- Determines, for every CPU, how long the current process has been running, and preempts it if it has exceeded the time allocated to it. The allocation of time slots (also called "quanta") is discussed in [Chapter 7](#).
- Updates resource usage statistics.
- Checks whether the interval of time associated with each software timer (see the later section "[Software Timers and Delay Functions](#)") has elapsed.

Linux's *timekeeping architecture* is the set of kernel data structures and functions related to the flow of time. Actually, 80 × 86-based multiprocessor machines have a timekeeping architecture that is slightly different from the timekeeping architecture of uniprocessor machines:

- In a uniprocessor system, all time-keeping activities are triggered by interrupts raised by the global timer (either the Programmable Interval Timer or the High Precision Event Timer).
- In a multiprocessor system, all general activities (such as handling of software timers) are triggered by the interrupts raised by the global timer, while CPU-specific activities (such as monitoring the execution time of the currently running process) are triggered by the interrupts raised by the local APIC timer.

Unfortunately, the distinction between the two cases is somewhat blurred. For instance, some early SMP systems based on Intel 80486 processors didn't have local APICs. Even nowadays, there are SMP motherboards so buggy that local timer interrupts are not usable at all. In these cases, the SMP kernel must resort to the UP timekeeping architecture. On the other hand, recent uniprocessor systems feature one local APIC, so the UP kernel often makes use of the SMP timekeeping architecture. However, to simplify our description, we won't discuss these hybrid cases and will stick to the two "pure" timekeeping architectures.

Linux's timekeeping architecture depends also on the availability of the Time Stamp Counter (TSC), of the ACPI Power Management Timer, and of the High Precision

Event Timer (HPET). The kernel uses two basic timekeeping functions: one to keep the current time up-to-date and another to count the number of nanoseconds that have elapsed within the current second. There are different ways to get the last value. Some methods are more precise and are available if the CPU has a Time Stamp Counter or a HPET; a less-precise method is used in the opposite case (see the later section "[The time\(.\) and gettimeofday\(.\) System Calls](#)").

Data Structures of the Timekeeping Architecture

The timekeeping architecture of Linux 2.6 makes use of a large number of data structures. As usual, we will describe the most important variables by referring to the 80 × 86 architecture.

The timer object

In order to handle the possible timer sources in a uniform way, the kernel makes use of a "timer object," which is a descriptor of type `timer_opts` consisting of the timer name and of four standard methods shown in [Table 6-1](#).

Table 6-1. The fields of the `timer_opts` data structure

Field name	Description
<code>name</code>	A string identifying the timer source
<code>mark_offset</code>	Records the exact time of the last tick; it is invoked by the timer interrupt handler
<code>get_offset</code>	Returns the time elapsed since the last tick
<code>monotonic_clock</code>	Returns the number of nanoseconds since the kernel initialization
<code>delay</code>	Waits for a given number of "loops" (see the later section " Delay Functions ")

The most important methods of the timer object are `mark_offset` and `get_offset`. The `mark_offset` method is invoked by the timer interrupt handler, and records in a suitable data structure the exact time at which the tick occurred. Using the saved value, the `get_offset` method computes the time in microseconds elapsed since the last timer interrupt (tick). Thanks to these two methods, Linux timekeeping architecture achieves a sub-tick resolution—that is, the kernel is able to determine the current time with a precision much higher than the tick duration. This operation is called *time interpolation*.

The `cur_timer` variable stores the address of the timer object corresponding to the "best" timer source available in the system. Initially, `cur_timer` points to `timer_none`, which is the object corresponding to a dummy timer source used when the kernel is being initialized. During kernel initialization, the `select_timer( )` function sets `cur_timer` to the address of the appropriate timer object. [Table 6-2](#) shows the most common timer objects used in the 80x86 architecture, in order of preference. As you see, `select_timer( )` selects the HPET, if available; otherwise, it selects the ACPI Power Management Timer, if available, or the TSC. As the last resort, `select_timer( )` selects the always-present PIT. The "Time interpolation" column lists the timer sources used by the `mark_offset` and `get_offset` methods of the timer object; the "Delay" column lists the timer sources used by the `delay` method.

Table 6-2. Typical timer objects of the 80x86 architecture, in order of preference

Timer object name	Description	Time interpolation	Delay
<code>timer_hpet</code>	High Precision Event Timer (HPET)	HPET	HPET
<code>timer_pmtmr</code>	ACPI Power Management Timer (ACPI PMT)	ACPI PMT	TSC
<code>timer_tsc</code>	Time Stamp Counter (TSC)	TSC	TSC
<code>timer_pit</code>	Programmable Interval Timer (PIT)	PIT	Tight loop
<code>timer_none</code>	Generic dummy timer source(used during kernel initialization)	(none)	Tight loop

Notice that local APIC timers do not have a corresponding timer object. The reason is that local APIC timers are used only to generate periodic interrupts and are never used to achieve sub-tick resolution.

The `jiffies` variable

The `jiffies` variable is a counter that stores the number of elapsed ticks since the system was started. It is increased by one when a timer interrupt occurs—that is, on every tick. In the 80 × 86 architecture, `jiffies` is a 32-bit variable, therefore it wraps around in approximately 50 days—a relatively short time interval for a Linux server. However, the kernel handles cleanly the overflow of `jiffies` thanks to the

`time_after`, `time_after_eq`, `time_before`, and `time_before_eq` macros: they yield the correct value even if a wraparound occurred.

You might suppose that `jiffies` is initialized to zero at system startup. Actually, this is not the case: `jiffies` is initialized to `0xffffb6c20`, which corresponds to the 32-bit signed value —300,000; therefore, the counter will overflow five minutes after the system boot. This is done on purpose, so that buggy kernel code that does not check for the overflow of `jiffies` shows up very soon in the developing phase and does not pass unnoticed in stable kernels.

In a few cases, however, the kernel needs the real number of system ticks elapsed since the system boot, regardless of the overflows of `jiffies`. Therefore, in the 80×86 architecture the `jiffies` variable is equated by the linker to the 32 less significant bits of a 64-bit counter called `jiffies_64`. With a tick of 1 millisecond, the `jiffies_64` variable wraps around in several hundreds of millions of years, thus we can safely assume that it never overflows.

You might wonder why `jiffies` has not been directly declared as a 64-bit `unsigned long long` integer on the 80×86 architecture. The answer is that accesses to 64-bit variables in 32-bit architectures cannot be done atomically. Therefore, every read operation on the whole 64 bits requires some synchronization technique to ensure that the counter is not updated while the two 32-bit half-counters are read; as a consequence, every 64-bit read operation is significantly slower than a 32-bit read operation.

The `get_jiffies_64( )` function reads the value of `jiffies_64` and returns its value:

```
unsigned long long get_jiffies_64(void)
{
    unsigned long seq;
    unsigned long long ret;
    do {
        seq = read_seqbegin(&xtime_lock);
        ret = jiffies_64;
    } while (read_seqretry(&xtime_lock, seq));
    return ret;
}
```

The 64-bit read operation is protected by the `xtime_lock` seqlock (see the section "[Seqlocks](#)" in [Chapter 5](#)): the function keeps reading the `jiffies_64` variable until it knows for sure that it has not been concurrently updated by another kernel control path.

Conversely, the critical region increasing the `jiffies_64` variable must be protected by means of `write_seqlock(&xtime_lock )` and `write_sequnlock(&xtime_lock)`. Notice that the `++jiffies_64` instruction also increases the 32-bit `jiffies` variable, because the latter corresponds to the lower half of `jiffies_64`.

The `xtime` variable

The `xtime` variable stores the current time and date; it is a structure of type `timespec` having two fields:

`tv_sec`

Stores the number of seconds that have elapsed since midnight of January 1, 1970 (UTC)

`tv_nsec`

Stores the number of nanoseconds that have elapsed within the last second (its value ranges between 0 and 999,999,999)

The `xtime` variable is usually updated once in a tick—that is, roughly 1000 times per second. As we'll see in the later section "[System Calls Related to Timing Measurements](#)," user programs get the current time and date from the `xtime` variable. The kernel also often refers to it, for instance, when updating inode timestamps (see the section "[File Descriptor and Inode](#)" in [Chapter 1](#)).

The `xtime_lock` seqlock avoids the race conditions that could occur due to concurrent accesses to the `xtime` variable. Remember that `xtime_lock` also protects the `jiffies_64` variable; in general, this seqlock is used to define several critical regions of the timekeeping architecture.

Timekeeping Architecture in Uniprocessor Systems

In a uniprocessor system, all time-related activities are triggered by the interrupts raised by the Programmable Interval Timer on IRQ line 0. As usual, in Linux, some of these activities are executed as soon as possible right after the interrupt is raised, while the remaining activities are carried on by deferrable functions (see the later section "[Dynamic Timers](#)").

Initialization phase

During kernel initialization, the `time_init( )` function is invoked to set up the timekeeping architecture. It usually^[*] performs the following operations:

1. Initializes the `xtime` variable. The number of seconds elapsed since the midnight of January 1, 1970 is read from the Real Time Clock by means of the `get_cmos_time( )` function. The `tv_nsec` field of `xtime` is set, so that the forthcoming overflow of the `jiffies` variable will coincide with an increment of the `tv_sec` field—that is, it will fall on a second boundary.
2. Initializes the `wall_to_monotonic` variable. This variable is of the same type `timespec` as `xtime`, and it essentially stores the number of seconds and nanoseconds to be added to `xtime` in order to get a monotonic (ever increasing) flow of time. In fact, both leap seconds and synchronization with external clocks might suddenly change the `tv_sec` and `tv_nsec` fields of `xtime` so that they are no longer monotonically increased. As we'll see in the later section "[System Calls for POSIX Timers](#)," sometimes the kernel needs a truly monotonic time source.
3. If the kernel supports HPET, it invokes the `hpet_enable( )` function to determine whether the ACPI firmware has probed the chip and mapped its registers in the memory address space. In the affirmative case, `hpet_enable( )` programs the first timer of the HPET chip so that it raises the IRQ 0 interrupt 1000 times per second. Otherwise, if the HPET chip is not available, the kernel will use the PIT: the chip has already been programmed by the `init_IRQ( )` function to raise 1000 timer interrupts per second, as described in the earlier section "[Programmable Interval Timer \(PIT\)](#)."
4. Invokes `select_timer( )` to select the best timer source available in the system, and sets the `cur_timer` variable to the address of the corresponding timer object.
5. Invokes `setup_irq( 0,&irq0)` to set up the interrupt gate corresponding to IRQ0—the line associated with the system timer interrupt source (PIT or HPET). The `irq0` variable is statically defined as:

```
struct irqaction irq0 = { timer_interrupt, SA_INTERRUPT, 0,  
                          "timer", NULL, NULL };
```

From now on, the `timer_interrupt( )` function will be invoked once every tick with interrupts disabled, because the `status` field of IRQ 0's main descriptor has the `SA_INTERRUPT` flag set.

The timer interrupt handler

The `timer_interrupt( )` function is the interrupt service routine (ISR) of the PIT or of the HPET; it performs the following steps:

1. Protects the time-related kernel variables by issuing a `write_seqlock( )` on the `xtime_lock` seqlock (see the section "[Seqlocks](#)" in [Chapter 5](#)).
2. Executes the `mark_offset` method of the `cur_timer` timer object. As explained in the earlier section "[Data Structures of the Timekeeping Architecture](#)," there are four possible cases:
 - a. `cur_timer` points to the `timer_hpet` object: in this case, the HPET chip is the source of timer interrupts. The `mark_offset` method checks that no timer interrupt has been lost since the last tick; in this unlikely case, it updates `jiffies_64` accordingly. Next, the method records the current value of the periodic HPET counter.
 - b. `cur_timer` points to the `timer_pmtmr` object: in this case, the PIT chip is the source of timer interrupts, but the kernel uses the APIC Power Management Timer to measure time with a finer resolution. The `mark_offset` method checks that no timer interrupt has been lost since the last tick and updates `jiffies_64` if necessary. Then, it records the current value of the APIC Power Management Timer counter.
 - c. `cur_timer` points to the `timer_tsc` object: in this case, the PIT chip is the source of timer interrupts, but the kernel uses the Time Stamp Counter to measure time with a finer resolution. The `mark_offset` method performs the same operations as in the previous case: it checks that no timer interrupt has been lost since the last tick and updates `jiffies_64` if necessary. Then, it records the current value of the TSC counter.
 - d. `cur_timer` points to the `timer_pit` object: in this case, the PIT chip is the source of timer interrupts, and there is no other timer circuit. The `mark_offset` method does nothing.
3. Invokes the `do_timer_interrupt( )` function, which in turn performs the following actions:
 - a. Increases by one the value of `jiffies_64`. Notice that this can be done safely, because the kernel control path still holds the `xtime_lock` seqlock for writing.

- b. Invokes the `update_times( )` function to update the system date and time and to compute the current system load; these activities are discussed later in the sections "[Updating the Time and Date](#)" and "Updating System Statistics."
 - c. Invokes the `update_process_times( )` function to perform several time-related accounting operations for the local CPU (see the section "[Updating Local CPU Statistics](#)" later in this chapter).
 - d. Invokes the `profile_tick( )` function (see the section "[Profiling the Kernel Code](#)" later in this chapter).
 - e. If the system clock is synchronized with an external clock (an `adjtimex( )` system call has been previously issued), invokes the `set_rtc_mmss( )` function once every 660 seconds (every 11 minutes) to adjust the Real Time Clock. This feature helps systems on a network synchronize their clocks (see the later section "[The adjtimex\(.\) System Call](#)").
- 4. Releases the `xtime_lock` seqlock by invoking `write_sequnlock( )`.
 - 5. Returns the value 1 to notify that the interrupt has been effectively handled (see the section "[I/O Interrupt Handling](#)" in [Chapter 4](#)).

Timekeeping Architecture in Multiprocessor Systems

Multiprocessor systems can rely on two different sources of timer interrupts: those raised by the Programmable Interval Timer or the High Precision Event Timer, and those raised by the CPU local timers.

In Linux 2.6, global timer interrupts—raised by the PIT or the HPET—signal activities not related to a specific CPU, such as handling of software timers and keeping the system time up-to-date. Conversely, a CPU local timer interrupt signals timekeeping activities related to the local CPU, such as monitoring how long the current process has been running and updating the resource usage statistics.

Initialization phase

The global timer interrupt handler is initialized by the `time_init( )` function, which has already been described in the earlier section "[Timekeeping Architecture in Uniprocessor Systems](#)."

The Linux kernel reserves the interrupt vector 239 (0xef) for local timer interrupts (see [Table 4-2](#) in [Chapter 4](#)). During kernel initialization, the `apic_intr_init( )` function sets up the IDT's interrupt gate corresponding to vector 239 with the address of the low-level interrupt handler `apic_timer_interrupt( )`. Moreover, each APIC has to be told how often to generate a local time interrupt. The `calibrate_APIC_clock( )` function computes how many bus clock signals are received by the local APIC of the booting CPU during a tick (1 ms). This exact value is then used to program the local APICs in such a way to generate one local timer interrupt every tick. This is done by the `setup_APIC_timer( )` function, which is executed once for every CPU in the system.

All local APIC timers are synchronized because they are based on the common bus clock signal. This means that the value computed by `calibrate_APIC_clock( )` for the boot CPU is also good for the other CPUs in the system.

The global timer interrupt handler

The SMP version of the `timer_interrupt( )` handler differs from the UP version in a few points:

- The `do_timer_interrupt( )` function, invoked by `timer_interrupt( )`, writes into a port of the I/O APIC chip to acknowledge the timer IRQ.
- The `update_process_times( )` function is not invoked, because this function performs actions related to a specific CPU.
- The `profile_tick( )` function is not invoked, because this function also performs actions related to a specific CPU.

The local timer interrupt handler

This handler performs the timekeeping activities related to a specific CPU in the system, namely profiling the kernel code and checking how long the current process has been running on a given CPU.

The `apic_timer_interrupt( )` assembly language function is equivalent to the following code:

```
apic_timer_interrupt:
    pushl $(239-256)
    SAVE_ALL
    movl %esp, %eax
    call smp_apic_timer_interrupt
    jmp ret_from_intr
```

As you can see, the low-level handler is very similar to the other low-level interrupt handlers already described in [Chapter 4](#). The high-level interrupt handler called `smp_apic_timer_interrupt( )` executes the following steps:

1. Gets the CPU logical number (say, n).
2. Increases the `apic_timer_irqs` field of the n^{th} entry of the `irq_stat` array (see the section "[Checking the NMI Watchdogs](#)" later in this chapter).
3. Acknowledges the interrupt on the local APIC.
4. Calls the `irq_enter( )` function (see the section "[The do_IRQ\(.\) function](#)" in [Chapter 4](#)).
5. Invokes the `smp_local_timer_interrupt( )` function.
6. Calls the `irq_exit( )` function.

The `smp_local_timer_interrupt( )` function executes the per-CPU timekeeping activities. Actually, it performs the following main steps:

1. Invokes the `profile_tick( )` function (see the section "[Profiling the Kernel Code](#)" later in this chapter).
2. Invokes the `update_process_times( )` function to check how long the current process has been running and to update some local CPU statistics (see the section "[Updating Local CPU Statistics](#)" later in this chapter).

The system administrator can change the sample frequency of the kernel code profiler by writing into the `/proc/profile` file. To carry out the change, the kernel modifies the frequency at which local timer interrupts are generated. However, the `smp_local_timer_interrupt( )` function keeps invoking the `update_process_times( )` function exactly once every tick.

[*] The `time_init( )` function is executed before `mem_init( )`, which initializes the memory data structures. Unfortunately, the HPET registers are memory mapped, therefore initialization of the HPET chip has to be done after the execution of `mem_init( )`. Linux 2.6 adopts a cumbersome solution: if the kernel supports the HPET chip, the `time_init( )` function limits itself to trigger the activation of the `hpet_time_init( )` function. The latter function is executed after `mem_init( )` and performs the operations described in this section.

Updating the Time and Date

User programs get the current time and date from the `xtime` variable. The kernel must periodically update this variable, so that its value is always reasonably accurate.

The `update_times( )` function, which is invoked by the global timer interrupt handler, updates the value of the `xtime` variable as follows:

```
void update_times(void)
{
    unsigned long ticks;
    ticks = jiffies - wall_jiffies;
    if (ticks) {
        wall_jiffies += ticks;
        update_wall_time(ticks);
    }
    calc_load(ticks);
}
```

We recall from the previous description of the timer interrupt handler that when the code of this function is executed, the `xtime_lock` seqlock has already been acquired for writing.

The `wall_jiffies` variable stores the time of the last update of the `xtime` variable. Observe that the value of `wall_jiffies` can be smaller than `jiffies-1`, since a few timer interrupts can be lost, for instance when interrupts remain disabled for a long period of time; in other words, the kernel does not necessarily update the `xtime` variable at every tick. However, no tick is definitively lost, and in the long run, `xtime` stores the correct system time. The check for lost timer interrupts is done in the `mark_offset` method of `cur_timer`; see the earlier section "[Timekeeping Architecture in Uniprocessor Systems](#)."

The `update_wall_time( )` function invokes the `update_wall_time_one_tick( )` function `ticks` consecutive times; normally, each invocation adds 1,000,000 to the `xtime.tv_nsec` field. If the value of `xtime.tv_nsec` becomes greater than 999,999,999, the `update_wall_time( )` function also updates the `tv_sec` field of `xtime`. If an `adjtimex( )` system call has been issued, for reasons explained in the section "[The adjtimex\(.\) System Call](#)" later in this chapter, the function might tune the value 1,000,000 slightly so the clock speeds up or slows down a little.

The `calc_load( )` function is described in the section "[Keeping Track of System Load](#)" later in this chapter.

Updating System Statistics

The kernel, among the other time-related duties, must periodically collect various data used for:

- Checking the CPU resource limit of the running processes
- Updating statistics about the local CPU workload
- Computing the average system load
- Profiling the kernel code

Updating Local CPU Statistics

We have mentioned that the `update_process_times( )` function is invoked—either by the global timer interrupt handler on uniprocessor systems or by the local timer interrupt handler in multiprocessor systems—to update some kernel statistics. This function performs the following steps:

1. Checks how long the current process has been running. Depending on whether the current process was running in User Mode or in Kernel Mode when the timer interrupt occurred, invokes either `account_user_time( )` or `account_system_time( )`. Each of these functions performs essentially the following steps:
 - a. Updates either the `utime` field (ticks spent in User Mode) or the `stime` field (ticks spent in Kernel Mode) of the current process descriptor. Two additional fields called `cutime` and `cstime` are provided in the process descriptor to count the number of CPU ticks spent by the process children in User Mode and Kernel Mode, respectively. For reasons of efficiency, these fields are not updated by `update_process_times( )`, but rather when the parent process queries the state of one of its children (see the section "[Destroying Processes](#)" in [Chapter 3](#)).
 - b. Checks whether the total CPU time limit has been reached; if so, sends `SIGXCPU` and `SIGKILL` signals to `current`. The section "[Process Resource Limits](#)" in [Chapter 3](#) describes how the limit is controlled by the `signal->rlim[RLIMIT_CPU].rlim_cur` field of each process descriptor.
 - c. Invokes `account_it_virt( )` and `account_it_prof( )` to check the process timers (see the section "[The setitimer\(.\) and alarm\(.\) System](#)

[Calls](#)" later in this chapter).

- d. Updates some kernel statistics stored in the `kstat` per-CPU variable.
2. Invokes `raise_softirq( )` to activate the `TIMER_SOFTIRQ` tasklet on the local CPU (see the section "[Software Timers and Delay Functions](#)" later in this chapter).
3. If some old version of an RCU-protected data structure has to be reclaimed, checks whether the local CPU has gone through a quiescent state and invokes `tasklet_schedule( )` to activate the `rcu_tasklet` tasklet of the local CPU (see the section "[Read-Copy Update \(RCU\)](#)" in [Chapter 5](#)).
4. Invokes the `scheduler_tick( )` function, which decreases the time slice counter of the current process, and checks whether its quantum is exhausted. We'll discuss in depth these operations in the section "[The scheduler tick\(\), Function](#)" in [Chapter 7](#).

Keeping Track of System Load

Every Unix kernel keeps track of how much CPU activity is being carried on by the system. These statistics are used by various administration utilities such as `top`. A user who enters the `uptime` command sees the statistics as the "load average" relative to the last minute, the last 5 minutes, and the last 15 minutes. On a uniprocessor system, a value of 0 means that there are no active processes (besides the *swapper* process 0) to run, while a value of 1 means that the CPU is 100 percent busy with a single process, and values greater than 1 mean that the CPU is shared among several active processes.^[*]

At every tick, `update_times( )` invokes the `calc_load( )` function, which counts the number of processes in the `TASK_RUNNING` or `TASK_UNINTERRUPTIBLE` state and uses this number to update the average system load.

Profiling the Kernel Code

Linux includes a minimalist code profiler called *readprofile* used by Linux developers to discover where the kernel spends its time in Kernel Mode. The profiler identifies the *hot spots* of the kernel — the most frequently executed fragments of kernel code. Identifying the kernel hot spots is very important, because they may point out kernel functions that should be further optimized.

The profiler is based on a simple Monte Carlo algorithm: at every timer interrupt occurrence, the kernel determines whether the interrupt occurred in Kernel Mode; if so, the kernel fetches the value of the `eip` register before the interruption from the stack and uses it to discover what the kernel was doing before the interrupt. In the long run, the samples accumulate on the hot spots.

The `profile_tick( )` function collects the data for the code profiler. It is invoked either by the `do_timer_interrupt( )` function in uniprocessor systems (by the global timer interrupt handler) or by the `smp_local_timer_interrupt( )` function in multiprocessor systems (by the local timer interrupt handler).

To enable the code profiler, the Linux kernel must be booted by passing as a parameter the string `profile=N`, where 2^N denotes the size of the code fragments to be profiled. The collected data can be read from the `/proc/profile` file. The counters are reset by writing in the same file; in multiprocessor systems, writing into the file can also change the sample frequency (see the earlier section "[Timekeeping Architecture in Multiprocessor Systems](#)"). However, kernel developers do not usually access `/proc/profile` directly; instead, they use the `readprofile` system command.

The Linux 2.6 kernel includes yet another profiler called *oprofile*. Besides being more flexible and customizable than *readprofile*, *oprofile* can be used to discover hot spots in kernel code, User Mode applications, and system libraries. When *oprofile* is being used, `profile_tick( )` invokes the `timer_notify( )` function to collect the data used by this new profiler.

Checking the NMI Watchdogs

In multiprocessor systems, Linux offers yet another feature to kernel developers: a *watchdog system*, which might be quite useful to detect kernel bugs that cause a system freeze. To activate such a watchdog, the kernel must be booted with the `nmi_watchdog` parameter.

The watchdog is based on a clever hardware feature of local and I/O APICs: they can generate periodic NMI interrupts on every CPU. Because NMI interrupts are not masked by the `cli` assembly language instruction, the watchdog can detect deadlocks even when interrupts are disabled.

As a consequence, once every tick, all CPUs, regardless of what they are doing, start executing the NMI interrupt handler; in turn, the handler invokes `do_nmi( )`. This function gets the logical number *n* of the CPU, and then checks the `apic_timer_irqs`

field of the n^{th} entry of `irq_stat` (see [Table 4-8](#) in [Chapter 4](#)). If the CPU is working properly, the value must be different from the value read at the previous NMI interrupt. When the CPU is running properly, the n^{th} entry of the `apic_timer_irqs` field is increased by the local timer interrupt handler (see the earlier section "[The local timer interrupt handler](#)"); if the counter is not increased, the local timer interrupt handler has not been executed in a whole tick. Not a good thing, you know.

When the NMI interrupt handler detects a CPU freeze, it rings all the bells: it logs scary messages in the system logfiles, dumps the contents of the CPU registers and of the kernel stack (kernel oops), and finally kills the current process. This gives kernel developers a chance to discover what's gone wrong.

[*] Linux includes in the load average all processes that are in the `TASK_RUNNING` and `TASK_UNINTERRUPTIBLE` states. However, under normal conditions, there are few `TASK_UNINTERRUPTIBLE` processes, so a high load usually means that the CPU is busy.

Software Timers and Delay Functions

A *timer* is a software facility that allows functions to be invoked at some future moment, after a given time interval has elapsed; a *time-out* denotes a moment at which the time interval associated with a timer has elapsed.

Timers are widely used both by the kernel and by processes. Most device drivers use timers to detect anomalous conditions — floppy disk drivers, for instance, use timers to switch off the device motor after the floppy has not been accessed for a while, and parallel printer drivers use them to detect erroneous printer conditions.

Timers are also used quite often by programmers to force the execution of specific functions at some future time (see the later section "[The `setitimer\(\)` and `alarm\(\)` System Calls](#)").

Implementing a timer is relatively easy. Each timer contains a field that indicates how far in the future the timer should expire. This field is initially calculated by adding the right number of ticks to the current value of `jiffies`. The field does not change. Every time the kernel checks a timer, it compares the expiration field to the value of `jiffies` at the current moment, and the timer expires when `jiffies` is greater than or equal to the stored value.

Linux considers two types of timers called *dynamic timers* and *interval timers*. The first type is used by the kernel, while interval timers may be created by processes in User Mode.

One word of caution about Linux timers: since checking for timer functions is always done by deferrable functions that may be executed a long time after they have been activated, the kernel cannot ensure that timer functions will start right at their expiration times. It can only ensure that they are executed either at the proper time or after with a delay of up to a few hundreds of milliseconds. For this reason, timers are not appropriate for real-time applications in which expiration times must be strictly enforced.

Besides software timers, the kernel also makes use of *delay functions*, which execute a tight instruction loop until a given time interval elapses. We will discuss them in the later section "[Delay Functions](#)."

Dynamic Timers

Dynamic timers may be dynamically created and destroyed. No limit is placed on the number of currently active dynamic timers.

A dynamic timer is stored in the following `timer_list` structure:

```
struct timer_list {
    struct list_head entry;
    unsigned long expires;
    spinlock_t lock;
    unsigned long magic;
    void (*function)(unsigned long);
    unsigned long data;
    tvec_base_t *base;
};
```

The `function` field contains the address of the function to be executed when the timer expires. The `data` field specifies a parameter to be passed to this timer function.

Thanks to the `data` field, it is possible to define a single general-purpose function that handles the time-outs of several device drivers; the `data` field could store the device ID or other meaningful data that could be used by the function to differentiate the device.

The `expires` field specifies when the timer expires; the time is expressed as the number of ticks that have elapsed since the system started up. All timers that have an `expires` value smaller than or equal to the value of `jiffies` are considered to be expired or decayed.

The `entry` field is used to insert the software timer into one of the doubly linked circular lists that group together the timers according to the value of their `expires` field. The algorithm that uses these lists is described later in this chapter.

To create and activate a dynamic timer, the kernel must:

1. Create, if necessary, a new `timer_list` object — for example, `t`. This can be done in several ways by:
 - Defining a static global variable in the code.
 - Defining a local variable inside a function; in this case, the object is stored on the Kernel Mode stack.
 - Including the object in a dynamically allocated descriptor.
2. Initialize the object by invoking the `init_timer(&t)` function. This essentially sets the `t.base` pointer field to NULL and sets the `t.lock` spin lock to "open."

3. Load the `function` field with the address of the function to be activated when the timer decays. If required, load the `data` field with a parameter value to be passed to the function.
4. If the dynamic timer is not already inserted in a list, assign a proper value to the `expires` field and invoke the `add_timer(&t)` function to insert the `t` element in the proper list.
5. Otherwise, if the dynamic timer is already inserted in a list, update the `expires` field by invoking the `mod_timer( )` function, which also takes care of moving the object into the proper list (discussed next).

Once the timer has decayed, the kernel automatically removes the `t` element from its list. Sometimes, however, a process should explicitly remove a timer from its list using the `del_timer( )`, `del_timer_sync( )`, or `del_singleshot_timer_sync( )` functions. Indeed, a sleeping process may be woken up before the time-out is over; in this case, the process may choose to destroy the timer. Invoking `del_timer( )` on a timer already removed from a list does no harm, so removing the timer within the timer function is considered a good practice.

In Linux 2.6, a dynamic timer is bound to the CPU that activated it—that is, the timer function will always run on the same CPU that first executed the `add_timer( )` or later the `mod_timer( )` function. The `del_timer( )` and companion functions, however, can deactivate every dynamic timer, even if it is not bound to the local CPU.

Dynamic timers and race conditions

Being asynchronously activated, dynamic timers are prone to race conditions. For instance, consider a dynamic timer whose function acts on a discardable resource (e.g., a kernel module or a file data structure). Releasing the resource without stopping the timer may lead to data corruption if the timer function got activated when the resource no longer exists. Thus, a rule of thumb is to stop the timer *before* releasing the resource:

```
...
del_timer(&t);
X_Release_Resources( );
...
```

In multiprocessor systems, however, this code is not safe because the timer function might already be running on another CPU when `del_timer( )` is invoked. As a result, resources may be released while the timer function is still acting on them. To avoid this kind of race condition, the kernel offers the `del_timer_sync( )` function. It

removes the timer from the list, and then it checks whether the timer function is executed on another CPU; in such a case, `del_timer_sync( )` waits until the timer function terminates.

The `del_timer_sync( )` function is rather complex and slow, because it has to carefully take into consideration the case in which the timer function reactivates itself. If the kernel developer knows that the timer function never reactivates the timer, she can use the simpler and faster `del_singleshot_timer_sync( )` function to deactivate a timer and wait until the timer function terminates.

Other types of race conditions exist, of course. For instance, the right way to modify the `expires` field of an already activated timer consists of using `mod_timer( )`, rather than deleting the timer and re-creating it thereafter. In the latter approach, two kernel control paths that want to modify the `expires` field of the same timer may mix each other up badly. The implementation of the timer functions is made SMP-safe by means of the lock spin lock included in every `timer_list` object: every time the kernel must access a dynamic timer, it disables the interrupts and acquires this spin lock.

Data structures for dynamic timers

Choosing the proper data structure to implement dynamic timers is not easy. Stringing together all timers in a single list would degrade system performance, because scanning a long list of timers at every tick is costly. On the other hand, maintaining a sorted list would not be much more efficient, because the insertion and deletion operations would also be costly.

The adopted solution is based on a clever data structure that partitions the `expires` values into blocks of ticks and allows dynamic timers to percolate efficiently from lists with larger `expires` values to lists with smaller ones. Moreover, in multiprocessor systems the set of active dynamic timers is split among the various CPUs.

The main data structure for dynamic timers is a per-CPU variable (see the section "[Per-CPU Variables](#)" in [Chapter 5](#)) named `tvec_bases`: it includes `NR_CPUS` elements, one for each CPU in the system. Each element is a `tvec_base_t` structure, which includes all data needed to handle the dynamic timers bound to the corresponding CPU:

```
typedef struct tvec_t_base_s {
    spinlock_t lock;
    unsigned long timer_jiffies;
    struct timer_list *running_timer;
```

```

    tvec_root_t tv1;
    tvec_t tv2;
    tvec_t tv3;
    tvec_t tv4;
    tvec_t tv5;
} tvec_base_t;

```

The `tv1` field is a structure of type `tvec_root_t`, which includes a `vec` array of 256 `list_head` elements — that is, lists of dynamic timers. It contains all dynamic timers, if any, that will decay within the next 255 ticks.

The `tv2`, `tv3`, and `tv4` fields are structures of type `tvec_t` consisting of a `vec` array of 64 `list_head` elements. These lists contain all dynamic timers that will decay within the next $2^{14}-1$, $2^{20}-1$, and $2^{26}-1$ ticks, respectively.

The `tv5` field is identical to the previous ones, except that the last entry of the `vec` array is a list that includes dynamic timers with extremely large `expires` fields. It never needs to be replenished from another array. [Figure 6-1](#) illustrates in a schematic way the five groups of lists.

The `timer_jiffies` field represents the earliest expiration time of the dynamic timers yet to be checked: if it coincides with the value of `jiffies`, no backlog of deferrable functions has accumulated; if it is smaller than `jiffies`, then lists of dynamic timers that refer to previous ticks must be dealt with. The field is set to `jiffies` at system startup and is increased only by the `run_timer_softirq( )` function described in the next section. Notice that the `timer_jiffies` field might drop a long way behind `jiffies` when the deferrable functions that handle dynamic timers are not executed for a long time—for instance because these functions have been disabled or because a large number of interrupt handlers have been executed.

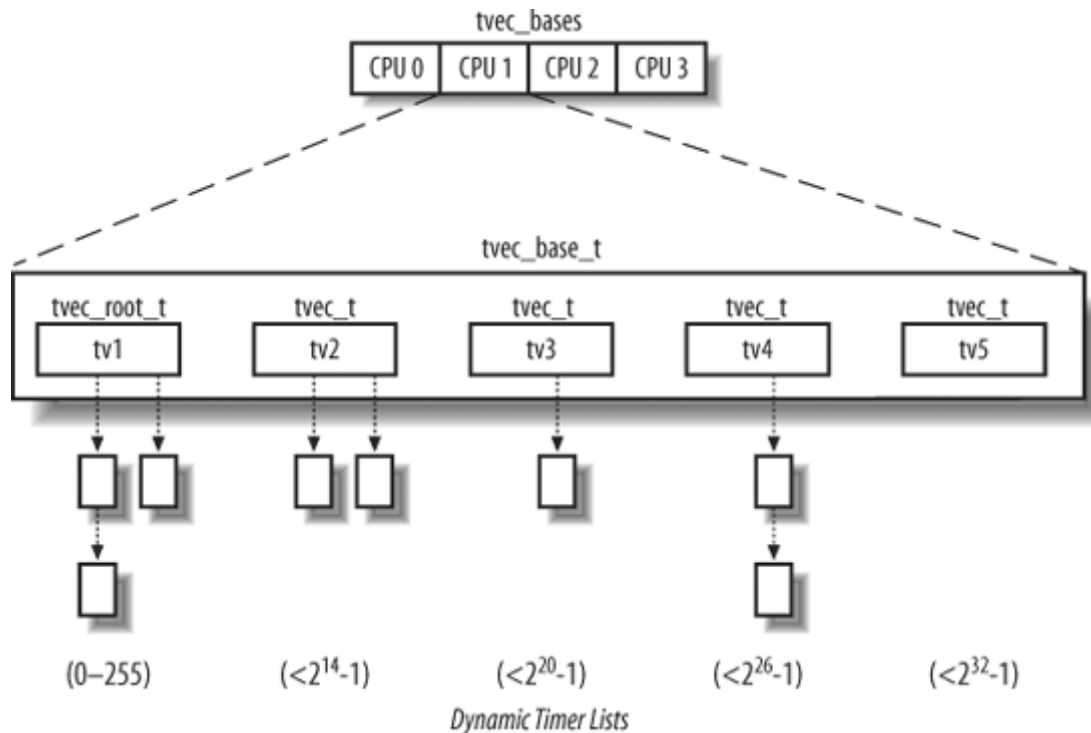


Figure 6-1. The groups of lists associated with dynamic timers

In multiprocessor systems, the `running_timer` field points to the `timer_list` structure of the dynamic timer that is currently handled by the local CPU.

Dynamic timer handling

Despite the clever data structures, handling software timers is a time-consuming activity that should not be performed by the timer interrupt handler. In Linux 2.6 this activity is carried on by a deferrable function, namely the `TIMER_SOFTIRQ` softirq.

The `run_timer_softirq( )` function is the deferrable function associated with the `TIMER_SOFTIRQ` softirq. It essentially performs the following actions:

1. Stores in the `base` local variable the address of the `tvec_base_t` data structure associated with the local CPU.
2. Acquires the `base->lock` spin lock and disables local interrupts.
3. Starts a `while` loop, which ends when `base->timer_jiffies` becomes greater than the value of `jiffies`. In every single execution of the cycle, performs the following substeps:
 - a. Computes the index of the list in `base->tv1` that holds the next timers to be handled:

```
index = base->timer_jiffies & 255;
```


- b. If `index` is zero, all lists in `base->tv1` have been checked, so they are empty: the function therefore percolates the dynamic timers by invoking `cascade( )`:

```
if (!index &&
    (!cascade(base, &base->tv2, (base->timer_jiffies>> 8)&63)) &&
    (!cascade(base, &base->tv3, (base->timer_jiffies>>14)&63)) &&
    (!cascade(base, &base->tv4, (base->timer_jiffies>>20)&63)))
    cascade(base, &base->tv5, (base->timer_jiffies>>26)&63);
```

Consider the first invocation of the `cascade( )` function: it receives as arguments the address in `base`, the address of `base->tv2`, and the index of the list in `base->tv2` including the timers that will decay in the next 256 ticks. This index is determined by looking at the proper bits of the `base->timer_jiffies` value. `cascade( )` moves all dynamic timers in the `base->tv2` list into the proper lists of `base->tv1`; then, it returns a positive value, unless all `base->tv2` lists are now empty. If so, `cascade( )` is invoked once more to replenish `base->tv2` with the timers included in a list of `base->tv3`, and so on.

- c. Increases by one `base->timer_jiffies`.
- d. For each dynamic timer in the `base->tv1.vec[index]` list, executes the corresponding timer function. In particular, for each `timer_list` element `t` in the list essentially performs the following steps:
- Removes `t` from the `base->tv1`'s list.
 - In multiprocessor systems, sets `base->running_timer` to `&t`.
 - Sets `t.base` to `NULL`.
 - Releases the `base->lock` spin lock, and enables local interrupts.
 - Executes the timer function `t.function` passing as argument `t.data`.
 - Acquires the `base->lock` spin lock, and disables local interrupts.
 - Continues with the next timer in the list, if any.
- e. All timers in the list have been handled. Continues with the next iteration of the outermost `while` cycle.
4. The outermost `while` cycle is terminated, which means that all decayed timers have been handled. In multiprocessor systems, sets `base->running_timer` to `NULL`.

5. Releases the `base->lock` spin lock and enables local interrupts.

Because the values of `jiffies` and `timer_jiffies` usually coincide, the outermost `while` cycle is often executed only once. In general, the outermost loop is executed `jiffies - base->timer_jiffies + 1` consecutive times. Moreover, if a timer interrupt occurs while `run_timer_softirq( )` is being executed, dynamic timers that decay at this tick occurrence are also considered, because the `jiffies` variable is asynchronously increased by the global timer interrupt handler (see the earlier section "[The timer interrupt handler](#)").

Notice that `run_timer_softirq( )` disables interrupts and acquires the `base->lock` spin lock just before entering the outermost loop; interrupts are enabled and the spin lock is released right before invoking each dynamic timer function, until its termination. This ensures that the dynamic timer data structures are not corrupted by interleaved kernel control paths.

To sum up, this rather complex algorithm ensures excellent performance. To see why, assume for the sake of simplicity that the `TIMER_SOFTIRQ` `softirq` is executed right after the corresponding timer interrupt occurs. Then, in 255 timer interrupt occurrences out of 256 (in 99.6% of the cases), the `run_timer_softirq( )` function just runs the functions of the decayed timers, if any. To replenish `base->tv1.vec` periodically, it is sufficient 63 times out of 64 to partition one list of `base->tv2` into the 256 lists of `base->tv1`. The `base->tv2.vec` array, in turn, must be replenished in 0.006 percent of the cases (that is, once every 16.4 seconds). Similarly, `base->tv3.vec` is replenished every 17 minutes and 28 seconds, and `base->tv4.vec` is replenished every 18 hours and 38 minutes. `base->tv5.vec` doesn't need to be replenished.

An Application of Dynamic Timers: the `nanosleep( )` System Call

To show how the outcomes of all the previous activities are actually used in the kernel, we'll show an example of the creation and use of a *process time-out*.

Let's consider the service routine of the `nanosleep( )` system call, that is, `sys_nanosleep( )`, which receives as its parameter a pointer to a `timespec` structure and suspends the invoking process until the specified time interval elapses. The service routine first invokes `copy_from_user( )` to copy the values contained in the User

Mode `timespec` structure into the local variable `t`. Assuming that the `timespec` structure defines a non-null delay, the function then executes the following code:

```
current->state = TASK_INTERRUPTIBLE;
remaining = schedule_timeout(timespec_to_jiffies(&t)+1);
```

The `timespec_to_jiffies( )` function converts in ticks the time interval stored in the `timespec` structure. To be on the safe side, `sys_nanosleep( )` adds one tick to the value computed by `timespec_to_jiffies( )`.

The kernel implements process time-outs by using dynamic timers. They appear in the `schedule_timeout( )` function, which essentially executes the following statements:

```
struct timer_list timer;
unsigned long expire = timeout + jiffies;
init_timer(&timer);
timer.expires = expire;
timer.data = (unsigned long) current;
timer.function = process_timeout;
add_timer(&timer);
schedule( ); /* process suspended until timer expires */
del_singleshot_timer_sync(&timer);
timeout = expire - jiffies;
return (timeout < 0 ? 0 : timeout);
```

When `schedule( )` is invoked, another process is selected for execution; when the former process resumes its execution, the function removes the dynamic timer. In the last statement, the function either returns 0, if the time-out is expired, or the number of ticks left to the time-out expiration if the process was awakened for some other reason.

When the time-out expires, the timer's function is executed:

```
void process_timeout(unsigned long __data)
{
    wake_up_process((task_t *)__data);
}
```

The `process_timeout( )` receives as its parameter the process descriptor pointer stored in the `data` field of the `timer` object. As a result, the suspended process is awakened.

Once awakened, the process continues the execution of the `sys_nanosleep( )` system call. If the value returned by `schedule_timeout( )` specifies that the process time-out is expired (value zero), the system call terminates. Otherwise, the system call is automatically restarted, as explained in the section "[Reexecution of System Calls](#)" in [Chapter 11](#).

Delay Functions

Software timers are useless when the kernel must wait for a short time interval—let's say, less than a few milliseconds. For instance, often a device driver has to wait for a predefined number of microseconds until the hardware completes some operation. Because a dynamic timer has a significant setup overhead and a rather large minimum wait time (1 millisecond), the device driver cannot conveniently use it.

In these cases, the kernel makes use of the `udelay( )` and `ndelay( )` functions: the former receives as its parameter a time interval in microseconds and returns after the specified delay has elapsed; the latter is similar, but the argument specifies the delay in nanoseconds.

Essentially, the two functions are defined as follows:

```
void udelay(unsigned long usecs)
{
    unsigned long loops;
    loops = (usecs*HZ*current_cpu_data.loops_per_jiffy)/1000000;
    cur_timer->delay(loops);
}

void ndelay(unsigned long nsecs)
{
    unsigned long loops;
    loops = (nsecs*HZ*current_cpu_data.loops_per_jiffy)/1000000000;
    cur_timer->delay(loops);
}
```

Both functions rely on the `delay` method of the `cur_timer` timer object (see the earlier section "[Data Structures of the Timekeeping Architecture](#)"), which receives as its parameter a time interval in "loops." The exact duration of one "loop," however, depends on the timer object referred by `cur_timer` (see [Table 6-2](#) earlier in this chapter):

- If `cur_timer` points to the `timer_hpet`, `timer_pmtmr`, and `timer_tsc` objects, one "loop" corresponds to one CPU cycle—that is, the time interval between two consecutive CPU clock signals (see the earlier section "[Time Stamp Counter \(TSC\)](#)").
- If `cur_timer` points to the `timer_none` or `timer_pit` objects, one "loop" corresponds to the time duration of a single iteration of a tight instruction loop.

During the initialization phase, after `cur_timer` has been set up by `select_timer( )`, the kernel executes the `calibrate_delay( )` function, which determines how many "loops" fit in a tick. This value is then saved in the

`current_cpu_data.loops_per_jiffy` variable, so that it can be used by `udelay( )` and `ndelay( )` to convert microseconds and nanoseconds, respectively, to "loops."

Of course, the `cur_timer->delay( )` method makes use of the HPET or TSC hardware circuitry, if available, to get an accurate measurement of time. Otherwise, if no HPET or TSC is available, the method executes `loops` iterations of a tight instruction loop.

System Calls Related to Timing Measurements

Several system calls allow User Mode processes to read and modify the time and date and to create timers. Let's briefly review these and discuss how the kernel handles them.

The `time( )` and `gettimeofday( )` System Calls

Processes in User Mode can get the current time and date by means of several system calls:

`time( )`

Returns the number of elapsed seconds since midnight at the start of January 1, 1970 (UTC).

`gettimeofday( )`

Returns, in a data structure named `timeval`, the number of elapsed seconds since midnight of January 1, 1970 (UTC) and the number of elapsed microseconds in the last second (a second data structure named `timezone` is not currently used).

The `time( )` system call is superseded by `gettimeofday( )`, but it is still included in Linux for backward compatibility. Another widely used function, `ftime( )`, which is no longer implemented as a system call, returns the number of elapsed seconds since midnight of January 1, 1970 (UTC) and the number of elapsed milliseconds in the last second.

The `gettimeofday( )` system call is implemented by the `sys_gettimeofday( )` function. To compute the current date and time of the day, this function invokes `do_gettimeofday( )`, which executes the following actions:

1. Acquires the `xtime_lock` seqlock for reading.
2. Determines the number of microseconds elapsed since the last timer interrupt by invoking the `get_offset` method of the `cur_timer` timer object:

```
usec = cur_timer->getoffset( );
```

As explained in the earlier section "[Data Structures of the Timekeeping Architecture](#)," there are four possible cases:

- a. If `cur_timer` points to the `timer_hpet` object, the method compares the current value of the HPET counter with the value of the same counter

saved in the last execution of the timer interrupt handler.

- b. If `cur_timer` points to the `timer_pmtmr` object, the method compares the current value of the ACPI PMT counter with the value of the same counter saved in the last execution of the timer interrupt handler.
- c. If `cur_timer` points to the `timer_tsc` object, the method compares the current value of the Time Stamp Counter with the value of the TSC saved in the last execution of the timer interrupt handler.
- d. If `cur_timer` points to the `timer_pit` object, the method reads the current value of the PIT counter to compute the number of microseconds elapsed since the last PIT's timer interrupt.

3. If some timer interrupt has been lost (see the section "[Updating the Time and Date](#)" earlier in this chapter), the function adds to `usec` the corresponding delay:

```
usec += (jiffies - wall_jiffies) * 1000;
```

4. Adds to `usec` the microseconds elapsed in the last second:

```
usec += (xtime.tv_nsec / 1000);
```

5. Copies the contents of `xtime` into the user-space buffer specified by the system call parameter `tv`, adding to the microseconds field the value of `usec`:

```
tv->tv_sec = xtime->tv_sec;  
tv->tv_usec = xtime->tv_usec + usec;
```

6. Invokes `read_seqretry( )` on the `xtime_lock` seqlock, and jumps back to step 1 if another kernel control path has concurrently acquired `xtime_lock` for writing.

7. Checks for an overflow in the microseconds field, adjusting both that field and the second field if necessary:

```
while (tv->tv_usec >= 1000000) {  
    tv->tv_usec -= 1000000;  
    tv->tv_sec++;  
}
```

Processes in User Mode with root privilege may modify the current date and time by using either the obsolete `stime( )` or the `settimeofday( )` system call. The `sys_settimeofday( )` function invokes `do_settimeofday( )`, which executes operations complementary to those of `do_gettimeofday( )`.

Notice that both system calls modify the value of `xtime` while leaving the RTC registers unchanged. Therefore, the new time is lost when the system shuts down,

unless the user executes the *clock* program to change the RTC value.

The `adjtimex( )` System Call

Although clock drift ensures that all systems eventually move away from the correct time, changing the time abruptly is both an administrative nuisance and risky behavior. Imagine, for instance, programmers trying to build a large program and depending on file timestamps to make sure that out-of-date object files are recompiled. A large change in the system's time could confuse the `make` program and lead to an incorrect build. Keeping the clocks tuned is also important when implementing a distributed filesystem on a network of computers. In this case, it is wise to adjust the clocks of the interconnected PCs, so that the timestamp values associated with the inodes of the accessed files are coherent. Thus, systems are often configured to run a time synchronization protocol such as Network Time Protocol (NTP) on a regular basis to change the time gradually at each tick. This utility depends on the `adjtimex( )` system call in Linux.

This system call is present in several Unix variants, although it should not be used in programs intended to be portable. It receives as its parameter a pointer to a `timex` structure, updates kernel parameters from the values in the `timex` fields, and returns the same structure with current kernel values. Such kernel values are used by `update_wall_time_one_tick( )` to slightly adjust the number of microseconds added to `xtime.tv_usec` at each tick.

The `setitimer( )` and `alarm( )` System Calls

Linux allows User Mode processes to activate special timers called *interval timers*.^[*] The timers cause Unix signals (see [Chapter 11](#)) to be sent periodically to the process. It is also possible to activate an interval timer so that it sends just one signal after a specified delay. Each interval timer is therefore characterized by:

- The frequency at which the signals must be emitted, or a null value if just one signal has to be generated
- The time remaining until the next signal is to be generated

The earlier warning about accuracy applies to these timers. They are guaranteed to execute after the requested time has elapsed, but it is impossible to predict exactly when they will be delivered.

Interval timers are activated by means of the POSIX `setitimer( )` system call. The first parameter specifies which of the following policies should be adopted:

ITIMER_REAL

The actual elapsed time; the process receives `SIGALRM` signals.

ITIMER_VIRTUAL

The time spent by the process in User Mode; the process receives `SIGVTALRM` signals.

ITIMER_PROF

The time spent by the process both in User and in Kernel Mode; the process receives `SIGPROF` signals.

The interval timers can be either single-shot or periodic. The second parameter of `setitimer( )` points to a structure of type `itimerval` that specifies the initial duration of the timer (in seconds and nanoseconds) and the duration to be used when the timer is automatically reactivated (or zero for single-shot timers). The third parameter of `setitimer( )` is an optional pointer to an `itimerval` structure that is filled by the system call with the previous timer parameters.

To implement an interval timer for each of the preceding policies, the process descriptor includes three pairs of fields:

- `it_real_incr` and `it_real_value`
- `it_virt_incr` and `it_virt_value`
- `it_prof_incr` and `it_prof_value`

The first field of each pair stores the interval in ticks between two signals; the other field stores the current value of the timer.

The `ITIMER_REAL` interval timer is implemented by using dynamic timers because the kernel must send signals to the process even when it is not running on the CPU.

Therefore, each process descriptor includes a dynamic timer object called `real_timer`. The `setitimer( )` system call initializes the `real_timer` fields and then invokes `add_timer( )` to insert the dynamic timer in the proper list. When the timer expires, the kernel executes the `it_real_fn( )` timer function. In turn, the `it_real_fn( )` function sends a `SIGALRM` signal to the process; then, if `it_real_incr` is not null, it sets the `expires` field again, reactivating the timer.

The `ITIMER_VIRTUAL` and `ITIMER_PROF` interval timers do not require dynamic timers, because they can be updated while the process is running. The `account_it_virt( )` and `account_it_prof( )` functions are invoked by `update_process_times( )`, which is called either by the PIT's timer interrupt handler (UP) or by the local timer interrupt handlers (SMP). Therefore, the two interval timers are updated once every tick, and if they are expired, the proper signal is sent to the current process.

The `alarm( )` system call sends a `SIGALRM` signal to the calling process when a specified time interval has elapsed. It is very similar to `setitimer( )` when invoked with the `ITIMER_REAL` parameter, because it uses the `real_timer` dynamic timer included in the process descriptor. Therefore, `alarm( )` and `setitimer( )` with parameter `ITIMER_REAL` cannot be used at the same time.

System Calls for POSIX Timers

The POSIX 1003.1b standard introduced a new type of software timers for User Mode programs—in particular, for multithreaded and real-time applications. These timers are often referred to as *POSIX timers*.

Every implementation of the POSIX timers must offer to the User Mode programs a few *POSIX clocks*, that is, virtual time sources having predefined resolutions and properties. Whenever an application wants to make use of a POSIX timer, it creates a new timer resource specifying one of the existing POSIX clocks as the timing base. The system calls that allow users to handle POSIX clocks and timers are listed in [Table 6-3](#).

Table 6-3. System calls for POSIX timers and clocks

System call	Description
<code>clock_gettime()</code>	Gets the current value of a POSIX clock
<code>clock_settime()</code>	Sets the current value of a POSIX clock
<code>clock_getres()</code>	Gets the resolution of a POSIX clock
<code>timer_create()</code>	Creates a new POSIX timer based on a specified POSIX clock
<code>timer_gettime()</code>	Gets the current value and increment of a POSIX timer
<code>timer_settime()</code>	Sets the current value and increment of a POSIX timer
<code>timer_getoverrun()</code>	Gets the number of overruns of a decayed POSIX timer
<code>timer_delete()</code>	Destroys a POSIX timer
<code>clock_nanosleep()</code>	Puts the process to sleep using a POSIX clock as time source

The Linux 2.6 kernel offers two types of POSIX clocks:

CLOCK_REALTIME

This virtual clock represents the real-time clock of the system—essentially the value of the `xtime` variable (see the earlier section "[Updating the Time and Date](#)"). The resolution returned by the `clock_getres( )` system call is 999,848 nanoseconds, which corresponds to roughly 1000 updates of `xtime` in a second.

CLOCK_MONOTONIC

This virtual clock represents the real-time clock of the system purged of every time warp due to the synchronization with an external time source. Essentially, this virtual clock is represented by the sum of the two variables `xtime` and `wall_to_monotonic` (see the earlier section "[Timekeeping Architecture in Uniprocessor Systems](#)"). The resolution of this POSIX clock, returned by `clock_getres( )`, is 999,848 nanoseconds.

The Linux kernel implements the POSIX timers by means of dynamic timers. Thus, they are similar to the traditional `ITIMER_REAL` interval timers we described in the previous section. POSIX timers, however, are much more flexible and reliable than traditional interval timers. A couple of significant differences between them are:

- When a traditional interval timer decays, the kernel always sends a `SIGALRM` signal to the process that activated the timer. Instead, when a POSIX timer decays, the kernel can send every kind of signal, either to the whole multithreaded application or to a single specified thread. The kernel can also force the execution of a notifier function in a thread of the application, or it can even do nothing (it is up to a User Mode library to handle the event).
- If a traditional interval timer decays many times but the User Mode process cannot receive the `SIGALRM` signal (for instance because the signal is blocked or the process is not running), only the first signal is received: all other occurrences of `SIGALRM` are lost. The same holds for POSIX timers, but the process can invoke the `timer_getoverrun( )` system call to get the number of times the timer decayed since the generation of the first signal.

[*] These software constructs have nothing in common with the Programmable Interval Timer chip described earlier in this chapter.

Chapter 7. Process Scheduling

Like every time sharing system, Linux achieves the magical effect of an apparent simultaneous execution of multiple processes by switching from one process to another in a very short time frame. Process switching itself was discussed in [Chapter 3](#); this chapter deals with *scheduling*, which is concerned with when to switch and which process to choose.

The chapter consists of three parts. The section "[Scheduling Policy](#)." introduces the choices made by Linux in the abstract to schedule processes. The section "[The Scheduling Algorithm](#)" discusses the data structures used to implement scheduling and the corresponding algorithm. Finally, the section "[System Calls Related to Scheduling](#)" describes the system calls that affect process scheduling.

To simplify the description, we refer as usual to the 80×86 architecture; in particular, we assume that the system uses the Uniform Memory Access model, and that the system tick is set to 1 ms.

Scheduling Policy

The scheduling algorithm of traditional Unix operating systems must fulfill several conflicting objectives: fast process response time, good throughput for background jobs, avoidance of process starvation, reconciliation of the needs of low- and high-priority processes, and so on. The set of rules used to determine when and how to select a new process to run is called *scheduling policy*.

Linux scheduling is based on the *time sharing* technique: several processes run in "time multiplexing" because the CPU time is divided into *slices*, one for each runnable process.^[*] Of course, a single processor can run only one process at any given instant. If a currently running process is not terminated when its time slice or *quantum* expires, a process switch may take place. Time sharing relies on timer interrupts and is thus transparent to processes. No additional code needs to be inserted in the programs to ensure CPU time sharing.

The scheduling policy is also based on ranking processes according to their priority. Complicated algorithms are sometimes used to derive the current priority of a process, but the end result is the same: each process is associated with a value that tells the scheduler how appropriate it is to let the process run on a CPU.

In Linux, process priority is dynamic. The scheduler keeps track of what processes are doing and adjusts their priorities periodically; in this way, processes that have been denied the use of a CPU for a long time interval are boosted by dynamically increasing their priority. Correspondingly, processes running for a long time are penalized by decreasing their priority.

When speaking about scheduling, processes are traditionally classified as *I/O-bound* or *CPU-bound*. The former make heavy use of I/O devices and spend much time waiting for I/O operations to complete; the latter carry on number-crunching applications that require a lot of CPU time.

An alternative classification distinguishes three classes of processes:

Interactive processes

These interact constantly with their users, and therefore spend a lot of time waiting for keypresses and mouse operations. When input is received, the process must be woken up quickly, or the user will find the system to be unresponsive. Typically, the average delay must fall between 50 and 150 milliseconds. The

variance of such delay must also be bounded, or the user will find the system to be erratic. Typical interactive programs are command shells, text editors, and graphical applications.

Batch processes

These do not need user interaction, and hence they often run in the background. Because such processes do not need to be very responsive, they are often penalized by the scheduler. Typical batch programs are programming language compilers, database search engines, and scientific computations.

Real-time processes

These have very stringent scheduling requirements. Such processes should never be blocked by lower-priority processes and should have a short guaranteed response time with a minimum variance. Typical real-time programs are video and sound applications, robot controllers, and programs that collect data from physical sensors.

The two classifications we just offered are somewhat independent. For instance, a batch process can be either I/O-bound (e.g., a database server) or CPU-bound (e.g., an image-rendering program). While real-time programs are explicitly recognized as such by the scheduling algorithm in Linux, there is no easy way to distinguish between interactive and batch programs. The Linux 2.6 scheduler implements a sophisticated heuristic algorithm based on the past behavior of the processes to decide whether a given process should be considered as interactive or batch. Of course, the scheduler tends to favor interactive processes over batch ones.

Programmers may change the scheduling priorities by means of the system calls illustrated in [Table 7-1](#). More details are given in the section "[System Calls Related to Scheduling](#)."

Table 7-1. System calls related to scheduling

System call	Description
<code>nice()</code>	Change the static priority of a conventional process
<code>getpriority()</code>	Get the maximum static priority of a group of conventional processes
<code>setpriority()</code>	Set the static priority of a group of conventional processes
<code>sched_getscheduler()</code>	Get the scheduling policy of a process
<code>sched_setscheduler()</code>	Set the scheduling policy and the real-time priority of a process
<code>sched_getparam()</code>	Get the real-time priority of a process
<code>sched_setparam()</code>	Set the real-time priority of a process
<code>sched_yield()</code>	Relinquish the processor voluntarily without blocking
<code>sched_get_ priority_min()</code>	Get the minimum real-time priority value for a policy
<code>sched_get_ priority_max()</code>	Get the maximum real-time priority value for a policy
<code>sched_rr_get_interval()</code>	Get the time quantum value for the Round Robin policy
<code>sched_setaffinity()</code>	Set the CPU affinity mask of a process
<code>sched_getaffinity()</code>	Get the CPU affinity mask of a process

Process Preemption

As mentioned in the first chapter, Linux processes are *preemptable*. When a process enters the `TASK_RUNNING` state, the kernel checks whether its dynamic priority is greater than the priority of the currently running process. If it is, the execution of `current` is interrupted and the scheduler is invoked to select another process to run (usually the process that just became runnable). Of course, a process also may be preempted when its time quantum expires. When this occurs, the `TIF_NEED_RESCHED` flag in the `thread_info` structure of the current process is set, so the scheduler is invoked when the timer interrupt handler terminates.

For instance, let's consider a scenario in which only two programs—a text editor and a compiler—are being executed. The text editor is an interactive program, so it has a

higher dynamic priority than the compiler. Nevertheless, it is often suspended, because the user alternates between pauses for think time and data entry; moreover, the average delay between two keypresses is relatively long. However, as soon as the user presses a key, an interrupt is raised and the kernel wakes up the text editor process. The kernel also determines that the dynamic priority of the editor is higher than the priority of `current`, the currently running process (the compiler), so it sets the `TIF_NEED_RESCHED` flag of this process, thus forcing the scheduler to be activated when the kernel finishes handling the interrupt. The scheduler selects the editor and performs a process switch; as a result, the execution of the editor is resumed very quickly and the character typed by the user is echoed to the screen. When the character has been processed, the text editor process suspends itself waiting for another keypress and the compiler process can resume its execution.

Be aware that a preempted process is not suspended, because it remains in the `TASK_RUNNING` state; it simply no longer uses the CPU. Moreover, remember that the Linux 2.6 kernel is preemptive, which means that a process can be preempted either when executing in Kernel or in User Mode; we discussed in depth this feature in the section "[Kernel Preemption](#)" in [Chapter 5](#).

How Long Must a Quantum Last?

The quantum duration is critical for system performance: it should be neither too long nor too short.

If the average quantum duration is too short, the system overhead caused by process switches becomes excessively high. For instance, suppose that a process switch requires 5 milliseconds; if the quantum is also set to 5 milliseconds, then at least 50 percent of the CPU cycles will be dedicated to process switching.^[*]

If the average quantum duration is too long, processes no longer appear to be executed concurrently. For instance, let's suppose that the quantum is set to five seconds; each runnable process makes progress for about five seconds, but then it stops for a very long time (typically, five seconds times the number of runnable processes).

It is often believed that a long quantum duration degrades the response time of interactive applications. This is usually false. As described in the section "[Process Preemption](#)" earlier in this chapter, interactive processes have a relatively high priority, so they quickly preempt the batch processes, no matter how long the quantum duration is.

In some cases, however, a very long quantum duration degrades the responsiveness of the system. For instance, suppose two users concurrently enter two commands at the respective shell prompts; one command starts a CPU-bound process, while the other launches an interactive application. Both shells fork a new process and delegate the execution of the user's command to it; moreover, suppose such new processes have the same initial priority (Linux does not know in advance if a program to be executed is batch or interactive). Now if the scheduler selects the CPU-bound process to run first, the other process could wait for a whole time quantum before starting its execution. Therefore, if the quantum duration is long, the system could appear to be unresponsive to the user that launched the interactive application.

The choice of the average quantum duration is always a compromise. The rule of thumb adopted by Linux is choose a duration as long as possible, while keeping good system response time.

[*] Recall that stopped and suspended processes cannot be selected by the scheduling algorithm to run on a CPU.

[*] Actually, things could be much worse than this; for example, if the time required for the process switch is counted in the process quantum, all CPU time is devoted to the process switch and no process can progress toward its termination.

The Scheduling Algorithm

The scheduling algorithm used in earlier versions of Linux was quite simple and straightforward: at every process switch the kernel scanned the list of runnable processes, computed their priorities, and selected the "best" process to run. The main drawback of that algorithm is that the time spent in choosing the best process depends on the number of runnable processes; therefore, the algorithm is too costly—that is, it spends too much time—in high-end systems running thousands of processes.

The scheduling algorithm of Linux 2.6 is much more sophisticated. By design, it scales well with the number of runnable processes, because it selects the process to run in constant time, independently of the number of runnable processes. It also scales well with the number of processors because each CPU has its own queue of runnable processes. Furthermore, the new algorithm does a better job of distinguishing interactive processes and batch processes. As a consequence, users of heavily loaded systems feel that interactive applications are much more responsive in Linux 2.6 than in earlier versions.

The scheduler always succeeds in finding a process to be executed; in fact, there is always at least one runnable process: the *swapper* process, which has PID 0 and executes only when the CPU cannot execute other processes. As mentioned in [Chapter 3](#), every CPU of a multiprocessor system has its own *swapper* process with PID equal to 0.

Every Linux process is always scheduled according to one of the following *scheduling classes* :

SCHED_FIFO

A First-In, First-Out real-time process. When the scheduler assigns the CPU to the process, it leaves the process descriptor in its current position in the runqueue list. If no other higher-priority real-time process is runnable, the process continues to use the CPU as long as it wishes, even if other real-time processes that have the same priority are runnable.

SCHED_RR

A Round Robin real-time process. When the scheduler assigns the CPU to the process, it puts the process descriptor at the end of the runqueue list. This policy ensures a fair assignment of CPU time to all SCHED_RR real-time processes that have the same priority.

SCHED_NORMAL

A conventional, time-shared process.

The scheduling algorithm behaves quite differently depending on whether the process is conventional or real-time.

Scheduling of Conventional Processes

Every conventional process has its own *static priority*, which is a value used by the scheduler to rate the process with respect to the other conventional processes in the system. The kernel represents the static priority of a conventional process with a number ranging from 100 (highest priority) to 139 (lowest priority); notice that static priority decreases as the values increase.

A new process always inherits the static priority of its parent. However, a user can change the static priority of the processes that he owns by passing some "nice values" to the `nice( )` and `setpriority( )` system calls (see the section "[System Calls Related to Scheduling](#)" later in this chapter).

Base time quantum

The static priority essentially determines the *base time quantum* of a process, that is, the time quantum duration assigned to the process when it has exhausted its previous time quantum. Static priority and base time quantum are related by the following formula:

$$\text{base time quantum (in milliseconds)} = \begin{cases} (140 - \text{static priority}) \times 20 & \text{if static priority} < 120 \\ (140 - \text{static priority}) \times 5 & \text{if static priority} \geq 120 \end{cases} \quad (1)$$

As you see, the higher the static priority (i.e., the lower its numerical value), the longer the base time quantum. As a consequence, higher priority processes usually get longer slices of CPU time with respect to lower priority processes. [Table 7-2](#) shows the static priority, the base time quantum values, and the corresponding nice values for a conventional process having highest static priority, default static priority, and lowest static priority. (The table also lists the values of the interactive delta and of the sleep time threshold, which are explained later in this chapter.)

Table 7-2. Typical priority values for a conventional process

Description	Static priority	Nice value	Base time quantum	Interactive delta	Sleep time threshold
Highest static priority	100	-20	800 ms	-3	299 ms
High static priority	110	-10	600 ms	-1	499 ms
Default static priority	120	0	100 ms	+2	799 ms
Low static priority	130	+10	50 ms	+4	999 ms
Lowest static priority	139	+19	5 ms	+6	1199 ms

Dynamic priority and average sleep time

Besides a static priority, a conventional process also has a *dynamic priority*, which is a value ranging from 100 (highest priority) to 139 (lowest priority). The dynamic priority is the number actually looked up by the scheduler when selecting the new process to run. It is related to the static priority by the following empirical formula:

$$\text{dynamic priority} = \max(100, \min(\text{static priority} - \text{bonus} + 5, 139)) \quad (2)$$

The *bonus* is a value ranging from 0 to 10; a value less than 5 represents a penalty that lowers the dynamic priority, while a value greater than 5 is a premium that raises the dynamic priority. The value of the bonus, in turn, depends on the past history of the process; more precisely, it is related to the *average sleep time* of the process.

Roughly, the average sleep time is the average number of nanoseconds that the process spent while sleeping. Be warned, however, that this is not an average operation on the elapsed time. For instance, sleeping in TASK_INTERRUPTIBLE state contributes to the average sleep time in a different way from sleeping in TASK_UNINTERRUPTIBLE state. Moreover, the average sleep time decreases while a process is running. Finally, the average sleep time can never become larger than 1 second.

The correspondence between average sleep times and bonus values is shown in [Table 7-3](#). (The table lists also the corresponding granularity of the time slice, which will be discussed later.)

Table 7-3. Average sleep times, bonus values, and time slice granularity

Average sleep time	Bonus	Granularity
Greater than or equal to 0 but smaller than 100 ms	0	5120
Greater than or equal to 100 ms but smaller than 200 ms	1	2560
Greater than or equal to 200 ms but smaller than 300 ms	2	1280
Greater than or equal to 300 ms but smaller than 400 ms	3	640
Greater than or equal to 400 ms but smaller than 500 ms	4	320
Greater than or equal to 500 ms but smaller than 600 ms	5	160
Greater than or equal to 600 ms but smaller than 700 ms	6	80
Greater than or equal to 700 ms but smaller than 800 ms	7	40
Greater than or equal to 800 ms but smaller than 900 ms	8	20
Greater than or equal to 900 ms but smaller than 1000 ms	9	10
1 second	10	10

The average sleep time is also used by the scheduler to determine whether a given process should be considered interactive or batch. More precisely, a process is considered "interactive" if it satisfies the following formula:

$$\text{dynamic priority} \leq 3 \times \text{static priority} / 4 + 28 \quad (3)$$

which is equivalent to the following:

$$\text{bonus} - 5 \geq \text{static priority} / 4 - 28$$

The expression $\text{static priority} / 4 - 28$ is called the *interactive delta* ; some typical values of this term are listed in [Table 7-2](#). It should be noted that it is far easier for high priority than for low priority processes to become interactive. For instance, a process having highest static priority (100) is considered interactive when its bonus value exceeds 2, that is, when its average sleep time exceeds 200 ms. Conversely, a process having lowest static priority (139) is never considered as interactive, because the bonus value is always smaller than the value 11 required to reach an interactive delta equal to 6. A process having default static priority (120) becomes interactive as soon as its average sleep time exceeds 700 ms.

Active and expired processes

Even if conventional processes having higher static priorities get larger slices of the CPU time, they should not completely lock out the processes having lower static priority. To avoid process starvation, when a process finishes its time quantum, it can be replaced by a lower priority process whose time quantum has not yet been exhausted. To implement this mechanism, the scheduler keeps two disjoint sets of runnable processes:

Active processes

These runnable processes have not yet exhausted their time quantum and are thus allowed to run.

Expired processes

These runnable processes have exhausted their time quantum and are thus forbidden to run until all active processes expire.

However, the general schema is slightly more complicated than this, because the scheduler tries to boost the performance of interactive processes. An active batch process that finishes its time quantum always becomes expired. An active interactive process that finishes its time quantum usually remains active: the scheduler refills its time quantum and leaves it in the set of active processes. However, the scheduler moves an interactive process that finished its time quantum into the set of expired processes if the eldest expired process has already waited for a long time, or if an expired process has higher static priority (lower value) than the interactive process. As a consequence, the set of active processes will eventually become empty and the expired processes will have a chance to run.

Scheduling of Real-Time Processes

Every real-time process is associated with a *real-time priority*, which is a value ranging from 1 (highest priority) to 99 (lowest priority). The scheduler always favors a higher priority runnable process over a lower priority one; in other words, a real-time process inhibits the execution of every lower-priority process while it remains runnable. Contrary to conventional processes, real-time processes are always considered active (see the previous section). The user can change the real-time priority of a process by means of the `sched_setparam( )` and `sched_setscheduler( )` system calls (see the section "[System Calls Related to Scheduling](#)" later in this chapter).

If several real-time runnable processes have the same highest priority, the scheduler chooses the process that occurs first in the corresponding list of the local CPU's runqueue (see the section "[The lists of TASK_RUNNING processes](#)" in [Chapter 3](#)).

A real-time process is replaced by another process only when one of the following events occurs:

- The process is preempted by another process having higher real-time priority.
- The process performs a blocking operation, and it is put to sleep (in state `TASK_INTERRUPTIBLE` or `TASK_UNINTERRUPTIBLE`).
- The process is stopped (in state `TASK_STOPPED` or `TASK_TRACED`), or it is killed (in state `EXIT_ZOMBIE` or `EXIT_DEAD`).
- The process voluntarily relinquishes the CPU by invoking the `sched_yield( )` system call (see the section "[System Calls Related to Scheduling](#)" later in this chapter).
- The process is Round Robin real-time (`SCHED_RR`), and it has exhausted its time quantum.

The `nice( )` and `setpriority( )` system calls, when applied to a Round Robin real-time process, do not change the real-time priority but rather the duration of the base time quantum. In fact, the duration of the base time quantum of Round Robin real-time processes does not depend on the real-time priority, but rather on the static priority of the process, according to the formula (1) in the earlier section "[Scheduling of Conventional Processes](#)."

Data Structures Used by the Scheduler

Recall from the section "[Identifying a Process](#)" in [Chapter 3](#) that the process list links all process descriptors, while the runqueue lists link the process descriptors of all runnable processes—that is, of those in a TASK_RUNNING state—except the *swapper* process (idle process).

The runqueue Data Structure

The runqueue data structure is the most important data structure of the Linux 2.6 scheduler. Each CPU in the system has its own runqueue; all `runqueue` structures are stored in the `runqueues` per-CPU variable (see the section "[Per-CPU Variables](#)" in [Chapter 5](#)). The `this_rq( )` macro yields the address of the runqueue of the local CPU, while the `cpu_rq(n)` macro yields the address of the runqueue of the CPU having index `n`.

[Table 7-4](#) lists the fields included in the `runqueue` data structure; we will discuss most of them in the following sections of the chapter.

Table 7-4. The fields of the runqueue structure

Type	Name	Description
spinlock_t	lock	Spin lock protecting the lists of processes
unsigned long	nr_running	Number of runnable processes in the runqueue lists
unsigned long	cpu_load	CPU load factor based on the average number of processes in the runqueue
unsigned long	nr_switches	Number of process switches performed by the CPU
unsigned long	nr_uninterruptible	Number of processes that were previously in the runqueue lists and are now sleeping in TASK_UNINTERRUPTIBLE state (only the sum of these fields across all runqueues is meaningful)
unsigned long	expired_timestamp	Insertion time of the eldest process in the expired lists
unsigned long long	timestamp_last_tick	Timestamp value of the last timer interrupt
task_t *	curr	Process descriptor pointer of the currently running process (same as <code>current</code> for the local CPU)
task_t *	idle	Process descriptor pointer of the <i>swapper</i> process for this CPU
struct mm_struct *	prev_mm	Used during a process switch to store the address of the memory descriptor of the process being replaced
prio_array_t *	active	Pointer to the lists of active processes
prio_array_t *	expired	Pointer to the lists of expired processes
prio_array_t [2]	arrays	The two sets of active and expired processes
int	best_expired_prio	The best static priority (lowest value) among the expired processes
atomic_t	nr_iowait	Number of processes that were previously in the runqueue lists and are now waiting for a disk I/O operation to complete
struct sched_domain *	sd	Points to the base scheduling domain of this CPU (see the section " Scheduling Domains " later in this chapter)

Type	Name	Description
int	active_balance	Flag set if some process shall be <i>migrated</i> from this runqueue to another (runqueue balancing)
int	push_cpu	Not used
task_t *	migration_thread	Process descriptor pointer of the <i>migration</i> kernel thread
struct list_head	migration_queue	List of processes to be removed from the runqueue

The most important fields of the `runqueue` data structure are those related to the lists of runnable processes. Every runnable process in the system belongs to one, and just one, runqueue. As long as a runnable process remains in the same runqueue, it can be executed only by the CPU owning that runqueue. However, as we'll see later, runnable processes may migrate from one runqueue to another.

The `arrays` field of the runqueue is an array consisting of two `prio_array_t` structures. Each data structure represents a set of runnable processes, and includes 140 doubly linked list heads (one list for each possible process priority), a priority bitmap, and a counter of the processes included in the set (see [Table 3-2](#) in the section [Chapter 3](#)).

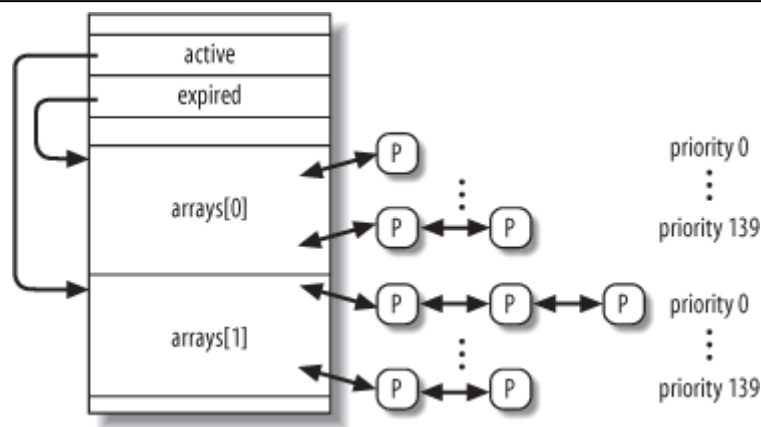


Figure 7-1. The runqueue structure and the two sets of runnable processes

As shown in [Figure 7-1](#), the `active` field of the runqueue structure points to one of the two `prio_array_t` data structures in `arrays`: the corresponding set of runnable processes includes the active processes. Conversely, the `expired` field points to the other `prio_array_t` data structure in `arrays`: the corresponding set of runnable processes includes the expired processes.

Periodically, the role of the two data structures in `arrays` changes: the active processes suddenly become the expired processes, and the expired processes become the active ones. To achieve this change, the scheduler simply exchanges the contents of the `active` and `expired` fields of the `runqueue`.

Process Descriptor

Each process descriptor includes several fields related to scheduling; they are listed in [Table 7-5](#).

Table 7-5. Fields of the process descriptor related to the scheduler

Type	Name	Description
unsigned long	thread_info->flags	Stores the TIF_NEED_RESCHED flag, which is set if the scheduler must be invoked (see the section " Returning from Interrupts and Exceptions " in Chapter 4)
unsigned int	thread_info->cpu	Logical number of the CPU owning the runqueue to which the runnable process belongs
unsigned long	state	The current state of the process (see the section " Process State " in Chapter 3)
int	prio	Dynamic priority of the process
int	static_prio	Static priority of the process
struct list_head	run_list	Pointers to the next and previous elements in the runqueue list to which the process belongs
prio_array_t *	array	Pointer to the runqueue's prio_array_t set that includes the process
unsigned long	sleep_avg	Average sleep time of the process
unsigned long long	timestamp	Time of last insertion of the process in the runqueue, or time of last process switch involving the process
unsigned long long	last_ran	Time of last process switch that replaced the process
int	activated	Condition code used when the process is awakened
unsigned long	policy	The scheduling class of the process (SCHED_NORMAL, SCHED_RR, or SCHED_FIFO)
cpumask_t	cpus_allowed	Bit mask of the CPUs that can execute the process
unsigned int	time_slice	Ticks left in the time quantum of the process
unsigned int	first_time_slice	Flag set to 1 if the process never exhausted its time quantum
unsigned long	rt_priority	Real-time priority of the process

When a new process is created, `sched_fork( )`, invoked by `copy_process( )`, sets the `time_slice` field of both current (the parent) and `p` (the child) processes in the

following way:

```
p->time_slice = (current->time_slice + 1) >> 1;
current->time_slice >= 1;
```

In other words, the number of ticks left to the parent is split in two halves: one for the parent and one for the child. This is done to prevent users from getting an unlimited amount of CPU time by using the following method: the parent process creates a child process that runs the same code and then kills itself; by properly adjusting the creation rate, the child process would always get a fresh quantum before the quantum of its parent expires. This programming trick does not work because the kernel does not reward forks. Similarly, a user cannot hog an unfair share of the processor by starting several background processes in a shell or by opening a lot of windows on a graphical desktop. More generally speaking, a process cannot hog resources (unless it has privileges to give itself a real-time policy) by forking multiple descendents.

If the parent had just one tick left in its time slice, the splitting operation forces `current->time_slice` to 0, thus exhausting the quantum of the parent. In this case, `copy_process( )` sets `current->time_slice` back to 1, then invokes `scheduler_tick( )` to decrease the field (see the following section).

The `copy_process( )` function also initializes a few other fields of the child's process descriptor related to scheduling:

```
p->first_time_slice = 1;
p->timestamp = sched_clock( );
```

The `first_time_slice` flag is set to 1, because the child has never exhausted its time quantum (if a process terminates or executes a new program during its first time slice, the parent process is rewarded with the remaining time slice of the child). The `timestamp` field is initialized with a timestamp value produced by `sched_clock( )`: essentially, this function returns the contents of the 64-bit TSC register (see the section "[Time Stamp Counter \(TSC\)](#)" in [Chapter 6](#)) converted to nanoseconds.

Functions Used by the Scheduler

The scheduler relies on several functions in order to do its work; the most important are:

`scheduler_tick( )`

Keeps the `time_slice` counter of current up-to-date

`try_to_wake_up( )`

Awakens a sleeping process

`recalc_task_prio( )`

Updates the dynamic priority of a process

`schedule( )`

Selects a new process to be executed

`load_balance( )`

Keeps the runqueues of a multiprocessor system balanced

The `scheduler_tick( )` Function

We have already explained in the section "[Updating Local CPU Statistics](#)" in [Chapter 6](#) how `scheduler_tick( )` is invoked once every tick to perform some operations related to scheduling. It executes the following main steps:

1. Stores in the `timestamp_last_tick` field of the local runqueue the current value of the TSC converted to nanoseconds; this timestamp is obtained from the `sched_clock( )` function (see the previous section).
2. Checks whether the current process is the *swapper* process of the local CPU. If so, it performs the following substeps:
 - a. If the local runqueue includes another runnable process besides *swapper*, it sets the `TIF_NEED_RESCHED` flag of the current process to force rescheduling. As we'll see in the section "[The `schedule\( \)` Function](#)" later in this chapter, if the kernel supports the hyper-threading technology (see the section "[Runqueue Balancing in Multiprocessor Systems](#)" later in this chapter), a logical CPU might be idle even if there are runnable processes in its runqueue, as long as those processes have significantly lower

priorities than the priority of a process already executing on another logical CPU associated with the same physical CPU.

b. Jumps to step 7 (there is no need to update the time slice counter of the *swapper* process).

3. Checks whether `current->array` points to the active list of the local runqueue. If not, the process has expired its time quantum, but it has not yet been replaced: sets the `TIF_NEED_RESCHED` flag to force rescheduling, and jumps to step 7.
4. Acquires the `this_rq()->lock` spin lock.
5. Decreases the time slice counter of the current process, and checks whether the quantum is exhausted. The operations performed by the function are quite different according to the scheduling class of the process; we will discuss them in a moment.
6. Releases the `this_rq( )->lock` spin lock.
7. Invokes the `rebalance_tick( )` function, which should ensure that the runqueues of the various CPUs contain approximately the same number of runnable processes. We will discuss runqueue balancing in the later section "[Runqueue Balancing in Multiprocessor Systems](#)."

Updating the time slice of a real-time process

If the current process is a FIFO real-time process, `scheduler_tick( )` has nothing to do. In this case, in fact, `current` cannot be preempted by lower or equal priority processes, thus it does not make sense to keep its time slice counter up-to-date.

If `current` is a Round Robin real-time process, `scheduler_tick( )` decreases its time slice counter and checks whether the quantum is exhausted:

```
if (current->policy == SCHED_RR && !--current->time_slice) {
    current->time_slice = task_timeslice(current);
    current->first_time_slice = 0;
    set_tsk_need_resched(current);
    list_del(&current->run_list);
    list_add_tail(&current->run_list,
                  this_rq( )->active->queue+current->prio);
}
```

If the function determines that the time quantum is effectively exhausted, it performs a few operations aimed to ensure that `current` will be preempted, if necessary, as soon as possible.

The first operation consists of refilling the time slice counter of the process by invoking `task_timeslice( )`. This function considers the static priority of the process and returns the corresponding base time quantum, according to the formula (1) shown in the earlier section "[Scheduling of Conventional Processes](#)." Moreover, the `first_time_slice` field of `current` is cleared: this flag is set by `copy_process( )` in the service routine of the `fork( )` system call, and should be cleared as soon as the first time quantum of the process elapses.

Next, `scheduler_tick( )` invokes the `set_tsk_need_resched( )` function to set the `TIF_NEED_RESCHED` flag of the process. As described in the section "[Returning from Interrupts and Exceptions](#)" in [Chapter 4](#), this flag forces the invocation of the `schedule( )` function, so that `current` can be replaced by another real-time process having equal (or higher) priority, if any.

The last operation of `scheduler_tick( )` consists of moving the process descriptor to the last position of the runqueue active list corresponding to the priority of `current`. Placing `current` in the last position ensures that it will not be selected again for execution until every real-time runnable process having the same priority as `current` will get a slice of the CPU time. This is the meaning of round-robin scheduling. The descriptor is moved by first invoking `list_del( )` to remove the process from the runqueue active list, then by invoking `list_add_tail( )` to insert back the process in the last position of the same list.

Updating the time slice of a conventional process

If the current process is a conventional process, the `scheduler_tick( )` function performs the following operations:

1. Decreases the time slice counter (`current->time_slice`).
2. Checks the time slice counter. If the time quantum is exhausted, the function performs the following operations:
 - a. Invokes `dequeue_task( )` to remove `current` from the `this_rq( )->active` set of runnable processes.
 - b. Invokes `set_tsk_need_resched( )` to set the `TIF_NEED_RESCHED` flag.
 - c. Updates the dynamic priority of `current`:

```
current->prio = effective_prio(current);
```

The `effective_prio( )` function reads the `static_prio` and `sleep_avg` fields of `current`, and computes the dynamic priority of the process

according to the formula (2) shown in the earlier section "[Scheduling of Conventional Processes](#)."

d. Refills the time quantum of the process:

```
current->time_slice = task_timeslice(current);
current->first_time_slice = 0;
```

e. If the `expired_timestamp` field of the local runqueue data structure is equal to zero (that is, the set of expired processes is empty), writes into the field the value of the current tick:

```
if (!this_rq( )->expired_timestamp)
    this_rq( )->expired_timestamp = jiffies;
```

f. Inserts the current process either in the active set or in the expired set:

```
if (!TASK_INTERACTIVE(current) || EXPIRED_STARVING(this_rq( )) {
    enqueue_task(current, this_rq( )->expired);
    if (current->static_prio < this_rq( )->best_expired_prio)
        this_rq( )->best_expired_prio = current->static_prio;
} else
    enqueue_task(current, this_rq( )->active);
```

The `TASK_INTERACTIVE` macro yields the value one if the process is recognized as interactive using the formula (3) shown in the earlier section "[Scheduling of Conventional Processes](#)." The `EXPIRED_STARVING` macro checks whether the first expired process in the runqueue had to wait for more than 1000 ticks times the number of runnable processes in the runqueue plus one; if so, the macro yields the value one. The `EXPIRED_STARVING` macro also yields the value one if the static priority value of the current process is greater than the static priority value of an already expired process.

3. Otherwise, if the time quantum is not exhausted (`current->time_slice` is not zero), checks whether the remaining time slice of the current process is too long:

```
if (TASK_INTERACTIVE(p) && !((task_timeslice(p) -
    p->time_slice) % TIMESLICE_GRANULARITY(p)) &&
    (p->time_slice >= TIMESLICE_GRANULARITY(p)) &&
    (p->array == rq->active)) {
    list_del(&current->run_list);
    list_add_tail(&current->run_list,
        this_rq( )->active->queue+current->prio);
    set_tsk_need_resched(p);
}
```

The `TIMESLICE_GRANULARITY` macro yields the product of the number of CPUs in the system and a constant proportional to the bonus of the current process

(see [Table 7-3](#) earlier in the chapter). Basically, the time quantum of interactive processes with high static priorities is split into several pieces of `TIMESLICE_GRANULARITY` size, so that they do not monopolize the CPU.

The `try_to_wake_up( )` Function

The `try_to_wake_up( )` function awakes a sleeping or stopped process by setting its state to `TASK_RUNNING` and inserting it into the runqueue of the local CPU. For instance, the function is invoked to wake up processes included in a wait queue (see the section "[How Processes Are Organized](#)" in [Chapter 3](#)) or to resume execution of processes waiting for a signal (see [Chapter 11](#)). The function receives as its parameters:

- The descriptor pointer (`p`) of the process to be awakened
- A mask of the process states (`state`) that can be awakened
- A flag (`sync`) that forbids the awakened process to preempt the process currently running on the local CPU

The function performs the following operations:

1. Invokes the `task_rq_lock( )` function to disable local interrupts and to acquire the lock of the runqueue `rq` owned by the CPU that was last executing the process (it could be different from the local CPU). The logical number of that CPU is stored in the `p->thread_info->cpu` field.
2. Checks if the state of the process `p->state` belongs to the mask of states `state` passed as argument to the function; if this is not the case, it jumps to step 9 to terminate the function.
3. If the `p->array` field is not `NULL`, the process already belongs to a runqueue; therefore, it jumps to step 8.
4. In multiprocessor systems, it checks whether the process to be awakened should be migrated from the runqueue of the lastly executing CPU to the runqueue of another CPU. Essentially, the function selects a target runqueue according to some heuristic rules. For example:
 - If some CPU in the system is idle, it selects its runqueue as the target. Preference is given to the previously executing CPU and to the local CPU, in this order.

- If the workload of the previously executing CPU is significantly lower than the workload of the local CPU, it selects the old runqueue as the target.
- If the process has been executed recently, it selects the old runqueue as the target (the hardware cache might still be filled with the data of the process).
- If moving the process to the local CPU reduces the unbalance between the CPUs, the target is the local runqueue (see the section "[Runqueue Balancing in Multiprocessor Systems](#)" later in this chapter).

After this step has been executed, the function has identified a target CPU that will execute the awakened process and, correspondingly, a target runqueue `rq` in which to insert the process.

1. If the process is in the `TASK_UNINTERRUPTIBLE` state, it decreases the `nr_uninterruptible` field of the target runqueue, and sets the `p->activated` field of the process descriptor to -1. See the later section "[The `recalc\_task\_prio\(\)` Function](#)" for an explanation of the `activated` field.

2. Invokes the `activate_task( )` function, which in turn performs the following substeps:

a. Invokes `sched_clock( )` to get the current timestamp in nanoseconds. If the target CPU is not the local CPU, it compensates for the drift of the local timer interrupts by using the timestamps relative to the last occurrences of the timer interrupts on the local and target CPUs:

```
now = (sched_clock( ) - this_rq( )->timestamp_last_tick)
      + rq->timestamp_last_tick;
```

b. Invokes `recalc_task_prio( )`, passing to it the process descriptor pointer and the timestamp computed in the previous step. The `recalc_task_prio( )` function is described in the next section.

c. Sets the value of the `p->activated` field according to [Table 7-6](#) later in this chapter.

d. Sets the `p->timestamp` field with the timestamp computed in step 6a.

e. Inserts the process descriptor in the active set:

```
enqueue_task(p, rq->active);
rq->nr_running++;
```

3. If either the target CPU is not the local CPU or if the `sync` flag is not set, it checks whether the new runnable process has a dynamic priority higher than that

of the current process of the `rq` runqueue (`p->prio < rq->curr->prio`); if so, invokes `resched_task( )` to preempt `rq->curr`. In uniprocessor systems the latter function just executes `set_tsk_need_resched( )` to set the `TIF_NEED_RESCHED` flag of the `rq->curr` process. In multiprocessor systems `resched_task( )` also checks whether the old value of whether `TIF_NEED_RESCHED` flag was zero, the target CPU is different from the local CPU, and whether the `TIF_POLLING_NRFLAG` flag of the `rq->curr` process is clear (the target CPU is not actively polling the status of the `TIF_NEED_RESCHED` flag of the process). If so, `resched_task( )` invokes `smp_send_reschedule( )` to raise an IPI and force rescheduling on the target CPU (see the section "[Interprocessor Interrupt Handling](#)" in [Chapter 4](#)).

4. Sets the `p->state` field of the process to `TASK_RUNNING`.
5. Invokes `task_rq_unlock( )` to unlock the `rq` runqueue and reenables the local interrupts.
6. Returns 1 (if the process has been successfully awakened) or 0 (if the process has not been awakened).

The `recalc_task_prio( )` Function

The `recalc_task_prio( )` function updates the average sleep time and the dynamic priority of a process. It receives as its parameters a process descriptor pointer `p` and a timestamp `now` computed by the `sched_clock( )` function.

The function executes the following operations:

1. Stores in the `sleep_time` local variable the result of:

$$\min(\text{now} - \text{p->timestamp}, 10^9)$$

The `p->timestamp` field contains the timestamp of the process switch that put the process to sleep; therefore, `sleep_time` stores the number of nanoseconds that the process spent sleeping since its last execution (or the equivalent of 1 second, if the process slept more).

2. If `sleep_time` is not greater than zero, it jumps to step 8 so as to skip updating the average sleep time of the process.
3. Checks whether the process is not a kernel thread, whether it is awakening from the `TASK_UNINTERRUPTIBLE` state (`p->activated` field equal to `-1`; see step 5 in the previous section), and whether it has been continuously asleep

beyond a given sleep time threshold. If these three conditions are fulfilled, the function sets the `p->sleep_avg` field to the equivalent of 900 ticks (an empirical value obtained by subtracting the duration of the base time quantum of a standard process from the maximum average sleep time). Then, it jumps to step 8.

The sleep time threshold depends on the static priority of the process; some typical values are shown in [Table 7-2](#). In short, the goal of this empirical rule is to ensure that processes that have been asleep for a long time in uninterruptible mode—usually waiting for disk I/O operations—get a predefined sleep average value that is large enough to allow them to be quickly serviced, but it is also not so large to cause starvation for other processes.

4. Executes the `CURRENT_BONUS` macro to compute the *bonus* value of the previous average sleep time of the process (see [Table 7-3](#)). If $(10 - \textit{bonus})$ is greater than zero, the function multiplies `sleep_time` by this value. Since `sleep_time` will be added to the average sleep time of the process (see step 6 below), the lower the current average sleep time is, the more rapidly it will rise.
5. If the process is in `TASK_UNINTERRUPTIBLE` mode and it is not a kernel thread, it performs the following substeps:
 - a. Checks whether the average sleep time `p->sleep_avg` is greater than or equal to its sleep time threshold (see [Table 7-2](#) earlier in this chapter). If so, it resets the `sleep_avg` local variable to zero—thus skipping the adjustment of the average sleep time—and jumps to step 6.
 - b. If the sum `sleep_avg + p->sleep_avg` is greater than or equal to the sleep time threshold, it sets the `p->sleep_avg` field to the sleep time threshold, and sets `sleep_avg` to zero.

By somewhat limiting the increment of the average sleep time of the process, the function does not reward too much batch processes that sleep for a long time.

6. Adds `sleep_time` to the average sleep time of the process (`p->sleep_avg`).
7. Checks whether `p->sleep_avg` exceeds 1000 ticks (in nanoseconds); if so, the function cuts it down to 1000 ticks (in nanoseconds).
8. Updates the dynamic priority of the process:

```
p->prio = effective_prio(p);
```

The `effective_prio( )` function has already been discussed in the section "[The scheduler tick\(.\) Function](#)" earlier in this chapter.

The `schedule( )` Function

The `schedule( )` function implements the scheduler. Its objective is to find a process in the runqueue list and then assign the CPU to it. It is invoked, directly or in a lazy (deferred) way, by several kernel routines.

Direct invocation

The scheduler is invoked directly when the `current` process must be blocked right away because the resource it needs is not available. In this case, the kernel routine that wants to block it proceeds as follows:

1. Inserts `current` in the proper wait queue.
2. Changes the state of `current` either to `TASK_INTERRUPTIBLE` or to `TASK_UNINTERRUPTIBLE`.
3. Invokes `schedule( )`.
4. Checks whether the resource is available; if not, goes to step 2.
5. Once the resource is available, removes `current` from the wait queue.

The kernel routine checks repeatedly whether the resource needed by the process is available; if not, it yields the CPU to some other process by invoking `schedule( )`. Later, when the scheduler once again grants the CPU to the process, the availability of the resource is rechecked. These steps are similar to those performed by `wait_event( )` and similar functions described in the section "[How Processes Are Organized](#)" in [Chapter 3](#).

The scheduler is also directly invoked by many device drivers that execute long iterative tasks. At each iteration cycle, the driver checks the value of the `TIF_NEED_RESCHED` flag and, if necessary, invokes `schedule( )` to voluntarily relinquish the CPU.

Lazy invocation

The scheduler can also be invoked in a lazy way by setting the `TIF_NEED_RESCHED` flag of `current` to 1. Because a check on the value of this flag is always made before resuming the execution of a User Mode process (see the section "[Returning from](#)

[Interrupts and Exceptions](#)" in [Chapter 4](#)), `schedule( )` will definitely be invoked at some time in the near future.

Typical examples of lazy invocation of the scheduler are:

- When `current` has used up its quantum of CPU time; this is done by the `scheduler_tick( )` function.
- When a process is woken up and its priority is higher than that of the current process; this task is performed by the `try_to_wake_up( )` function.
- When a `sched_setscheduler( )` system call is issued (see the section "[System Calls Related to Scheduling](#)" later in this chapter).

Actions performed by `schedule( )` before a process switch

The goal of the `schedule( )` function consists of replacing the currently executing process with another one. Thus, the key outcome of the function is to set a local variable called `next`, so that it points to the descriptor of the process selected to replace `current`. If no runnable process in the system has priority greater than the priority of `current`, at the end, `next` coincides with `current` and no process switch takes place.

The `schedule( )` function starts by disabling kernel preemption and initializing a few local variables:

```
need_resched;  
  
preempt_disable( );  
prev = current;  
rq = this_rq( );
```

As you see, the pointer returned by `current` is saved in `prev`, and the address of the runqueue data structure corresponding to the local CPU is saved in `rq`.

Next, `schedule( )` makes sure that `prev` doesn't hold the big kernel lock (see the section "[The Big Kernel Lock](#)" in [Chapter 5](#)):

```
if (prev->lock_depth >= 0)  
    up(&kernel_sem);
```

Notice that `schedule( )` doesn't change the value of the `lock_depth` field; when `prev` resumes its execution, it reacquires the `kernel_flag` spin lock if the value of this field is not negative. Thus, the big kernel lock is automatically released and reacquired across a process switch.

The `sched_clock( )` function is invoked to read the TSC and convert its value to nanoseconds; the timestamp obtained is saved in the `now` local variable. Then, `schedule( )` computes the duration of the CPU time slice used by `prev`:

```
now = sched_clock( );
run_time = now - prev->timestamp;
if (run_time > 1000000000)
    run_time = 1000000000;
```

The usual cut-off at 1 second (converted to nanoseconds) applies. The `run_time` value is used to charge the process for the CPU usage. However, a process having a high average sleep time is favored:

```
run_time /= (CURRENT_BONUS(prev) ? : 1);
```

Remember that `CURRENT_BONUS` returns a value between 0 and 10 that is proportional to the average sleep time of the process.

Before starting to look at the runnable processes, `schedule( )` must disable the local interrupts and acquire the spin lock that protects the runqueue:

```
spin_lock_irq(&rq->lock);
```

As explained in the section "[Process Termination](#)" in [Chapter 3](#), `prev` might be a process that is being terminated. To recognize this case, `schedule( )` looks at the `PF_DEAD` flag:

```
if (prev->flags & PF_DEAD)
    prev->state = EXIT_DEAD;
```

Next, `schedule( )` examines the state of `prev`. If it is not runnable and it has not been preempted in Kernel Mode (see the section "[Returning from Interrupts and Exceptions](#)" in [Chapter 4](#)), then it should be removed from the runqueue. However, if it has nonblocked pending signals and its state is `TASK_INTERRUPTIBLE`, the function sets the process state to `TASK_RUNNING` and leaves it into the runqueue. This action is not the same as assigning the processor to `prev`; it just gives `prev` a chance to be selected for execution:

```
if (prev->state != TASK_RUNNING &&
    !(preempt_count() & PREEMPT_ACTIVE)) {
    if (prev->state == TASK_INTERRUPTIBLE && signal_pending(prev))
        prev->state = TASK_RUNNING;
    else {
        if (prev->state == TASK_UNINTERRUPTIBLE)
            rq->nr_uninterruptible++;
        deactivate_task(prev, rq);
    }
}
```

The `deactivate_task( )` function removes the process from the runqueue:

```
rq->nr_running--;  
dequeue_task(p, p->array);  
p->array = NULL;
```

Now, `schedule( )` checks the number of runnable processes left in the runqueue. If there are some runnable processes, the function invokes the `dependent_sleeper( )` function. In most cases, this function immediately returns zero. If, however, the kernel supports the hyper-threading technology (see the section "[Runqueue Balancing in Multiprocessor Systems](#)" later in this chapter), the function checks whether the process that is going to be selected for execution has significantly lower priority than a sibling process already running on a logical CPU of the same physical CPU; in this particular case, `schedule( )` refuses to select the lower privilege process and executes the *swapper* process instead.

```
if (rq->nr_running) {  
    if (dependent_sleeper(smp_processor_id( ), rq)) {  
        next = rq->idle;  
        goto switch_tasks;  
    }  
}
```

If no runnable process exists, the function invokes `idle_balance( )` to move some runnable process from another runqueue to the local runqueue; `idle_balance( )` is similar to `load_balance( )`, which is described in the later section "[The load_balance\(.\) Function](#)."

```
if (!rq->nr_running) {  
    idle_balance(smp_processor_id( ), rq);  
    if (!rq->nr_running) {  
        next = rq->idle;  
        rq->expired_timestamp = 0;  
        wake_sleeping_dependent(smp_processor_id( ), rq);  
        if (!rq->nr_running)  
            goto switch_tasks;  
    }  
}
```

If `idle_balance( )` fails in moving some process in the local runqueue, `schedule( )` invokes `wake_sleeping_dependent( )` to reschedule runnable processes in *idle CPUs* (that is, in every CPU that runs the *swapper* process). As explained earlier when discussing the `dependent_sleeper( )` function, this unusual case might happen when the kernel supports the hyper-threading technology. However, in uniprocessor systems, or when all attempts to move a runnable process in the local runqueue have failed, the function picks the *swapper* process as `next` and continues with the next phase.

Let's suppose that the `schedule( )` function has determined that the runqueue includes some runnable processes; now it has to check that at least one of these runnable processes is active. If not, the function exchanges the contents of the `active` and `expired` fields of the runqueue data structure; thus, all expired processes become active, while the empty set is ready to receive the processes that will expire in the future.

```
array = rq->active;
if (!array->nr_active) {
    rq->active = rq->expired;
    rq->expired = array;
    array = rq->active;
    rq->expired_timestamp = 0;
    rq->best_expired_prio = 140;
}
```

It is time to look up a runnable process in the active `prio_array_t` data structure (see the section "[Identifying a Process](#)" in [Chapter 3](#)). First of all, `schedule( )` searches for the first nonzero bit in the bitmask of the active set. Remember that a bit in the bitmask is set when the corresponding priority list is not empty. Thus, the index of the first nonzero bit indicates the list containing the best process to run. Then, the first process descriptor in that list is retrieved:

```
idx = sched_find_first_bit(array->bitmap);
next = list_entry(array->queue[idx].next, task_t, run_list);
```

The `sched_find_first_bit( )` function is based on the `bsfl` assembly language instruction, which returns the bit index of the least significant bit set to one in a 32-bit word.

The `next` local variable now stores the descriptor pointer of the process that will replace `prev`. The `schedule( )` function looks at the `next->activated` field. This field encodes the state of the process when it was awakened, as illustrated in [Table 7-6](#).

Table 7-6. The meaning of the activated field in the process descriptor

Value	Description
0	The process was in TASK_RUNNING state.
1	The process was in TASK_INTERRUPTIBLE or TASK_STOPPED state, and it is being awakened by a system call service routine or a kernel thread.
2	The process was in TASK_INTERRUPTIBLE or TASK_STOPPED state, and it is being awakened by an interrupt handler or a deferrable function.
-1	The process was in TASK_UNINTERRUPTIBLE state and it is being awakened.

If `next` is a conventional process and it is being awakened from the TASK_INTERRUPTIBLE or TASK_STOPPED state, the scheduler adds to the average sleep time of the process the nanoseconds elapsed since the process was inserted into the runqueue. In other words, the sleep time of the process is increased to cover also the time spent by the process in the runqueue waiting for the CPU:

```
if (next->prio >= 100 && next->activated > 0) {
    unsigned long long delta = now - next->timestamp;
    if (next->activated == 1)
        delta = (delta * 38) / 128;
    array = next->array;
    dequeue_task(next, array);
    recalc_task_prio(next, next->timestamp + delta);
    enqueue_task(next, array);
}
next->activated = 0;
```

Observe that the scheduler makes a distinction between a process awakened by an interrupt handler or deferrable function, and a process awakened by a system call service routine or a kernel thread. In the former case, the scheduler adds the whole runqueue waiting time, while in the latter it adds just a fraction of that time. This is because interactive processes are more likely to be awakened by asynchronous events (think of the user pressing keys on the keyboard) rather than by synchronous ones.

Actions performed by `schedule( )` to make the process switch

Now the `schedule( )` function has determined the next process to run. In a moment, the kernel will access the `thread_info` data structure of `next`, whose address is stored close to the top of `next`'s process descriptor:

```
switch_tasks:

prefetch(next);
```

The `prefetch` macro is a hint to the CPU control unit to bring the contents of the first fields of `next`'s process descriptor in the hardware cache. It is here just to improve the performance of `schedule( )`, because the data are moved in parallel to the execution of the following instructions, which do not affect `next`.

Before replacing `prev`, the scheduler should do some administrative work:

```
clear_tsk_need_resched(prev);
rcu_qsctr_inc(prev->thread_info->cpu);
```

The `clear_tsk_need_resched( )` function clears the `TIF_NEED_RESCHED` flag of `prev`, just in case `schedule( )` has been invoked in the lazy way. Then, the function records that the CPU is going through a quiescent state (see the section "[Read-Copy Update \(RCU\)](#)" in [Chapter 5](#)).

The `schedule( )` function must also decrease the average sleep time of `prev`, charging to it the slice of CPU time used by the process:

```
prev->sleep_avg -= run_time;
if ((long)prev->sleep_avg <= 0)
    prev->sleep_avg = 0;
prev->timestamp = prev->last_ran = now;
```

The timestamps of the process are then updated.

It is quite possible that `prev` and `next` are the same process: this happens if no other higher or equal priority active process is present in the runqueue. In this case, the function skips the process switch:

```
if (prev == next) {
    spin_unlock_irq(&rq->lock);
    goto finish_schedule;
}
```

At this point, `prev` and `next` are different processes, and the process switch is for real:

```
next->timestamp = now;
rq->nr_switches++;
rq->curr = next;
prev = context_switch(rq, prev, next);
```

The `context_switch( )` function sets up the address space of `next`. As we'll see in "[Memory Descriptor of Kernel Threads](#)" in [Chapter 9](#), the `active_mm` field of the process descriptor points to the memory descriptor that is used by the process, while the `mm` field points to the memory descriptor owned by the process. For normal processes, the two fields hold the same address; however, a kernel thread does not have its own address space and its `mm` field is always set to `NULL`. The `context_switch( )` function ensures that if `next` is a kernel thread, it uses the address space used by `prev`:

```

if (!next->mm) {
    next->active_mm = prev->active_mm;
    atomic_inc(&prev->active_mm->mm_count);
    enter_lazy_tlb(prev->active_mm, next);
}

```

Up to Linux version 2.2, kernel threads had their own address space. That design choice was suboptimal, because the Page Tables had to be changed whenever the scheduler selected a new process, even if it was a kernel thread. Because kernel threads run in Kernel Mode, they use only the fourth gigabyte of the linear address space, whose mapping is the same for all processes in the system. Even worse, writing into the `cr3` register invalidates all TLB entries (see "[Translation Lookaside Buffers \(TLB\)](#)" in [Chapter 2](#)), which leads to a significant performance penalty. Linux is nowadays much more efficient because Page Tables aren't touched at all if `next` is a kernel thread. As further optimization, if `next` is a kernel thread, the `schedule( )` function sets the process into lazy TLB mode (see the section "[Translation Lookaside Buffers \(TLB\)](#)" in [Chapter 2](#)).

Conversely, if `next` is a regular process, the `context_switch( )` function replaces the address space of `prev` with the one of `next`:

```

if (next->mm)
    switch_mm(prev->active_mm, next->mm, next);

```

If `prev` is a kernel thread or an exiting process, the `context_switch( )` function saves the pointer to the memory descriptor used by `prev` in the runqueue's `prev_mm` field, then resets `prev->active_mm`:

```

if (!prev->mm) {
    rq->prev_mm = prev->active_mm;
    prev->active_mm = NULL;
}

```

Now `context_switch( )` can finally invoke `switch_to( )` to perform the process switch between `prev` and `next` (see the section "[Performing the Process Switch](#)" in [Chapter 3](#)):

```

switch_to(prev, next, prev);
return prev;

```

Actions performed by `schedule( )` after a process switch

The instructions of the `context_switch( )` and `schedule( )` functions following the `switch_to` macro invocation will not be performed right away by the `next` process, but at a later time by `prev` when the scheduler selects it again for execution. However, at that moment, the `prev` local variable does not point to our original

process that was to be replaced when we started the description of `schedule( )`, but rather to the process that was replaced by our original `prev` when it was scheduled again. (If you are confused, go back and read the section "[Performing the Process Switch](#)" in [Chapter 3](#).) The first instructions after a process switch are:

```
barrier( );
finish_task_switch(prev);
```

Right after the invocation of the `context_switch( )` function in `schedule( )`, the `barrier( )` macro yields an optimization barrier for the code (see the section "[Optimization and Memory Barriers](#)" in [Chapter 5](#)). Then, the `finish_task_switch( )` function is executed:

```
mm = this_rq( )->prev_mm;
this_rq( )->prev_mm = NULL;
prev_task_flags = prev->flags;
spin_unlock_irq(&this_rq( )->lock);
if (mm)
    mmdrop(mm);
if (prev_task_flags & PF_DEAD)
    put_task_struct(prev);
```

If `prev` is a kernel thread, the `prev_mm` field of the runqueue stores the address of the memory descriptor that was lent to `prev`. As we'll see in [Chapter 9](#), `mmdrop( )` decreases the usage counter of the memory descriptor; if the counter reaches 0 (likely because `prev` is a zombie process), the function also frees the descriptor together with the associated Page Tables and virtual memory regions.

The `finish_task_switch( )` function also releases the spin lock of the runqueue and enables the local interrupts. Then, it checks whether `prev` is a zombie task that is being removed from the system (see the section "[Process Termination](#)" in [Chapter 3](#)); if so, it invokes `put_task_struct( )` to free the process descriptor reference counter and drop all remaining references to the process (see the section "[Process Removal](#)" in [Chapter 3](#)).

The very last instructions of the `schedule( )` function are:

```
finish_schedule:

prev = current;
if (prev->lock_depth >= 0)
    __reacquire_kernel_lock( );
preempt_enable_no_resched();
if (test_bit(TIF_NEED_RESCHED, &current_thread_info( )->flags)
    goto need_resched;
return;
```

As you see, `schedule( )` reacquires the big kernel lock if necessary, reenables kernel preemption, and checks whether some other process has set the `TIF_NEED_RESCHED` flag of the current process. In this case, the whole `schedule( )` function is reexecuted from the beginning; otherwise, the function terminates.

Runqueue Balancing in Multiprocessor Systems

We have seen in [Chapter 4](#) that Linux sticks to the Symmetric Multiprocessing model (SMP); this means, essentially, that the kernel should not have any bias toward one CPU with respect to the others. However, multiprocessor machines come in many different flavors, and the scheduler behaves differently depending on the hardware characteristics. In particular, we will consider the following three types of multiprocessor machines:

Classic multiprocessor architecture

Until recently, this was the most common architecture for multiprocessor machines. These machines have a common set of RAM chips shared by all CPUs.

Hyper-threading

A hyper-threaded chip is a microprocessor that executes several threads of execution at once; it includes several copies of the internal registers and quickly switches between them. This technology, which was invented by Intel, allows the processor to exploit the machine cycles to execute another thread while the current thread is stalled for a memory access. A hyper-threaded physical CPU is seen by Linux as several different logical CPUs.

NUMA

CPUs and RAM chips are grouped in local "nodes" (usually a node includes one CPU and a few RAM chips). The memory arbiter (a special circuit that serializes the accesses to RAM performed by the CPUs in the system, see the section "[Memory Addresses](#)" in [Chapter 2](#)) is a bottleneck for the performance of the classic multiprocessor systems. In a NUMA architecture, when a CPU accesses a "local" RAM chip inside its own node, there is little or no contention, thus the access is usually fast; on the other hand, accessing a "remote" RAM chip outside of its node is much slower. We'll mention in the section "[Non-Uniform Memory Access \(NUMA\)](#)" in [Chapter 8](#) how the Linux kernel memory allocator supports NUMA architectures.

These basic kinds of multiprocessor systems are often combined. For instance, a motherboard that includes two different hyper-threaded CPUs is seen by the kernel as four logical CPUs.

As we have seen in the previous section, the `schedule( )` function picks the new process to run from the runqueue of the local CPU. Therefore, a given CPU can execute only the runnable processes that are contained in the corresponding runqueue. On the other hand, a runnable process is always stored in exactly one runqueue: no runnable process ever appears in two or more runqueues. Therefore, until a process remains runnable, it is usually bound to one CPU.

This design choice is usually beneficial for system performance, because the hardware cache of every CPU is likely to be filled with data owned by the runnable processes in the runqueue. In some cases, however, binding a runnable process to a given CPU might induce a severe performance penalty. For instance, consider a large number of batch processes that make heavy use of the CPU: if most of them end up in the same runqueue, one CPU in the system will be overloaded, while the others will be nearly idle.

Therefore, the kernel periodically checks whether the workloads of the runqueues are balanced and, if necessary, moves some process from one runqueue to another. However, to get the best performance from a multiprocessor system, the load balancing algorithm should take into consideration the topology of the CPUs in the system. Starting from kernel version 2.6.7, Linux sports a sophisticated runqueue balancing algorithm based on the notion of "scheduling domains." Thanks to the scheduling domains, the algorithm can be easily tuned for all kinds of existing multiprocessor architectures (and even for recent architectures such as those based on the "multi-core" microprocessors).

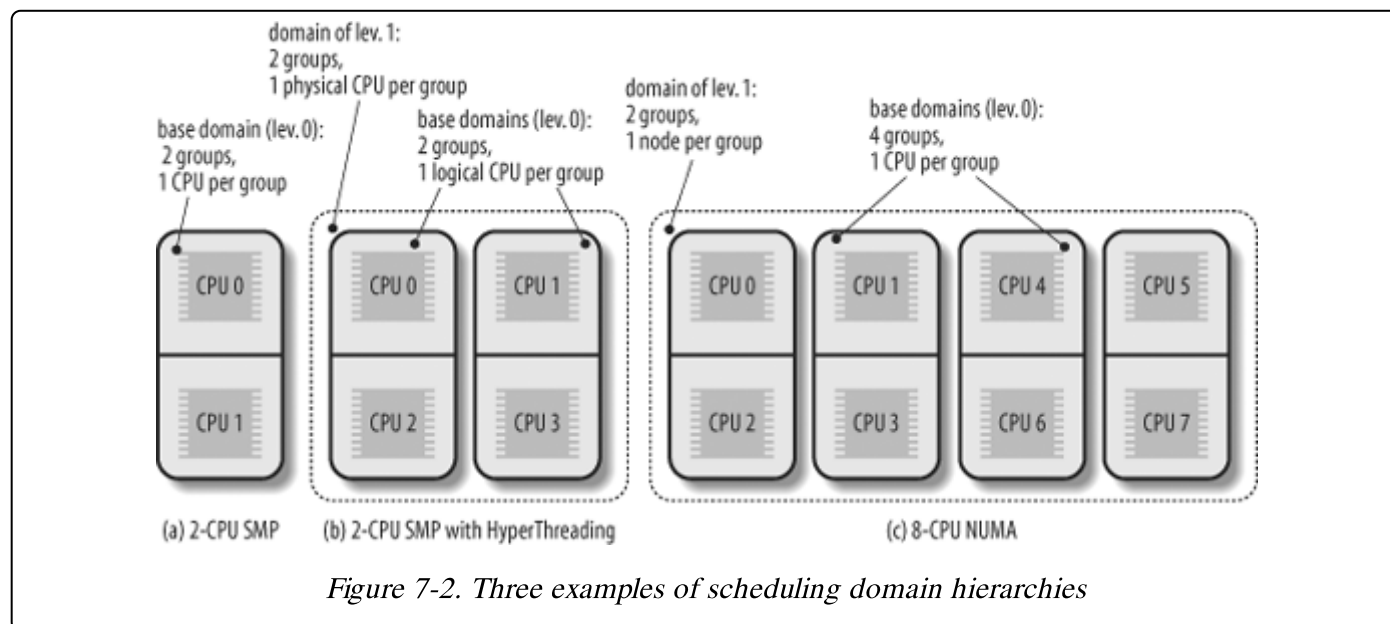
Scheduling Domains

Essentially, a *scheduling domain* is a set of CPUs whose workloads should be kept balanced by the kernel. Generally speaking, scheduling domains are hierarchically organized: the top-most scheduling domain, which usually spans all CPUs in the system, includes children scheduling domains, each of which include a subset of the CPUs. Thanks to the hierarchy of scheduling domains, workload balancing can be done in a rather efficient way.

Every scheduling domain is partitioned, in turn, in one or more *groups*, each of which represents a subset of the CPUs of the scheduling domain. Workload balancing is always done between groups of a scheduling domain. In other words, a process is moved from one CPU to another only if the total workload of some group in some

scheduling domain is significantly lower than the workload of another group in the same scheduling domain.

[Figure 7-2](#) illustrates three examples of scheduling domain hierarchies, corresponding to the three main architectures of multiprocessor machines.



[Figure 7-2](#) (a) represents a hierarchy composed by a single scheduling domain for a 2-CPU classic multiprocessor architecture. The scheduling domain includes only two groups, each of which includes one CPU.

[Figure 7-2](#) (b) represents a two-level hierarchy for a 2-CPU multiprocessor box with hyper-threading technology. The top-level scheduling domain spans all four logical CPUs in the system, and it is composed by two groups. Each group of the top-level domain corresponds to a child scheduling domain and spans a physical CPU. The bottom-level scheduling domains (also called *base scheduling domains*) include two groups, one for each logical CPU.

Finally, [Figure 7-2](#) (c) represents a two-level hierarchy for an 8-CPU NUMA architecture with two nodes and four CPUs per node. The top-level domain is organized in two groups, each of which corresponds to a different node. Every base scheduling domain spans the CPUs inside a single node and has four groups, each of which spans a single CPU.

Every scheduling domain is represented by a `sched_domain` descriptor, while every group inside a scheduling domain is represented by a `sched_group` descriptor. Each `sched_domain` descriptor includes a field `groups`, which points to the first element in

a list of group descriptors. Moreover, the `parent` field of the `sched_domain` structure points to the descriptor of the parent scheduling domain, if any.

The `sched_domain` descriptors of all physical CPUs in the system are stored in the per-CPU variable `phys_domains`. If the kernel does not support the hyper-threading technology, these domains are at the bottom level of the domain hierarchy, and the `sd` fields of the runqueue descriptors point to them—that is, they are the base scheduling domains. Conversely, if the kernel supports the hyper-threading technology, the bottom-level scheduling domains are stored in the per-CPU variable `cpu_domains`.

The `rebalance_tick( )` Function

To keep the runqueues in the system balanced, the `rebalance_tick( )` function is invoked by `scheduler_tick( )` once every tick. It receives as its parameters the index `this_cpu` of the local CPU, the address `this_rq` of the local runqueue, and a flag, `idle`, which can assume the following values:

`SCHED_IDLE`

The CPU is currently idle, that is, `current` is the *swapper* process.

`NOT_IDLE`

The CPU is not currently idle, that is, `current` is not the *swapper* process.

The `rebalance_tick( )` function determines first the number of processes in the runqueue and updates the runqueue's average workload; to do this, the function accesses the `nr_running` and `cpu_load` fields of the runqueue descriptor.

Then, `rebalance_tick( )` starts a loop over all scheduling domains in the path from the base domain (referenced by the `sd` field of the local runqueue descriptor) to the top-level domain. In each iteration the function determines whether the time has come to invoke the `load_balance( )` function, thus executing a rebalancing operation on the scheduling domain. The value of `idle` and some parameters stored in the `sched_domain` descriptor determine the frequency of the invocations of `load_balance( )`. If `idle` is equal to `SCHED_IDLE`, then the runqueue is empty, and `rebalance_tick( )` invokes `load_balance( )` quite often (roughly once every one or two ticks for scheduling domains corresponding to logical and physical CPUs). Conversely, if `idle` is equal to `NOT_IDLE`, `rebalance_tick( )` invokes `load_balance( )` sparingly (roughly once every 10 milliseconds for scheduling domains corresponding to logical CPUs, and once every 100 milliseconds for scheduling domains corresponding to physical CPUs).

The `load_balance( )` Function

The `load_balance( )` function checks whether a scheduling domain is significantly unbalanced; more precisely, it checks whether unbalancing can be reduced by moving some processes from the busiest group to the runqueue of the local CPU. If so, the function attempts this migration. It receives four parameters:

`this_cpu`

The index of the local CPU

`this_rq`

The address of the descriptor of the local runqueue

`sd`

Points to the descriptor of the scheduling domain to be checked

`idle`

Either `SCHED_IDLE` (local CPU is idle) or `NOT_IDLE`

The function performs the following operations:

1. Acquires the `this_rq->lock` spin lock.
2. Invokes the `find_busiest_group( )` function to analyze the workloads of the groups inside the scheduling domain. The function returns the address of the `sched_group` descriptor of the busiest group, provided that this group does not include the local CPU; in this case, the function also returns the number of processes to be moved into the local runqueue to restore balancing. On the other hand, if either the busiest group includes the local CPU or all groups are essentially balanced, the function returns `NULL`. This procedure is not trivial, because the function tries to filter the statistical fluctuations in the workloads.
3. If `find_busiest_group( )` did not find a group not including the local CPU that is significantly busier than the other groups in the scheduling domain, the function releases the `this_rq->lock` spin lock, tunes the parameters in the scheduling domain descriptor so as to delay the next invocation of `load_balance( )` on the local CPU, and terminates.
4. Invokes the `find_busiest_queue( )` function to find the busiest CPUs in the group found in step 2. The function returns the descriptor address `busiest` of the corresponding runqueue.

5. Acquires a second spin lock, namely the `busiest->lock` spin lock. To prevent deadlocks, this has to be done carefully: the `this_rq->lock` is first released, then the two locks are acquired by increasing CPU indices.
6. Invokes the `move_tasks( )` function to try moving some processes from the `busiest` runqueue to the local runqueue `this_rq` (see the next section).
7. If the `move_task( )` function failed in migrating some process to the local runqueue, the scheduling domain is still unbalanced. Sets to 1 the `busiest->active_balance` flag and wakes up the *migration* kernel thread whose descriptor is stored in `busiest->migration_thread`. The *migration* kernel thread walks the chain of the scheduling domain, from the base domain of the `busiest` runqueue to the top domain, looking for an idle CPU. If an idle CPU is found, the kernel thread invokes `move_tasks( )` to move one process into the idle runqueue.
8. Releases the `busiest->lock` and `this_rq->lock` spin locks.
9. Terminates.

The `move_tasks( )` Function

The `move_tasks( )` function moves processes from a source runqueue to the local runqueue. It receives six parameters: `this_rq` and `this_cpu` (the local runqueue descriptor and the local CPU index), `busiest` (the source runqueue descriptor), `max_nr_move` (the maximum number of processes to be moved), `sd` (the address of the scheduling domain descriptor in which this balancing operation is carried on), and the `idle` flag (beside `SCHED_IDLE` and `NOT_IDLE`, this flag can also be set to `NEWLY_IDLE` when the function is indirectly invoked by `idle_balance( )`; see the section "[The `schedule\( \)` Function](#)" earlier in this chapter).

The function first analyzes the expired processes of the `busiest` runqueue, starting from the higher priority ones. When all expired processes have been scanned, the function scans the active processes of the `busiest` runqueue. For each candidate process, the function invokes `can_migrate_task( )`, which returns 1 if all the following conditions hold:

- The process is not being currently executed by the remote CPU.
- The local CPU is included in the `cpus_allowed` bitmask of the process descriptor.
- At least one of the following holds:

- The local CPU is idle. If the kernel supports the hyper-threading technology, all logical CPUs in the local physical chip must be idle.
- The kernel is having trouble in balancing the scheduling domain, because repeated attempts to move processes have failed.
- The process to be moved is not "cache hot" (it has not recently executed on the remote CPU, so one can assume that no data of the process is included in the hardware cache of the remote CPU).

If `can_migrate_task( )` returns the value 1, `move_tasks( )` invokes the `pull_task( )` function to move the candidate process to the local runqueue. Essentially, `pull_task( )` executes `dequeue_task( )` to remove the process from the remote runqueue, then executes `enqueue_task( )` to insert the process in the local runqueue, and finally, if the process just moved has higher dynamic priority than current, invokes `resched_task( )` to preempt the current process of the local CPU.

System Calls Related to Scheduling

Several system calls have been introduced to allow processes to change their priorities and scheduling policies. As a general rule, users are always allowed to lower the priorities of their processes. However, if they want to modify the priorities of processes belonging to some other user or if they want to increase the priorities of their own processes, they must have superuser privileges.

The `nice( )` System Call

The `nice( )` ^[*] system call allows processes to change their base priority. The integer value contained in the `increment` parameter is used to modify the `nice` field of the process descriptor. The *nice* Unix command, which allows users to run programs with modified scheduling priority, is based on this system call.

The `sys_nice( )` service routine handles the `nice( )` system call. Although the `increment` parameter may have any value, absolute values larger than 40 are trimmed down to 40. Traditionally, negative values correspond to requests for priority increments and require superuser privileges, while positive ones correspond to requests for priority decreases. In the case of a negative increment, the function invokes the `capable( )` function to verify whether the process has a `CAP_SYS_NICE` capability. Moreover, the function invokes the `security_task_setnice( )` security hook. We discuss that function in [Chapter 20](#). If the user turns out to have the privilege required to change priorities, `sys_nice( )` converts `current->static_prio` to the range of nice values, adds the value of `increment`, and invokes the `set_user_nice( )` function. In turn, the latter function gets the local runqueue lock, updates the static priority of `current`, invokes the `resched_task( )` function to allow other processes to preempt `current`, and release the runqueue lock.

The `nice( )` system call is maintained for backward compatibility only; it has been replaced by the `setpriority( )` system call described next.

The `getpriority( )` and `setpriority( )` System Calls

The `nice( )` system call affects only the process that invokes it. Two other system calls, denoted as `getpriority( )` and `setpriority( )`, act on the base priorities of all processes in a given group. `getpriority( )` returns 20 minus the lowest `nice` field value among all processes in a given group—that is, the highest priority among

those processes; `setpriority( )` sets the base priority of all processes in a given group to a given value.

The kernel implements these system calls by means of the `sys_getpriority( )` and `sys_setpriority( )` service routines. Both of them act essentially on the same group of parameters:

`which`

The value that identifies the group of processes; it can assume one of the following:

`PRIO_PROCESS`

Selects the processes according to their process ID (`pid` field of the process descriptor).

`PRIO_PGRP`

Selects the processes according to their group ID (`pgrp` field of the process descriptor).

`PRIO_USER`

Selects the processes according to their user ID (`uid` field of the process descriptor).

`who`

The value of the `pid`, `pgrp`, or `uid` field (depending on the value of `which`) to be used for selecting the processes. If `who` is 0, its value is set to that of the corresponding field of the `current` process.

`niceval`

The new base priority value (needed only by `sys_setpriority( )`). It should range between - 20 (highest priority) and + 19 (lowest priority).

As stated before, only processes with a `CAP_SYS_NICE` capability are allowed to increase their own base priority or to modify that of other processes.

As we will see in [Chapter 10](#), system calls return a negative value only if some error occurred. For this reason, `getpriority( )` does not return a normal nice value ranging between - 20 and + 19, but rather a nonnegative value ranging between 1 and 40.

The `sched_getaffinity( )` and `sched_setaffinity( )` System Calls

The `sched_getaffinity( )` and `sched_setaffinity( )` system calls respectively return and set up the CPU affinity mask of a process—the bit mask of the CPUs that are allowed to execute the process. This mask is stored in the `cpus_allowed` field of the process descriptor.

The `sys_sched_getaffinity( )` system call service routine looks up the process descriptor by invoking `find_task_by_pid( )`, and then returns the value of the corresponding `cpus_allowed` field ANDed with the bitmap of the available CPUs.

The `sys_sched_setaffinity( )` system call is a bit more complicated. Besides looking for the descriptor of the target process and updating the `cpus_allowed` field, this function has to check whether the process is included in a runqueue of a CPU that is no longer present in the new affinity mask. In the worst case, the process has to be moved from one runqueue to another one. To avoid problems due to deadlocks and race conditions, this job is done by the *migration* kernel threads (there is one thread per CPU). Whenever a process has to be moved from a runqueue `rq1` to another runqueue `rq2`, the system call awakes the migration thread of `rq1` (`rq1->migration_thread`), which in turn removes the process from `rq1` and inserts it into `rq2`.

System Calls Related to Real-Time Processes

We now introduce a group of system calls that allow processes to change their scheduling discipline and, in particular, to become real-time processes. As usual, a process must have a `CAP_SYS_NICE` capability to modify the values of the `rt_priority` and `policy` process descriptor fields of any process, including itself.

The `sched_getscheduler( )` and `sched_setscheduler( )` system calls

The `sched_getscheduler( )` system call queries the scheduling policy currently applied to the process identified by the `pid` parameter. If `pid` equals 0, the policy of the calling process is retrieved. On success, the system call returns the policy for the process: `SCHED_FIFO`, `SCHED_RR`, or `SCHED_NORMAL` (the latter is also called `SCHED_OTHER`). The corresponding `sys_sched_getscheduler( )` service routine invokes `find_process_by_pid( )`, which locates the process descriptor corresponding to the given `pid` and returns the value of its `policy` field.

The `sched_setscheduler( )` system call sets both the scheduling policy and the associated parameters for the process identified by the parameter `pid`. If `pid` is equal to 0, the scheduler parameters of the calling process will be set.

The corresponding `sys_sched_setscheduler( )` system call service routine simply invokes `do_sched_setscheduler( )`. The latter function checks whether the scheduling policy specified by the `policy` parameter and the new priority specified by the `param->sched_priority` parameter are valid. It also checks whether the process has `CAP_SYS_NICE` capability or whether its owner has superuser rights. If everything is OK, it removes the process from its runqueue (if it is runnable); updates the static, real-time, and dynamic priorities of the process; inserts the process back in the runqueue; and finally invokes, if necessary, the `resched_task( )` function to preempt the current process of the runqueue.

The `sched_getparam( )` and `sched_setparam( )` system calls

The `sched_getparam( )` system call retrieves the scheduling parameters for the process identified by `pid`. If `pid` is 0, the parameters of the current process are retrieved. The corresponding `sys_sched_getparam( )` service routine, as one would expect, finds the process descriptor pointer associated with `pid`, stores its `rt_priority` field in a local variable of type `sched_param`, and invokes `copy_to_user( )` to copy it into the process address space at the address specified by the `param` parameter.

The `sched_setparam( )` system call is similar to `sched_setscheduler( )`. The difference is that `sched_setparam( )` does not let the caller set the `policy` field's value.^[*] The corresponding `sys_sched_setparam( )` service routine invokes `do_sched_setscheduler( )`, with almost the same parameters as `sys_sched_setscheduler( )`.

The `sched_yield( )` system call

The `sched_yield( )` system call allows a process to relinquish the CPU voluntarily without being suspended; the process remains in a `TASK_RUNNING` state, but the scheduler puts it either in the expired set of the runqueue (if the process is a conventional one), or at the end of the runqueue list (if the process is a real-time one). The `schedule( )` function is then invoked. In this way, other processes that have the same dynamic priority have a chance to run. The call is used mainly by `SCHED_FIFO` real-time processes.

The `sched_get_priority_min( )` and `sched_get_priority_max( )` system calls

The `sched_get_priority_min( )` and `sched_get_priority_max( )` system calls return, respectively, the minimum and the maximum real-time static priority value that can be used with the scheduling policy identified by the `policy` parameter.

The `sys_sched_get_priority_min( )` service routine returns 1 if `current` is a real-time process, 0 otherwise.

The `sys_sched_get_priority_max( )` service routine returns 99 (the highest priority) if `current` is a real-time process, 0 otherwise.

The `sched_rr_get_interval( )` system call

The `sched_rr_get_interval( )` system call writes into a structure stored in the User Mode address space the Round Robin time quantum for the real-time process identified by the `pid` parameter. If `pid` is zero, the system call writes the time quantum of the current process.

The corresponding `sys_sched_rr_get_interval( )` service routine invokes, as usual, `find_process_by_pid( )` to retrieve the process descriptor associated with `pid`. It then converts the base time quantum of the selected process into seconds and nanoseconds and copies the numbers into the User Mode structure. Conventionally, the time quantum of a FIFO real-time process is equal to zero.

[*] Because this system call is usually invoked to lower the priority of a process, users who invoke it for their processes are "nice" to other users.

[*] This anomaly is caused by a specific requirement of the POSIX standard.

Appendix A. System Startup

This appendix explains what happens right after users switch on their computers—that is, how a Linux kernel image is copied into memory and executed. In short, we discuss how the kernel, and thus the whole system, is "bootstrapped."

Traditionally, the term *bootstrap* refers to a person who tries to stand up by pulling his own boots. In operating systems, the term denotes bringing at least a portion of the operating system into main memory and having the processor execute it. It also denotes the initialization of kernel data structures, the creation of some user processes, and the transfer of control to one of them.

Computer bootstrapping is a tedious, long task, because initially, nearly every hardware device, including the RAM, is in a random, unpredictable state. Moreover, the bootstrap process is highly dependent on the computer architecture; as usual in this book, we refer to the 80×86 architecture.

Prehistoric Age: the BIOS

The moment after a computer is powered on, it is practically useless because the RAM chips contain random data and no operating system is running. To begin the boot, a special hardware circuit raises the logical value of the RESET pin of the CPU. After RESET is asserted, some registers of the processor (including `cs` and `eip`) are set to fixed values, and the code found at physical address `0xffffffff0` is executed. This address is mapped by the hardware to a certain read-only, persistent memory chip that is often called Read-Only Memory (ROM). The set of programs stored in ROM is traditionally called the *Basic Input/Output System (BIOS)* in the 80×86 architecture, because it includes several interrupt-driven low-level procedures used by all operating systems in the booting phase to handle the hardware devices that make up the computer. Some operating systems, such as Microsoft's MS-DOS, rely on BIOS to implement most system calls.

Once in protected mode (see the section "[Segmentation in Hardware](#)" in [Chapter 2](#)), Linux does not use BIOS any longer, but it provides its own device driver for every hardware device on the computer. In fact, the BIOS procedures must be executed in real mode, so they cannot share functions even if that would be beneficial.

The BIOS uses Real Mode addresses because they are the only ones available when the computer is turned on. A Real Mode address is composed of a *seg* segment and an *off* offset; the corresponding physical address is given by $seg * 16 + off$. As a result, no Global Descriptor Table, Local Descriptor Table, or paging table is needed by the CPU addressing circuit to translate a logical address into a physical one. Clearly, the code that initializes the GDT, LDT, and paging tables must run in Real Mode.

Linux is forced to use BIOS in the bootstrapping phase, when it must retrieve the kernel image from disk or from some other external device. The BIOS bootstrap procedure essentially performs the following four operations:

1. Executes a series of tests on the computer hardware to establish which devices are present and whether they are working properly. This phase is often called *Power-On Self-Test (POST)*. During this phase, several messages, such as the BIOS version banner, are displayed.

Recent 80×86 , AMD64, and Itanium computers make use of the *Advanced Configuration and Power Interface (ACPI)* standard. The bootstrap code in an ACPI-compliant BIOS builds several tables that describe the hardware devices

present in the system. These tables have a vendor-independent format and can be read by the operating system kernel to learn how to handle the devices.

2. Initializes the hardware devices. This phase is crucial in modern PCI-based architectures, because it guarantees that all hardware devices operate without conflicts on the IRQ lines and I/O ports. At the end of this phase, a table of installed PCI devices is displayed.
3. Searches for an operating system to boot. Actually, depending on the BIOS setting, the procedure may try to access (in a predefined, customizable order) the first sector (*boot sector*) of every floppy disk, hard disk, and CD-ROM in the system.
4. As soon as a valid device is found, it copies the contents of its first sector into RAM, starting from physical address `0x00007c00`, and then jumps into that address and executes the code just loaded.

The rest of this appendix takes you from the most primitive starting state to the full glory of a running Linux system.

Ancient Age: the Boot Loader

The *boot loader* is the program invoked by the BIOS to load the image of an operating system kernel into RAM. Let's briefly sketch how boot loaders work in IBM's PC architecture.

To boot from a floppy disk, the instructions stored in its first sector are loaded in RAM and executed; these instructions copy all the remaining sectors containing the kernel image into RAM.

Booting from a hard disk is done differently. The first sector of the hard disk, named the *Master Boot Record (MBR)*, includes the partition table^[*] and a small program, which loads the first sector of the partition containing the operating system to be started. Some operating systems, such as Microsoft Windows 98, identify this partition by means of an *active* flag included in the partition table;^[‡] following this approach, only the operating system whose kernel image is stored in the active partition can be booted. As we will see later, Linux is more flexible because it replaces the rudimentary program included in the MBR with a sophisticated program—the "boot loader"—that allows users to select the operating system to be booted.

Kernel images of earlier Linux versions—up to the 2.4 series—included a minimal "boot loader" program in the first 512 bytes; thus, copying a kernel image starting from the first sector made the floppy bootable. On the other hand, kernel images of Linux 2.6 no longer include such boot loader; thus, in order to boot from floppy disk, a suitable boot loader has to be stored in the first disk sector. Nowadays, booting from a floppy is very similar to booting from a hard disk or from a CD-ROM.

Booting Linux from a Disk

A two-stage boot loader is required to boot a Linux kernel from disk. A well-known Linux boot loader on 80 × 86 systems is named LInux LOader (LILO). Other boot loaders for 80 × 86 systems do exist; for instance, the GRand Unified Bootloader (GRUB) is also widely used. GRUB is more advanced than LILO, because it recognizes several disk-based filesystems and is thus capable of reading portions of the boot program from files. Of course, specific boot loader programs exist for all architectures supported by Linux.

LILO may be installed either on the MBR (replacing the small program that loads the boot sector of the active partition) or in the boot sector of every disk partition. In both

cases, the final result is the same: when the loader is executed at boot time, the user may choose which operating system to load.

Actually, the LILO boot loader is too large to fit into a single sector, thus it is broken into two parts. The MBR or the partition boot sector includes a small boot loader, which is loaded into RAM starting from address `0x00007c00` by the BIOS. This small program moves itself to the address `0x00096a00`, sets up the Real Mode stack (ranging from `0x00098000` to `0x000969ff`), loads the second part of the LILO boot loader into RAM starting from address `0x00096c00`, and jumps into it.

In turn, this latter program reads a map of bootable operating systems from disk and offers the user a prompt so she can choose one of them. Finally, after the user has chosen the kernel to be loaded (or let a time-out elapse so that LILO chooses a default), the boot loader may either copy the boot sector of the corresponding partition into RAM and execute it or directly copy the kernel image into RAM.

Assuming that a Linux kernel image must be booted, the LILO boot loader, which relies on BIOS routines, performs essentially the following operations:

1. Invokes a BIOS procedure to display a "Loading" message.
2. Invokes a BIOS procedure to load an initial portion of the kernel image from disk: the first 512 bytes of the kernel image are put in RAM at address `0x00090000`, while the code of the `setup( )` function (see below) is put in RAM starting from address `0x00090200`.
3. Invokes a BIOS procedure to load the rest of the kernel image from disk and puts the image in RAM starting from either low address `0x00010000` (for small kernel images compiled with `make zImage`) or high address `0x00100000` (for big kernel images compiled with `make bzImage`). In the following discussion, we say that the kernel image is "loaded low" or "loaded high" in RAM, respectively. Support for big kernel images uses essentially the same booting scheme as the other one, but it places data in different physical memory addresses to avoid problems with the ISA hole mentioned in the section "[Physical Memory Layout](#)" in [Chapter 2](#).
4. Jumps to the `setup( )` code.

[*] Each partition table entry typically includes the starting and ending sectors of a partition and the kind of operating system that handles it.

 The active flag may be set through programs such as *fdisk*.

Middle Ages: the `setup( )` Function

The code of the `setup( )` assembly language function has been placed by the linker at offset `0x200` of the kernel image file. The boot loader can therefore easily locate the code and copy it into RAM, starting from physical address `0x00090200`.

The `setup( )` function must initialize the hardware devices in the computer and set up the environment for the execution of the kernel program. Although the BIOS already initialized most hardware devices, Linux does not rely on it, but reinitializes the devices in its own manner to enhance portability and robustness. `setup( )` performs essentially the following operations:

1. In ACPI -compliant systems, it invokes a BIOS routine that builds a table in RAM describing the layout of the system's physical memory (the table can be seen in the boot kernel messages by looking for the "BIOS-e820" label). In older systems, it invokes a BIOS routine that just returns the amount of RAM available in the system.
2. Sets the keyboard repeat delay and rate. (When the user keeps a key pressed past a certain amount of time, the keyboard device sends the corresponding keycode over and over to the CPU.)
3. Initializes the video adapter card.
4. Reinitializes the disk controller and determines the hard disk parameters.
5. Checks for an IBM Micro Channel bus (MCA).
6. Checks for a PS/2 pointing device (bus mouse).
7. Checks for Advanced Power Management (APM) BIOS support.
8. If the BIOS supports the *Enhanced Disk Drive Services (EDD)*, it invokes the proper BIOS procedure to build a table in RAM describing the hard disks available in the system. (The information included in the table can be seen by reading the files in the *firmware/edd* directory of the *sysfs* special filesystem.)
9. If the kernel image was loaded low in RAM (at physical address `0x00010000`), the function moves it to physical address `0x00001000`. Conversely, if the kernel image was loaded high in RAM, the function does not move it. This step is necessary because to be able to store the kernel image on a floppy disk and to reduce the booting time, the kernel image stored on disk is compressed, and the

decompression routine needs some free space to use as a temporary buffer following the kernel image in RAM.

10. Sets the A20 pin located on the 8042 keyboard controller. The A20 pin is a hack introduced in the 80286 -based systems to make physical addresses compatible with those of the ancient 8088 microprocessors. Unfortunately, the A20 pin must be properly set before switching to Protected Mode, otherwise the 21st bit of every physical address will always be regarded as zero by the CPU. Setting the A20 pin is a messy operation.
11. Sets up a provisional Interrupt Descriptor Table (IDT) and a provisional Global Descriptor Table (GDT).
12. Resets the floating-point unit (FPU), if any.
13. Reprograms the Programmable Interrupt Controllers (PIC) to mask all interrupts, except IRQ2 which is the cascading interrupt between the two PICs.
14. Switches the CPU from Real Mode to Protected Mode by setting the PE bit in the cr0 status register. The PG bit in the cr0 register is cleared, so paging is still disabled.
15. Jumps to the `startup_32( )` assembly language function.

Renaissance: the `startup_32( )` Functions

There are two different `startup_32( )` functions; the one we refer to here is coded in the *arch/i386/boot/compressed/head.S* file. After `setup( )` terminates, the function has been moved either to physical address `0x00100000` or to physical address `0x00001000`, depending on whether the kernel image was loaded high or low in RAM.

This function performs the following operations:

1. Initializes the segmentation registers and a provisional stack.
2. Clears all bits in the `eflags` register.
3. Fills the area of uninitialized data of the kernel identified by the `_edata` and `_end` symbols with zeros (see the section "[Physical Memory Layout](#)" in [Chapter 2](#)).
4. Invokes the `decompress_kernel( )` function to decompress the kernel image. The "Uncompressing Linux..." message is displayed first. After the kernel image is decompressed, the "O K, booting the kernel." message is shown. If the kernel image was loaded low, the decompressed kernel is placed at physical address `0x00100000`. Otherwise, if the kernel image was loaded high, the decompressed kernel is placed in a temporary buffer located after the compressed image. The decompressed image is then moved into its final position, which starts at physical address `0x00100000`.
5. Jumps to physical address `0x00100000`.

The decompressed kernel image begins with another `startup_32( )` function included in the *arch/i386/kernel/head.S* file. Using the same name for both the functions does not create any problems (besides confusing our readers), because both functions are executed by jumping to their initial physical addresses.

The second `startup_32( )` function sets up the execution environment for the first Linux process (process 0). The function performs the following operations:

1. Initializes the segmentation registers with their final values.
2. Fills the bss segment of the kernel (see the section "[Program Segments and Process Memory Regions](#)" in [Chapter 20](#)) with zeros.
3. Initializes the provisional kernel Page Tables contained in `swapper_pg_dir` and `pg0` to identically map the linear addresses to the same physical addresses, as

explained in the section "[Kernel Page Tables](#)" in [Chapter 2](#).

4. Stores the address of the Page Global Directory in the `cr3` register, and enables paging by setting the PG bit in the `cr0` register.
5. Sets up the Kernel Mode stack for process 0 (see the section "[Kernel Threads](#)" in [Chapter 3](#)).
6. Once again, the function clears all bits in the `eflags` register.
7. Invokes `setup_idt( )` to fill the IDT with null interrupt handlers (see the section "[Preliminary Initialization of the IDT](#)" in [Chapter 4](#)).
8. Puts the system parameters obtained from the BIOS and the parameters passed to the operating system into the first page frame (see the section "[Physical Memory Layout](#)" in [Chapter 2](#)).
9. Identifies the model of the processor.
10. Loads the `gdtr` and `idtr` registers with the addresses of the GDT and IDT tables.
11. Jumps to the `start_kernel( )` function.

Modern Age: the `start_kernel( )` Function

The `start_kernel( )` function completes the initialization of the Linux kernel. Nearly every kernel component is initialized by this function; we mention just a few of them:

- The scheduler is initialized by invoking the `sched_init( )` function (see [Chapter 7](#)).
- The memory zones are initialized by invoking the `build_all_zonelists( )` function (see the section "[Memory Zones](#)" in [Chapter 8](#)).
- The Buddy system allocators are initialized by invoking the `page_alloc_init( )` and `mem_init( )` functions (see the section "[The Buddy System Algorithm](#)" in [Chapter 8](#)).
- The final initialization of the IDT is performed by invoking `trap_init( )` (see the section "[Exception Handling](#)" in [Chapter 4](#)) and `init_IRQ( )` (see the section "[IRQ data structures](#)" in [Chapter 4](#)).
- The `TASKLET_SOFTIRQ` and `HI_SOFTIRQ` are initialized by invoking the `softirq_init( )` function (see the section "[Softirqs](#)" in [Chapter 4](#)).
- The system date and time are initialized by the `time_init( )` function (see the section "[The Linux Timekeeping Architecture](#)" in [Chapter 6](#)).
- The slab allocator is initialized by the `kmem_cache_init( )` function (see the section "[General and Specific Caches](#)" in [Chapter 8](#)).
- The speed of the CPU clock is determined by invoking the `calibrate_delay( )` function (see the section "[Delay Functions](#)" in [Chapter 6](#)).
- The kernel thread for process 1 is created by invoking the `kernel_thread( )` function. In turn, this kernel thread creates the other kernel threads and executes the `/sbin/init` program, as described in the section "[Kernel Threads](#)" in [Chapter 3](#).

Besides the "Linux version 2.6.11..." message, which is displayed right after the beginning of `start_kernel( )`, many other messages are displayed in this last phase, both by the `init` program and by the kernel threads. At the end, the familiar login prompt appears on the console (or in the graphical screen, if the X Window System is launched at startup), telling the user that the Linux kernel is up and running.